UNIVERSIDAD AUTÓNOMA “GABRIEL RENÉ MORENO”

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

SISTEMA WEB PARA LA GESTIÓN DE CITAS MÉDICAS PARA LA CLÍNICA “VIDA SALUD”

Materia:	Sistemas de Información I
Docente:	MSc. Ing. Angélica Gazón Cuéllar
Grupo:	12
Integrantes:	• Cervantes Dueñas Juan Clider 222007907 • Cusi Mamani Benjamin 222008199
Tecnologías:	PHP 8 · PostgreSQL · HTML5 · CSS3 · JavaScript
Presentación:	Ciclo #1 + Ciclo #2

URL

github

Santa Cruz de la Sierra – Bolivia
Noviembre de 2026

TABLA DE CONTENIDO

1 PERFIL 6
1.1 Introducción 6
1.2 Antecedentes 6
1.3 Justificación 7
1.4 Descripción del problema 7
1.4.1 Situación actual de la programación de citas 7
1.4.2 Problemas en el registro de pacientes 8
1.4.3 Problemas en el control de cancelaciones e inasistencias 8
1.4.4 Problemas en la consulta de información y reportes 8
1.5 Formulación del problema 8
1.6 Objetivos 9
1.6.1 Objetivo General 9
1.6.2 Objetivos Específicos 9
1.7 Alcance 9
2 ELEMENTOS DEL SISTEMA BASADO EN COMPUTADORAS 12
2.1 HARDWARE 12
2.1.1 Servidor 12
2.1.2 Cliente 12
2.1.3 Medios de Comunicación 12
2.1.4 Otros Dispositivos 12
2.2 SOFTWARE 13
2.2.1 Servidor 13
2.2.2 Cliente 13
2.2.3 Otro software adicional 13
2.3 DATOS 13
2.4 PROCESOS 14
2.5 GENTE/USUARIO 15
2.6 DOCUMENTO 16
3 TECNOLOGÍA PARA EL DESARROLLO DEL SOFTWARE 17
3.1 Estrategia para el desarrollo del software 17
3.2 Metodología para el desarrollo del software 17
3.2.1 Características del PUDS 17
3.2.2 Características de UML 17
3.3 HERRAMIENTAS DE DESARROLLO 18
3.3.1 Software 18
3.3.2 Hardware 18

4 POSIBLES COSTOS 19
5 POSIBLES BENEFICIOS 20
5.1 TIEMPO 20
5.2 ESFUERZO 20
5.3 COSTOS 20
6 POSIBLES CLIENTES 21
7 MODELO DE DOMINIO 22
8 MODELO DE NEGOCIO 23
8.1 Diagrama de actividades 23
Proceso actual de programación de citas 23
Proceso propuesto de programación de citas 24
Proceso de atención de la cita y control de asistencia 24
Reglas del negocio 25
9 MARCO TEÓRICO 26
9.1 Sistemas de información 26
9.2 Diagrama de Ishikawa 26
9.3 Proceso Unificado de Desarrollo de Software (PUDS) 26
9.4 Flujos de trabajo del PUDS 26
9.5 Lenguaje Unificado de Modelado (UML) 26
9.6 Clases de análisis: interfaz, control y entidad 26
9.7 Arquitectura en capas y patrón MVC 27
9.8 PHP y PDO 27
9.9 PostgreSQL 27
9.10 Bases de datos relacionales y normalización 27
9.11 Seguridad en aplicaciones web 27
CAPÍTULO 1: MÉTODO DE ISHIKAWA 28
1) IDENTIFICAR PROBLEMA 28
1.1) LISTA DE PROBLEMAS 28
1.2) DEPURAR PROBLEMAS 28
1.3) LISTA FINAL DE PROBLEMAS 29
1.4) PROPIETARIOS DE PROBLEMAS 29
1.5) ANÁLISIS DE PROBLEMAS 30
1.6) ESTIMACIÓN Y CUANTIFICACIÓN DE PROBLEMA 31
1.7) ALTERNATIVAS DE CAMBIO 32
1.8) CONCLUSIÓN Y RECOMENDACIÓN 33
1.9) DISEÑAR EL DIAGRAMA DE ISHIKAWA 34
2) IDENTIFICAR LAS PRINCIPALES CATEGORÍAS 34
3) IDENTIFICAR LAS CAUSAS 34

4) ANALIZAR Y DISCUTIR EL DIAGRAMA 35
CAPÍTULO 2: FLUJO DE TRABAJO: CAPTURA DE REQUISITOS 36
2.1 Identificar actores y casos de uso 36
2.1.1 Actores 36
2.1.2 Casos de uso 36
2.2 Priorizar casos de uso 37
2.3 Detallar casos de uso 38
CU01: Iniciar y cerrar sesión 38
CU02: Gestionar usuarios 39
CU03: Gestionar roles y permisos 40
CU04: Gestionar pacientes 41
CU05: Gestionar médicos 42
CU06: Gestionar especialidades 42
CU07: Gestionar horarios de atención 43
CU08: Consultar disponibilidad de citas 44
CU09: Programar cita médica 45
CICLO # 2 45
CU10: Reprogramar cita 45
CU11: Cancelar cita 46
CU12: Registrar asistencia de la cita 47
CU13: Consultar agenda médica 48
CU14: Consultar historial de citas del paciente 49
CU15: Generar reportes 50
CU16: Gestionar perfil del paciente 51
CU17: Reservar y dar seguimiento a mis citas 51
2.4 Prototipar la interfaz de usuario 52
Ciclo #1 53
Ciclo #2 58
2.5 Estructurar el modelo de casos de uso 61
Ciclo #1: Seguridad, registros base, horarios y programación de citas 61
Ciclo #2: Gestión de citas, consultas, reportes y portal del paciente 61
Escenario completo 63
CAPÍTULO 3: FLUJO DE TRABAJO: ANÁLISIS 64
3.1 Análisis de arquitectura 64
3.1.1 Identificar paquetes 64
3.1.2 Relacionar paquetes y casos de uso 65
3.1.3 Vista de casos de uso 67
3.2 Analizar casos de uso 70

Ciclo #1 70
Ciclo #2 73
3.3 Análisis de clases 75
Ciclo #1 77
Ciclo #2 80
3.4 Analizar paquetes 83
3.4.1 Acoplamiento y cohesión 84
CAPÍTULO 4: FLUJO DE TRABAJO: DISEÑO 86
4.1 Diseño de la arquitectura 86
4.1.1 Diseño físico de la arquitectura 86
4.1.2 Diseño lógico de la arquitectura 86
4.2 Diseñar casos de uso 87
Ciclo #1 88
Ciclo #2 96
4.3 Diseño de datos 102
4.3.1 Diseño lógico de datos 102
4.3.2 Diseño físico de datos 105
CAPÍTULO 5: FLUJO DE TRABAJO: IMPLEMENTACIÓN 117
5.1 Elección de la plataforma de desarrollo del software 117
5.2 Implementación de la arquitectura del sistema 117
Código fuente principal 119
5.3 Implementación de la arquitectura de los subsistemas 119
CAPÍTULO 6: FLUJO DE TRABAJO: PRUEBAS 123
6.1 Casos de prueba 123
6.2 Pruebas de la base de datos 124
CONCLUSIÓN 125
RECOMENDACIÓN 125
BIBLIOGRAFÍA 125
REPOSITORIO DEL CÓDIGO FUENTE 126
ANEXOS 127
Anexo A. Glosario 127

1 PERFIL

1.1 Introducción

El presente proyecto se enmarca en las Tecnologías de la Información aplicadas a la gestión de servicios de salud. El caso de estudio corresponde a la Clínica Vida Salud, un establecimiento médico privado (ficticio) que brinda atención en distintas especialidades, como medicina general, pediatría, ginecología y odontología. Su recepción es responsable de registrar a los pacientes, consultar la disponibilidad de los médicos, programar las citas y comunicar a los pacientes la fecha y hora de su atención.

Actualmente la clínica administra sus citas mediante cuadernos, llamadas telefónicas y hojas de cálculo. Este procedimiento dificulta organizar los horarios de los médicos, controlar las citas programadas y mantener actualizada la información de los pacientes; como consecuencia se producen citas duplicadas para un mismo médico y horario, pérdida de datos, falta de control de cancelaciones e inasistencias, y demoras para obtener el historial de un paciente o elaborar reportes.

Ante esta situación, la dirección de la clínica decidió implementar un sistema web que centralice la información y automatice los principales procesos de la gestión de citas. El sistema se desarrolla con PHP como lenguaje del lado del servidor y PostgreSQL como sistema gestor de base de datos, siguiendo el Proceso Unificado de Desarrollo de Software (PUDS) y modelando sus artefactos con el Lenguaje Unificado de Modelado (UML). El desarrollo se organiza en dos ciclos iterativos e incrementales: el Ciclo #1 cubre la seguridad, los registros base, los horarios y la programación de citas; el Ciclo #2 completa la reprogramación, cancelación, control de asistencia, agenda, historial, reportes y el portal del paciente.

1.2 Antecedentes

La Clínica Vida Salud es una clínica privada de atención ambulatoria ubicada en la ciudad de Santa Cruz de la Sierra. Ofrece consultas en medicina general, pediatría, ginecología, odontología y cardiología, atendidas por médicos que trabajan en horarios semanales distintos según su especialidad. La atención al público está a cargo del personal de recepción, que coordina las citas de todos los médicos, y la dirección de la clínica supervisa la operación y toma las decisiones sobre horarios y personal.

Desde su apertura, la clínica ha registrado sus citas de forma tradicional: un cuaderno de citas por día en recepción, llamadas telefónicas para reservar o cancelar y hojas de cálculo para llevar la lista de pacientes. Con el crecimiento del número de pacientes y de especialidades, este procedimiento se volvió insuficiente: la información está dispersa en varios soportes, solo una persona puede usar el cuaderno a la vez y no existe un registro histórico confiable de las atenciones. Por ello, la dirección decidió implementar un sistema web que digitalice la gestión de citas.

1.3 Justificación

La gestión manual de las citas limita la capacidad de atención de la clínica y afecta directamente la experiencia del paciente. Cada cita duplicada obliga a reprogramar a un paciente que ya se trasladó a la clínica; cada cancelación no registrada deja un horario sin usar; y cada búsqueda en el cuaderno retrasa la atención en recepción.

La implementación de un sistema web de gestión de citas se justifica porque permitirá:

- Centralizar la información de pacientes, médicos, especialidades, horarios y citas en una única base de datos.
- Eliminar las citas duplicadas o superpuestas mediante la verificación automática de la disponibilidad en el servidor y en la base de datos.
- Conocer en todo momento los horarios disponibles de cada médico, calculados a partir de su horario de atención y de las citas ocupadas.
- Controlar las cancelaciones y las inasistencias, liberando los horarios y conservando el historial de cada cita.
- Consultar en segundos el historial de citas de un paciente y la agenda diaria de cada médico.
- Obtener reportes administrativos confiables por fecha, médico, especialidad y estado, sin conteos manuales.
- Proteger la información de los pacientes mediante control de acceso por roles, contraseñas cifradas con hash y trazabilidad de las operaciones.
- Ofrecer a los pacientes un portal para gestionar su perfil, reservar citas y dar seguimiento a su estado.

En conclusión, el sistema mejorará la organización interna de la clínica, reducirá los errores en la programación de citas, disminuirá los tiempos de atención y proporcionará a la dirección información actualizada para la toma de decisiones.

1.4 Descripción del problema

En la Clínica Vida Salud, la recepcionista atiende personalmente o por teléfono las solicitudes de cita. Para cada solicitud busca al paciente en el cuaderno o en la hoja de cálculo, consulta de forma verbal o en papel el horario del médico, anota la cita y comunica la fecha y hora al paciente. El médico lleva su propio registro de atenciones y la administración cuenta manualmente las citas cuando necesita información.

1.4.1 Situación actual de la programación de citas

El proceso inicia cuando el paciente solicita una cita en recepción o por teléfono. La recepcionista revisa el cuaderno del día solicitado para encontrar un espacio libre del médico; como el horario de cada médico no está escrito en un lugar único, depende de su memoria o de preguntar al médico. Si dos personas atienden a la vez —una en el mostrador y otra por teléfono— pueden anotar dos pacientes en el mismo horario, lo que se descubre recién cuando ambos llegan a la clínica.

1.4.2 Problemas en el registro de pacientes

Los datos de los pacientes se anotan a mano y se repiten en cada cita. No existe un control del documento de identidad, por lo que un mismo paciente puede estar registrado varias veces con datos distintos, y la letra ilegible provoca errores en nombres y teléfonos que impiden contactar al paciente para confirmar o cancelar su cita.

1.4.3 Problemas en el control de cancelaciones e inasistencias

Cuando un paciente cancela, a veces la cita no se tacha en el cuaderno y el horario sigue apareciendo ocupado. Cuando un paciente no se presenta, no queda ningún registro, por lo que la clínica no puede identificar a los pacientes que faltan con frecuencia ni medir cuántos horarios se pierden.

1.4.4 Problemas en la consulta de información y reportes

Para conocer el historial de citas de un paciente es necesario revisar cuadernos de varios meses. Los reportes de atenciones por médico o por especialidad se elaboran contando a mano, demoran horas y contienen errores; en consecuencia, la dirección no dispone de información actualizada para organizar los horarios de los médicos ni para planificar el personal.

Tabla 1. *Resumen de los problemas identificados en la gestión actual de citas*

N.º	Problema	Consecuencia
1	Duplicación de citas para un mismo médico y horario.	Pacientes que esperan sin ser atendidos; reprogramaciones de último momento.
2	Pérdida o registro incorrecto de los datos de los pacientes.	Pacientes registrados dos veces, teléfonos erróneos, imposibilidad de contactarlos.
3	Dificultad para conocer los horarios disponibles de los médicos.	Llamadas largas y horarios ofrecidos que en realidad están ocupados.
4	Falta de control sobre las citas canceladas y las inasistencias.	Horarios bloqueados innecesariamente; no se identifica a los pacientes que no asisten.
5	Demoras en la búsqueda del historial de citas de un paciente.	Atención lenta; el médico no conoce las consultas anteriores.
6	Dificultad para obtener reportes de las atenciones programadas.	La administración invierte horas en conteos manuales con errores.
7	Falta de información actualizada para la administración.	Decisiones sobre horarios y personal sin datos confiables.

1.5 Formulación del problema

La Clínica Vida Salud carece de un sistema de información centralizado para la gestión de sus citas médicas. El registro manual en cuadernos, llamadas telefónicas y hojas de cálculo provoca citas duplicadas, datos de pacientes incorrectos, desconocimiento de la disponibilidad real de los médicos, falta de control de cancelaciones e inasistencias, demoras en la consulta del historial y reportes poco confiables, lo que afecta la atención de los pacientes y la toma de decisiones de la dirección.

Pregunta de investigación: *¿Cómo desarrollar un sistema web utilizando PHP y PostgreSQL que permita administrar pacientes, médicos, especialidades y citas médicas, evitando conflictos de horarios y facilitando el control de las atenciones de la Clínica Vida Salud?*

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un sistema web de gestión de citas médicas para la Clínica Vida Salud utilizando PHP como lenguaje de programación del lado del servidor y PostgreSQL como sistema gestor de base de datos, con el propósito de mejorar el registro, la programación, el seguimiento y el control de las citas médicas.

1.6.2 Objetivos Específicos

- Analizar los procesos de registro y programación de citas de la clínica mediante el método de Ishikawa y el modelo de negocio.
- Capturar los requisitos funcionales como casos de uso, priorizarlos y distribuirlos en dos ciclos de desarrollo.
- Diseñar una base de datos relacional normalizada, con tablas relacionadas mediante claves primarias y foráneas.
- Implementar formularios para el registro, consulta, modificación y desactivación de pacientes y médicos.
- Gestionar las especialidades médicas y los horarios de atención de cada médico.
- Programar citas verificando la disponibilidad del médico en el servidor e impidiendo citas superpuestas.
- Controlar los estados de las citas (Programada, Confirmada, Atendida, Cancelada e Inasistencia) y sus transiciones válidas.
- Generar reportes de citas por fecha, médico, especialidad y estado.
- Aplicar validaciones de datos en el cliente y en el servidor, y mecanismos básicos de seguridad (hash de contraseñas, sesiones, consultas parametrizadas).
- Realizar pruebas para verificar el correcto funcionamiento del sistema.

1.7 Alcance

El sistema será una aplicación web accesible desde un navegador, desarrollada con PHP, PostgreSQL, HTML5 y CSS3, con JavaScript para mejorar la interacción de los formularios. Está destinado principalmente al personal administrativo de la clínica (administrador y recepcionista), a los médicos y, mediante un portal, a los pacientes. Se organiza en seis paquetes:

P1 Seguridad: Acceso, usuarios, roles, permisos y bitácora. Casos de uso: CU01 Iniciar y cerrar sesión; CU02 Gestionar usuarios; CU03 Gestionar roles y permisos.

P2 Personas: Pacientes, médicos y especialidades. Casos de uso: CU04 Gestionar pacientes; CU05 Gestionar médicos; CU06 Gestionar especialidades.

P3 Agenda: Horarios de atención y disponibilidad. Casos de uso: CU07 Gestionar horarios de atención; CU08 Consultar disponibilidad de citas.

P4 Citas: Programación, reprogramación, cancelación, asistencia, agenda e historial. Casos de uso: CU09 Programar cita médica; CU10 Reprogramar cita; CU11 Cancelar cita; CU12 Registrar asistencia de la cita; CU13 Consultar agenda médica; CU14 Consultar historial de citas del paciente.

P5 Reportes: Reportes administrativos de citas y pacientes. Casos de uso: CU15 Generar reportes.

P6 Portal del paciente: Perfil del paciente y reserva en línea. Casos de uso: CU16 Gestionar perfil del paciente; CU17 Reservar y dar seguimiento a mis citas.

El sistema SÍ permitirá: registrar y consultar pacientes; registrar médicos y sus especialidades; definir horarios de atención; consultar la disponibilidad de citas; registrar, reprogramar y cancelar citas; registrar la asistencia o inasistencia del paciente; consultar el historial de citas; generar reportes administrativos; administrar usuarios y permisos básicos; y que el paciente gestione su perfil y reserve sus citas.

Fuera del alcance: facturación y cobros, historia clínica electrónica completa (diagnósticos, recetas, resultados de laboratorio), telemedicina por video, integración con seguros médicos y envío de mensajes SMS/WhatsApp. Los recordatorios se muestran dentro del portal del paciente; el envío automático por correo electrónico queda como mejora futura.

Requisitos funcionales

Tabla 2. *Requisitos funcionales*

Código	Requisito	CU
RF01	Inicio de sesión y control de acceso: validar usuario/correo y contraseña (password_verify), iniciar sesión segura, identificar el rol, mostrar el menú según permisos, cerrar sesión e impedir el acceso sin sesión válida.	CU01, CU02, CU03
RF02	Gestión de pacientes: registrar, listar, buscar por CI/nombre/apellido, modificar, consultar sus citas anteriores y desactivar sin eliminar el historial; CI no duplicado.	CU04
RF03	Gestión de médicos: registrar, listar, buscar por nombre o especialidad, modificar, consultar su agenda y activar/desactivar.	CU05
RF04	Gestión de especialidades con identificador, nombre, descripción opcional y estado; sin nombres normalizados duplicados.	CU06
RF05	Gestión de horarios de atención por médico (día, hora inicial, hora final, duración) y cálculo de horarios disponibles.	CU07, CU08
RF06	Registro de citas: paciente, especialidad, médico, fecha, hora disponible y motivo; código único y fecha de creación.	CU09, CU17
RF07	Prevención de citas duplicadas o superpuestas, validada en el servidor y protegida con transacciones y restricciones.	CU09, CU10, CU17
RF08	Reprogramación de citas Programadas o Confirmadas a un horario disponible.	CU10
RF09	Cancelación de citas con motivo, sin eliminación física, liberando el horario.	CU11
RF10	Control de asistencia y transiciones de estado: Programada, Confirmada, Atendida, Cancelada, Inasistencia.	CU12
RF11	Consulta de agenda médica por médico, fecha y estado.	CU13
RF12	Historial de citas por paciente, incluidas pasadas y canceladas.	CU14
RF13	Reportes: citas por fecha, médico, especialidad y estado; pacientes registrados; atendidas/canceladas/inasistencias por período; agenda diaria; exportación CSV/PDF opcional.	CU15
RF14	Portal del paciente: registro, edición del perfil (datos, contacto, información médica, contacto de emergencia, cuenta), reserva y seguimiento de citas, recordatorios.	CU16, CU17

Requisitos no funcionales

Tabla 3. *Requisitos no funcionales*

Código	Requisito
RNF01	Tecnología: backend en PHP 8, base de datos PostgreSQL 16, conexión PDO con pdo_pgsql, frontend HTML5 y CSS3, JavaScript para validaciones; servidor Apache (XAMPP).
RNF02	Seguridad: contraseñas con password_hash(); sesiones PHP con regeneración de identificador; consultas parametrizadas; token CSRF en formularios; verificación de permisos en el servidor.
RNF03	Validación de datos en el cliente (JavaScript) y en el servidor (PHP), además de restricciones en la base de datos.
RNF04	Manejo de errores: mensajes claros para el usuario sin mostrar credenciales ni detalles internos; los detalles se registran en el log del servidor.
RNF05	Integridad: base de datos normalizada (3FN), con PK, FK, UNIQUE, CHECK, restricciones de exclusión y triggers.
RNF06	Mantenibilidad: separación entre interfaz, lógica de negocio y acceso a datos (MVC por paquetes) y código comentado.
RNF07	Usabilidad: diseño profesional y consistente, adaptable a pantallas de computadora y celular.
RNF08	Rendimiento: las consultas frecuentes (agenda, disponibilidad, historial) deben responder en menos de 2 segundos, con índices sobre las columnas de búsqueda.
RNF09	Zona horaria: todas las fechas y horas se manejan en la hora de Bolivia (America/La_Paz).
RNF10	Versionado: código fuente organizado y publicado en un repositorio GitHub.

2 ELEMENTOS DEL SISTEMA BASADO EN COMPUTADORAS

Un sistema basado en computadoras está formado por hardware, software, datos, procesos, personas y documentos que trabajan en conjunto. A continuación se describen los elementos del sistema de gestión de citas de la Clínica Vida Salud.

2.1 HARDWARE

El hardware comprende los componentes físicos necesarios para que el sistema funcione correctamente.

2.1.1 Servidor

Se utilizará una computadora de la clínica como servidor local, donde se instalarán el servidor web Apache con PHP (XAMPP) y el gestor de base de datos PostgreSQL. Características mínimas recomendadas:

- Procesador: Intel Core i5 de 10.ª generación o superior.
- Memoria RAM: 16 GB.
- Almacenamiento: SSD de 512 GB.
- Sistema operativo: Windows 10/11 Pro o Ubuntu Server 22.04 LTS.
- Conexión de red cableada y unidad de respaldo externa para las copias de seguridad de la base de datos.

2.1.2 Cliente

Las computadoras de recepción, de los consultorios y de la administración deben tener como mínimo:

- Procesador: Intel Core i3 o equivalente.
- Memoria RAM: 8 GB.
- Almacenamiento: 256 GB SSD.
- Monitor de 21 pulgadas o superior, teclado y mouse.

Los pacientes acceden al portal desde su propia computadora o celular con un navegador web actualizado.

2.1.3 Medios de Comunicación

Se utilizará la red local (LAN) de la clínica, cableada y WiFi, para conectar las computadoras de recepción, consultorios y administración con el servidor. El acceso a Internet permitirá que los pacientes utilicen el portal y que el sistema se publique en un hosting cuando la clínica lo requiera.

2.1.4 Otros Dispositivos

- Impresora en recepción para imprimir los comprobantes de cita y los reportes.
- Teléfonos celulares (smartphones) de los pacientes para acceder al portal.
- Router y switch de red.
- UPS (sistema de alimentación ininterrumpida) para el servidor.

2.2 SOFTWARE

El software incluye los programas y sistemas operativos que permiten la operación lógica del sistema.

2.2.1 Servidor

- Sistema operativo: Windows 10/11 Pro o Ubuntu Server 22.04 LTS.
- Servidor web: Apache HTTP Server 2.4 con PHP 8.2 (paquete XAMPP) y la extensión pdo_pgsql habilitada.
- Gestor de base de datos: PostgreSQL 16.

2.2.2 Cliente

- Sistemas operativos: Windows 10/11, Android e iOS.
- Navegadores web: Google Chrome, Microsoft Edge y Mozilla Firefox en sus versiones actuales.

2.2.3 Otro software adicional

- Visual Studio Code (editor de código).
- pgAdmin 4 (administración de PostgreSQL).
- PlantUML y StarUML (diagramas UML).
- Git y GitHub (control de versiones y repositorio del código).
- Microsoft Word (documentación).

2.3 DATOS

Los datos que gestiona el sistema permiten registrar y controlar la información de pacientes, médicos, horarios y citas. Las entidades principales y sus atributos son:

Rol: Roles del sistema (Administrador, Recepcionista, Médico, Paciente). Atributos: id_rol, nombre, descripcion, estado.

Permiso: Operaciones que pueden autorizarse a un rol. Atributos: id_permiso, codigo, descripcion.

Rol permiso: Asignación de permisos a roles (relación muchos a muchos). Atributos: id_rol, id_permiso.

Usuario: Cuentas de acceso; la contraseña se guarda como hash de password_hash(). Atributos: id_usuario, id_rol, username, correo, password_hash, estado, intentos_fallidos, ultimo_acceso, fecha_creacion.

Bitacora: Registro de operaciones relevantes (auditoría). Atributos: id_bitacora, id_usuario, accion, tabla, id_registro, detalle, ip, fecha.

Paciente: Pacientes de la clínica con datos personales, de contacto e información médica. Atributos: id_paciente, codigo, ci, nombres, apellidos, fecha_nacimiento, sexo, telefono, correo, direccion, tipo_sangre, alergias, enfermedades_cronicas, medicamentos, antecedentes, recibir_notificaciones, id_usuario, estado, fecha_registro.

Contacto emergencia: Contactos de emergencia de cada paciente. Atributos: id_contacto, id_paciente, nombre, parentesco, telefono.

Especialidad: Especialidades médicas que ofrece la clínica. Atributos: id_especialidad, nombre, descripcion, estado.

Medico: Médicos de la clínica. Atributos: id_medico, codigo, registro_profesional, nombres, apellidos, telefono, correo, id_usuario, estado.

Medico especialidad: Especialidades que ejerce cada médico (muchos a muchos). Atributos: id_medico, id_especialidad.

Horario atencion: Bloques semanales de atención de cada médico y duración de la cita. Atributos: id_horario, id_medico, dia_semana, hora_inicio, hora_fin, duracion_cita, estado.

Bloqueo agenda: Feriados o fechas bloqueadas por médico o para toda la clínica. Atributos: id_bloqueo, id_medico, fecha, motivo.

Cita: Citas médicas: tabla central del sistema. Atributos: id_cita, codigo, id_paciente, id_medico, id_especialidad, fecha, hora_inicio, hora_fin, motivo, modalidad, estado, motivo_cancelacion, id_usuario_registro, fecha_creacion, fecha_actualizacion.

Historial estado cita: Trazabilidad de los cambios de estado y reprogramaciones de cada cita. Atributos: id_historial, id_cita, estado_anterior, estado_nuevo, fecha_anterior, hora_anterior, observacion, id_usuario, fecha.

2.4 PROCESOS

Los procesos del sistema representan las funcionalidades principales de la gestión de citas de la clínica:

1. Gestión de usuarios y acceso

Actores: Administrador, todos los usuarios. **Descripción:** Permite administrar las cuentas de usuario y controlar el acceso al sistema según el rol.

- Inicio y cierre de sesión
- Registro, modificación y desactivación de usuarios
- Asignación de roles y permisos

2. Gestión de pacientes

Actores: Recepcionista, Administrador. **Descripción:** Permite mantener actualizada la información de los pacientes sin perder su historial.

- Registro de pacientes con CI único
- Búsqueda por CI, nombre o apellido
- Modificación y desactivación
- Consulta de citas anteriores

3. Gestión de médicos y especialidades

Actores: Administrador. **Descripción:** Permite registrar a los médicos con sus especialidades y las especialidades que ofrece la clínica.

- Registro y modificación de médicos
- Asignación de una o varias especialidades
- Activación y desactivación
- Registro de especialidades sin duplicados

4. Gestión de horarios de atención

Actores: Administrador. **Descripción:** Permite definir cuándo atiende cada médico y la duración de las citas.

- Registro de bloques semanales (día, hora inicial y final)
- Configuración de la duración de la cita
- Registro de fechas bloqueadas o feriados

5. Programación de citas

Actores: Recepcionista, Paciente. **Descripción:** Permite registrar citas solo en horarios disponibles, sin conflictos.

- Consulta de disponibilidad
- Registro de la cita con código único
- Verificación de disponibilidad en el servidor
- Emisión del comprobante

6. Seguimiento de citas

Actores: Recepcionista, Médico, Paciente. **Descripción:** Permite controlar el ciclo de vida de la cita.

- Reprogramación
- Cancelación con motivo
- Confirmación de asistencia, atención e inasistencia
- Consulta de la agenda médica y del historial

7. Reportes

Actores: Administrador, Recepcionista. **Descripción:** Permite obtener información para la toma de decisiones.

- Citas por fecha, médico, especialidad y estado
- Pacientes registrados
- Atendidas, canceladas e inasistencias por período
- Agenda diaria de cada médico

8. Portal del paciente

Actores: Paciente. **Descripción:** Permite al paciente gestionar su información y sus citas.

- Registro y edición del perfil
- Información médica y contacto de emergencia
- Reserva y seguimiento de citas
- Recordatorios de próximas consultas

2.5 GENTE/USUARIO

El sistema contempla cuatro tipos de usuarios, con permisos definidos según su rol:

Administrador: Configura y supervisa el sistema: registra y administra usuarios, asigna roles y permisos, registra médicos, especialidades y horarios de atención, activa o desactiva registros y consulta los reportes generales.

Recepcionista: Gestiona la atención de los pacientes: registra y busca pacientes, consulta médicos y horarios disponibles, programa, reprograma y cancela citas, registra la asistencia o inasistencia y consulta citas por fecha y estado.

Médico: Consulta su agenda de atención, visualiza los datos básicos del paciente, identifica las citas pendientes, atendidas y canceladas, y marca una cita como atendida cuando tiene ese permiso.

Paciente: Accede al portal del paciente: registra y actualiza su perfil (datos personales, contacto, información médica y contacto de emergencia), reserva citas y da seguimiento a su estado.

2.6 DOCUMENTO

La documentación que respalda al sistema es:

- **Documentación PUDS-UML:** el presente documento, con los artefactos de captura de requisitos, análisis, diseño, implementación y pruebas.
- **Manual de instalación:** pasos para instalar XAMPP y PostgreSQL, crear la base de datos con los scripts y configurar la conexión.
- **Manual de usuario:** instrucciones de uso de cada módulo según el rol (administrador, recepcionista, médico y paciente).
- **Comprobante de cita:** documento impreso o en pantalla con el código, paciente, médico, especialidad, fecha y horario de la cita.
- **Reportes administrativos:** listados y totales de citas exportables a CSV o PDF.

3 TECNOLOGÍA PARA EL DESARROLLO DEL SOFTWARE

3.1 Estrategia para el desarrollo del software

La estrategia de desarrollo se basa en construir el sistema por incrementos, comenzando por las funcionalidades sin las cuales no es posible programar una cita: el control de acceso, el registro de pacientes, médicos y especialidades, la definición de los horarios de atención y la programación de citas sin conflictos (Ciclo #1). Sobre esta base se incorporaran la reprogramación, cancelación, control de asistencia, agenda médica, historial, reportes y portal del paciente (Ciclo #2).

Se empleará el Proceso Unificado de Desarrollo de Software (PUDS) como metodología, por estar dirigido por casos de uso, centrado en la arquitectura y ser iterativo e incremental. Cada ciclo recorre los flujos de trabajo de captura de requisitos, análisis, diseño, implementación y pruebas, y se valida con los usuarios de la clínica. En complemento se utilizará UML para representar de forma gráfica los casos de uso, clases, secuencias, actividades, paquetes y despliegue, facilitando la comunicación entre los integrantes del equipo y la documentación del sistema.

3.2 Metodología para el desarrollo del software

3.2.1 Características del PUDS

- **Dirigido por casos de uso:** los casos de uso representan los requisitos funcionales y guían el análisis, el diseño, la implementación y las pruebas.
- **Centrado en la arquitectura:** se define tempranamente una arquitectura estable (capas y paquetes) que permite realizar todos los casos de uso.
- **Iterativo e incremental:** el trabajo se divide en ciclos; cada ciclo produce un incremento ejecutable del sistema.
- **Gestión de riesgos:** los casos de uso más críticos (programación de citas sin conflictos) se abordan en el primer ciclo.
- **Fases y flujos de trabajo:** inicio, elaboración, construcción y transición; con los flujos de captura de requisitos, análisis, diseño, implementación y pruebas.

3.2.2 Características de UML

- **Representación visual:** utiliza diagramas para modelar la estructura y el comportamiento del sistema de forma comprensible.
- **Estandarización:** es un lenguaje estándar del OMG, ampliamente aceptado, que facilita la comunicación y la documentación consistente.
- **Independencia del proceso y del lenguaje:** puede utilizarse con distintos procesos de desarrollo y lenguajes de programación; aquí se usa con PUDS y PHP.
- **Diversidad de vistas:** permite modelar aspectos estáticos (clases, paquetes, despliegue) y dinámicos (casos de uso, actividades, secuencia, comunicación, estados).

3.3 HERRAMIENTAS DE DESARROLLO

3.3.1 Software

- **PHP 8.2:** lenguaje del lado del servidor para los controladores y modelos; incluye PDO, password_hash() y password_verify().
- **PostgreSQL 16:** gestor de base de datos relacional de código abierto, con restricciones de exclusión, funciones PL/pgSQL, triggers y vistas.
- **HTML5, CSS3 y JavaScript:** construcción de la interfaz de usuario y validaciones en el navegador.
- **XAMPP (Apache 2.4):** servidor web local para ejecutar la aplicación PHP.
- **Visual Studio Code, pgAdmin 4, Git y GitHub:** edición de código, administración de la base de datos y control de versiones.
- **PlantUML / StarUML:** elaboración de los diagramas UML.

3.3.2 Hardware

Para el desarrollo se utilizarán las computadoras personales de los integrantes del equipo, con las siguientes características mínimas:

- Procesador Intel Core i5 o AMD Ryzen 5.
- Memoria RAM de 8 GB (16 GB recomendado).
- Almacenamiento SSD de 256 GB o superior.
- Sistema operativo Windows 10/11 y conexión a Internet.

Equipo	Procesador	RAM	Almacenamiento	GPU
HP Laptop	Core i5-1235U	16 GB	512 GB SSD	GTX 3050
Dell Inspiron 15 3000	Ryzen5-7535HS	8 GB	512 GB SSD	AMD Radeon

4 POSIBLES COSTOS

Los costos se estiman para el desarrollo del sistema durante cuatro semanas por un equipo de estudiantes. Son valores referenciales en bolivianos (Bs); el hardware y el software no representan un gasto adicional porque se utilizan equipos existentes y herramientas gratuitas o de código abierto.

Tabla 4. *Estimación de costos del proyecto (valores referenciales)*

Concepto	**Detalle**	**Costo estimado (Bs)**
Software de desarrollo	PHP, PostgreSQL, XAMPP, VS Code, pgAdmin, Git, PlantUML (licencias libres)	0
Hardware de desarrollo	Computadoras de los integrantes (recurso existente)	10.000
Servidor de producción	Computadora existente de la clínica como servidor local	0
Internet	Conexión de los integrantes durante 1 mes (2 × 200 Bs)	400
Hosting y dominio (opcional)	Publicación en Internet del portal del paciente durante 1 mes	62
TOTAL		**10.462**

5 POSIBLES BENEFICIOS

5.1 TIEMPO

El sistema reducirá el tiempo necesario para programar, buscar y controlar las citas:

- La recepcionista verá de inmediato los horarios disponibles del médico, sin revisar cuadernos ni consultar verbalmente.
- La búsqueda de un paciente por CI o nombre y de su historial de citas se realizará en segundos.
- Los reportes de atenciones se generarán automáticamente en lugar de elaborarse durante horas.
- El médico conocerá su agenda del día con anticipación y los pacientes podrán reservar desde el portal sin esperar en línea telefónica.

5.2 ESFUERZO

Actualmente el personal anota las citas a mano, verifica los horarios de memoria y cuenta las atenciones para los reportes. Con el sistema:

- La disponibilidad se calcula y valida automáticamente, eliminando el esfuerzo de revisar posibles cruces.
- Los datos del paciente se registran una sola vez y se reutilizan en cada cita.
- El cambio de estado de las citas (cancelada, atendida, inasistencia) queda registrado con un clic.
- El personal podrá concentrarse en la atención al paciente en lugar de tareas administrativas repetitivas.

5.3 COSTOS

El sistema permitirá reducir costos operativos de la clínica:

- Menor uso de cuadernos, papel e impresiones.
- Menos horarios perdidos por cancelaciones no registradas y por citas duplicadas, lo que aumenta el número de consultas atendidas.
- Menos llamadas telefónicas para confirmar o reprogramar citas.
- Mejor planificación de los horarios de los médicos a partir de reportes confiables.

6 POSIBLES CLIENTES

El sistema está dirigido en primer lugar a la Clínica Vida Salud, pero por su diseño general puede adaptarse a otros establecimientos de salud que gestionan citas por especialidad y horario:

- **Clínicas privadas de atención ambulatoria:** con varias especialidades y médicos que atienden en horarios distintos.
- **Consultorios médicos y policonsultorios:** que necesitan ordenar la agenda de uno o varios profesionales.
- **Centros odontológicos:** con citas de duración fija y alta frecuencia de reprogramaciones.
- **Centros de fisioterapia, psicología o nutrición:** que trabajan con sesiones programadas.
- **Pacientes:** como usuarios finales del portal, beneficiados con la reserva en línea y el seguimiento de sus citas.

7 MODELO DE DOMINIO

El modelo de dominio representa los conceptos más importantes del negocio de la clínica y cómo se relacionan, sin detalles de implementación. Se obtuvo del caso de estudio propuesto por la docente: un paciente solicita citas; cada cita corresponde a una especialidad y la atiende un médico que ejerce esa especialidad dentro de su horario de atención; los cambios de cada cita se registran en su historial. Los usuarios del sistema tienen un rol con permisos; un paciente o un médico pueden tener una cuenta de usuario para ingresar al sistema, y las acciones de los usuarios quedan en la bitácora.

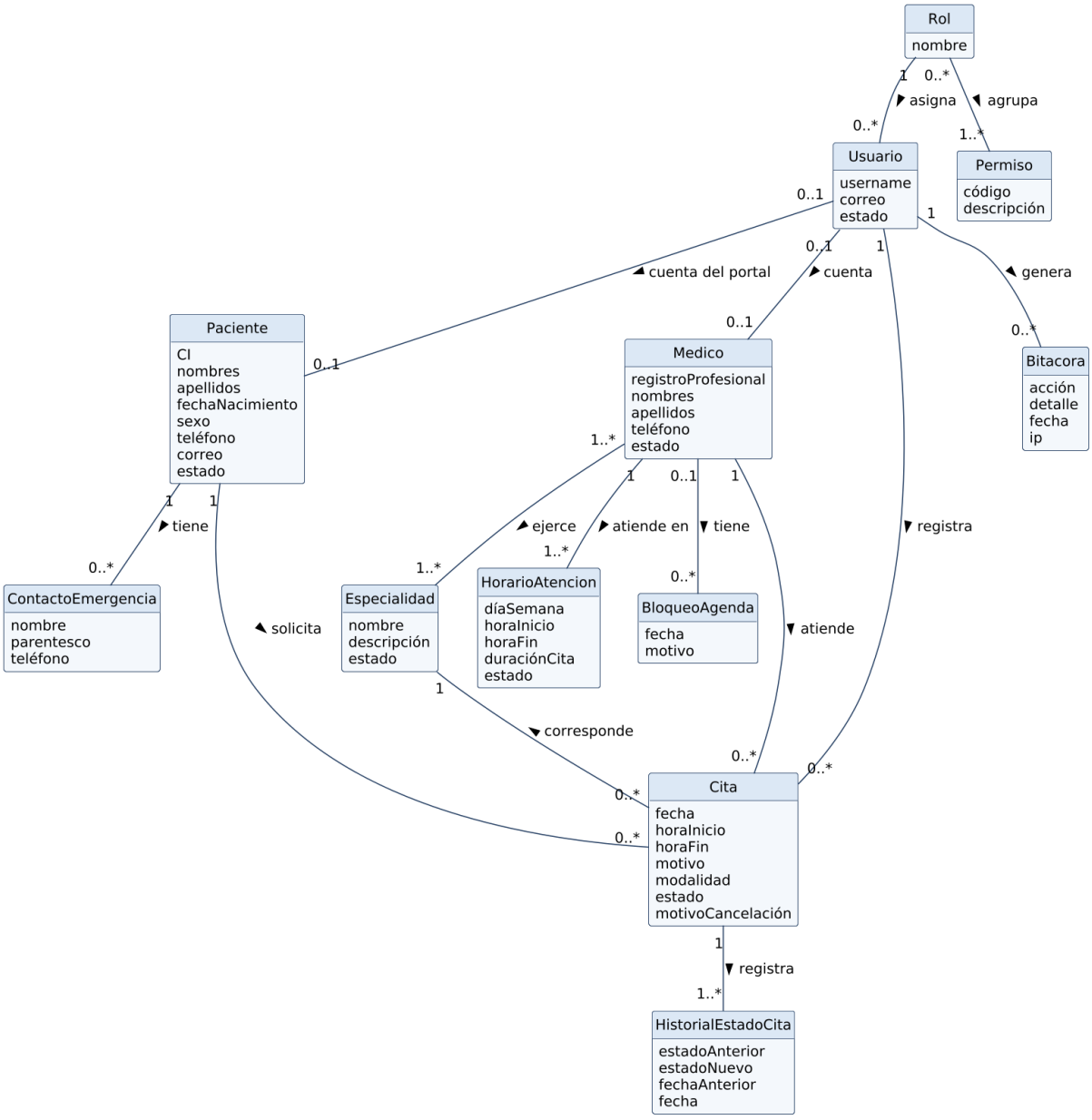


Figura 1. *Modelo de dominio de la Clínica Vida Salud*

8 MODELO DE NEGOCIO

El modelo de negocio describe cómo trabaja actualmente la clínica y cómo trabajará con el sistema. Se representa con diagramas de actividades con calles (swimlanes) para cada trabajador del negocio; de ellos se obtienen los actores y casos de uso del sistema.

8.1 Diagrama de actividades

Proceso actual de programación de citas

El proceso actual depende de registros manuales: la recepcionista no tiene una vista confiable de la disponibilidad, por lo que la cita puede anotarse sobre un horario ya ocupado; las inasistencias no se registran y los reportes se elaboran contando a mano.

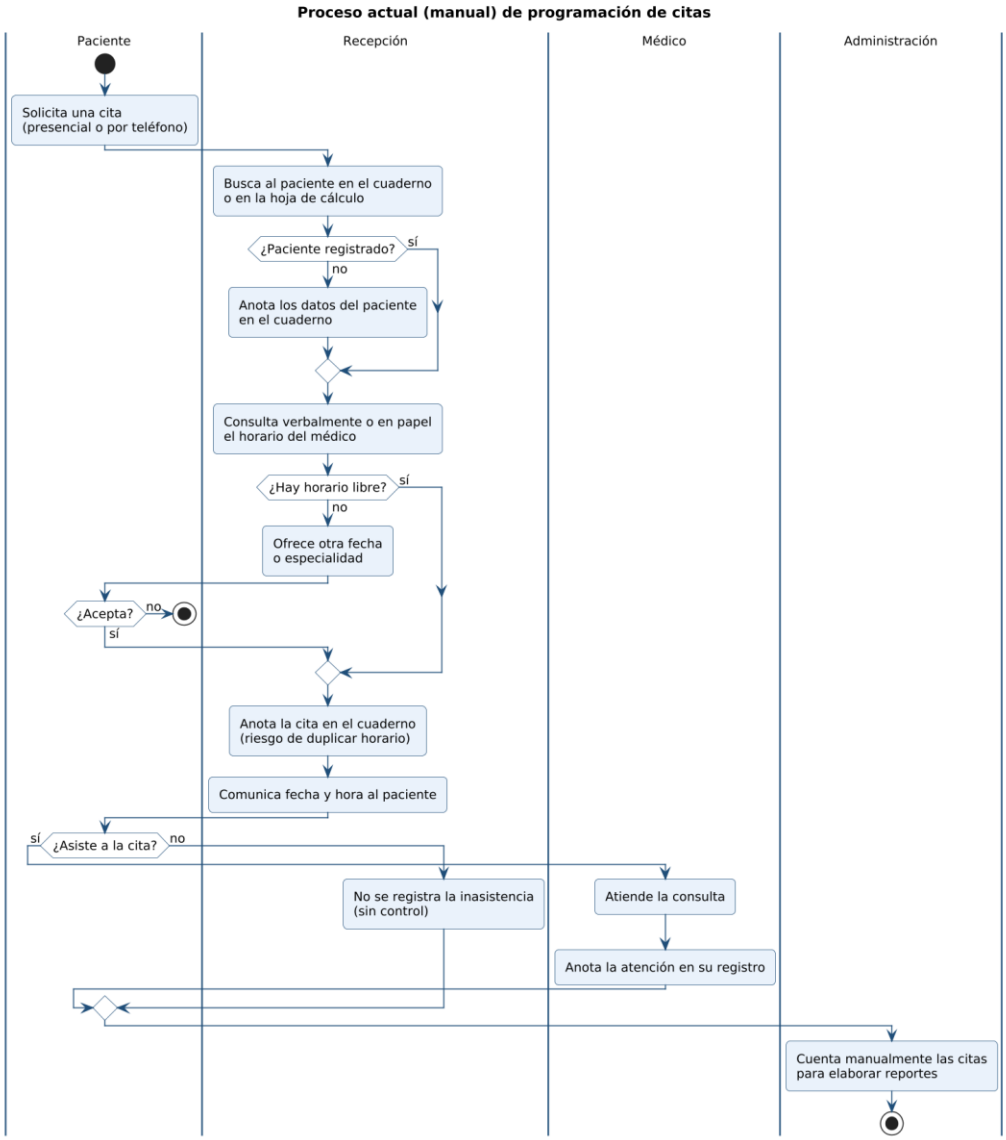


Figura 2. *Diagrama de actividades — proceso actual de programación de citas*

Proceso propuesto de programación de citas

Con el sistema, la disponibilidad se calcula automáticamente a partir del horario de atención y de las citas ocupadas, y se vuelve a verificar en el servidor al guardar.

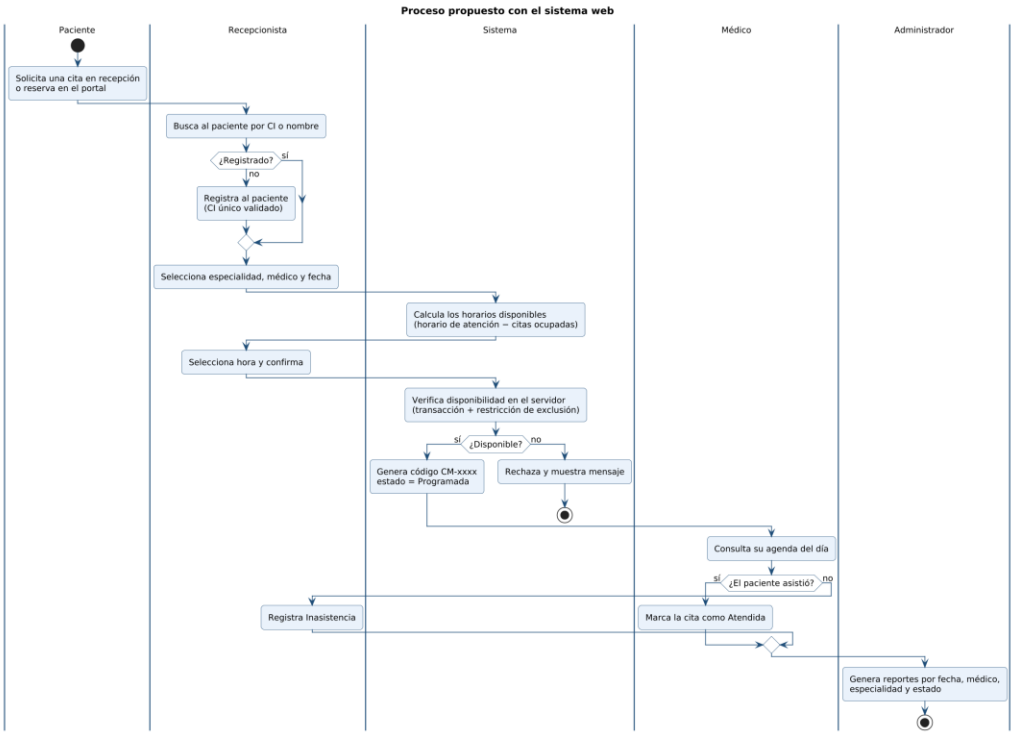


Figura 3. *Diagrama de actividades — proceso propuesto con el sistema web*

Proceso de atención de la cita y control de asistencia

El día de la cita, la recepcionista confirma la llegada del paciente o registra su inasistencia; el médico consulta su agenda, atiende y marca la cita como atendida, y cada cambio queda en el historial.

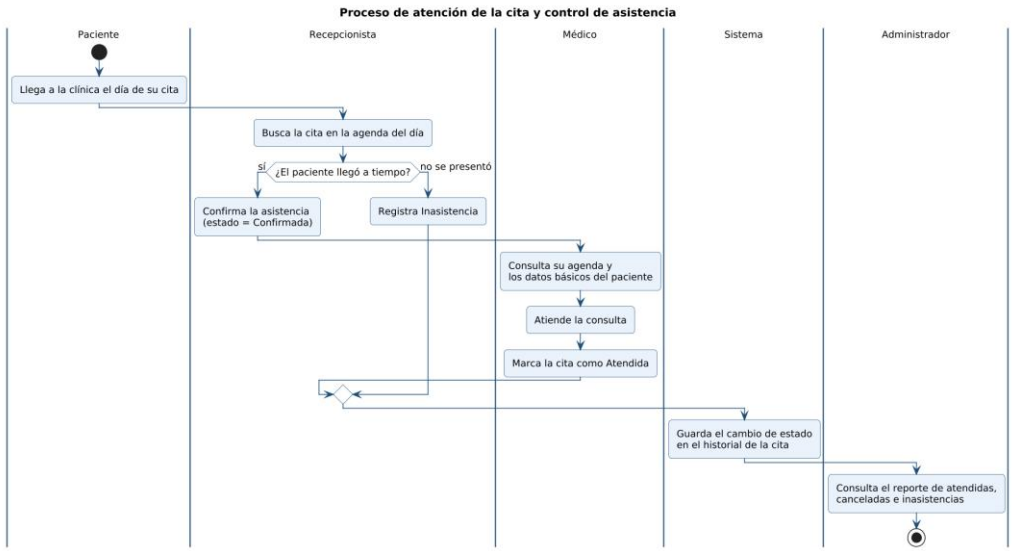


Figura 4. *Diagrama de actividades — atención de la cita y control de asistencia*

Reglas del negocio

Las siguientes reglas se aplican independientemente de lo que el usuario escriba en los formularios; se indica dónde se hace cumplir cada una.

Tabla 5. *Reglas del negocio y mecanismo que las garantiza*

Código	Regla	Dónde se garantiza
RN01	Cada paciente tendrá un identificador único (código y CI único cuando se proporciona).	PK id_paciente, UNIQUE codigo y ci
RN02	Cada médico tendrá un identificador único.	PK id_medico, UNIQUE codigo y registro_profesional
RN03	Una cita deberá estar asociada a un paciente y a un médico válidos.	FK cita → paciente, medico, especialidad
RN04	El médico deberá estar activo para recibir nuevas citas.	Controlador + trigger fn_validar_cita
RN05	La cita deberá pertenecer al horario de atención del médico.	Controlador + trigger fn_validar_cita
RN06	No se podrán registrar citas en fechas pasadas.	Validación JS/PHP + trigger fn_validar_cita
RN07	Un médico no podrá tener citas superpuestas.	Restricción EXCLUDE ex_cita_superpuesta
RN08	La duración de la cita deberá respetar la configuración del sistema.	duracion_cita en horario_atencion + trigger
RN09	Las citas canceladas no ocuparán disponibilidad.	EXCLUDE ... WHERE estado <> 'Cancelada'
RN10	Las citas atendidas conservarán su registro histórico.	Trigger tg_no_borrar_cita + historial_estado_cita
RN11	No se eliminarán pacientes o médicos con citas; se desactivarán.	Campo estado + triggers que impiden DELETE
RN12	Solo los usuarios autorizados podrán realizar cada operación.	Auth::requerir(permiso) en cada controlador
RN13	La programación y reprogramación deben verificar la disponibilidad en el servidor.	Transacción en CitaModel + EXCLUDE
RN14	Los datos se almacenarán de forma persistente en PostgreSQL.	Base de datos clinica_vida_salud

9 MARCO TEÓRICO

9.1 Sistemas de información

Un sistema de información es un conjunto de componentes interrelacionados (personas, procedimientos, datos, software y hardware) que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones y el control de una organización. En una clínica, el sistema de información de citas coordina la oferta de atención (médicos, especialidades y horarios) con la demanda (pacientes), y produce información para la administración.

9.2 Diagrama de Ishikawa

El diagrama de Ishikawa, o diagrama de causa y efecto (espina de pescado), es una herramienta de calidad que organiza las posibles causas de un problema en categorías. En la variante de las 6M las categorías son mano de obra, métodos, máquina, materiales, medición y medio ambiente. Permite identificar las causas raíz de un problema antes de proponer una solución.

9.3 Proceso Unificado de Desarrollo de Software (PUDS)

El Proceso Unificado es un proceso de desarrollo de software, es decir, el conjunto de actividades necesarias para transformar los requisitos del usuario en un sistema de software. Está dirigido por casos de uso, centrado en la arquitectura y es iterativo e incremental. Su ciclo de vida se organiza en cuatro fases —inicio, elaboración, construcción y transición— y cinco flujos de trabajo fundamentales: captura de requisitos, análisis, diseño, implementación y pruebas (Jacobson, Booch y Rumbaugh, 2000).

9.4 Flujos de trabajo del PUDS

Captura de requisitos: identifica actores y casos de uso, los prioriza, los detalla, prototipa la interfaz de usuario y estructura el modelo de casos de uso. Análisis: refina los requisitos en un modelo de análisis con clases de interfaz, control y entidad, organizadas en paquetes. Diseño: define la arquitectura física y lógica, realiza cada caso de uso mediante diagramas de secuencia y diseña los datos. Implementación: transforma el diseño en componentes, archivos y subsistemas. Pruebas: verifica el resultado de la implementación mediante casos de prueba.

9.5 Lenguaje Unificado de Modelado (UML)

UML es un lenguaje gráfico estándar para visualizar, especificar, construir y documentar los artefactos de un sistema de software. En este proyecto se utilizan los diagramas de actividades (modelo de negocio), casos de uso (requisitos), comunicación y clases de análisis (análisis), despliegue, paquetes, secuencia, estados y clases (diseño), y componentes (implementación).

9.6 Clases de análisis: interfaz, control y entidad

En el modelo de análisis, las clases de interfaz (boundary) modelan la interacción entre el sistema y sus actores; las clases de control coordinan la secuencia de un caso de uso y aplican sus reglas; y las clases entidad modelan

la información persistente. El actor solo se comunica con interfaces, las interfaces con controles y los controles con entidades; las entidades no dialogan entre sí.

9.7 Arquitectura en capas y patrón MVC

La arquitectura en capas separa la presentación, la lógica de negocio y el acceso a datos, de modo que un cambio en una capa afecte lo menos posible a las demás. El patrón Modelo–Vista–Controlador (MVC) aplica esta idea en aplicaciones web: la vista genera la interfaz HTML, el controlador recibe la petición, valida y coordina, y el modelo accede a la base de datos.

9.8 PHP y PDO

PHP es un lenguaje de programación interpretado del lado del servidor, ampliamente usado para aplicaciones web y ejecutado comúnmente sobre el servidor Apache. PDO (PHP Data Objects) es una capa de abstracción de acceso a datos; con el controlador pdo_pgsql permite conectarse a PostgreSQL y ejecutar consultas preparadas, que previenen la inyección SQL. PHP ofrece password_hash() y password_verify() para almacenar y verificar contraseñas con algoritmos de hash seguros.

9.9 PostgreSQL

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto que cumple las propiedades ACID. Además de claves primarias, foráneas, restricciones UNIQUE y CHECK, ofrece restricciones de exclusión (EXCLUDE USING gist), que impiden que dos filas tengan rangos superpuestos; en este proyecto se usan para garantizar que un médico no tenga dos citas en el mismo intervalo, incluso si dos usuarios registran citas al mismo tiempo. También permite funciones en PL/pgSQL, disparadores (triggers) y vistas.

9.10 Bases de datos relacionales y normalización

El modelo relacional organiza los datos en tablas vinculadas mediante claves. La normalización estructura las tablas para eliminar redundancias y anomalías: la primera forma normal (1FN) exige valores atómicos; la segunda (2FN), que todo atributo no clave dependa de toda la clave; y la tercera (3FN), que no existan dependencias transitivas entre atributos no clave (Elmasri y Navathe, 2007).

9.11 Seguridad en aplicaciones web

Las principales amenazas de una aplicación web incluyen la inyección SQL, el control de acceso deficiente, la exposición de datos sensibles y la falsificación de peticiones (CSRF), recogidas en el OWASP Top 10. Las contramedidas aplicadas son: consultas parametrizadas, hash de contraseñas, sesiones con regeneración de identificador, verificación de permisos en cada página, tokens CSRF, validación en cliente y servidor, y mensajes de error que no revelan detalles internos.

CAPÍTULO 1: MÉTODO DE ISHIKAWA

Para identificar las causas de la deficiente gestión de citas se aplicó el método de Ishikawa a partir del caso de estudio, la descripción de los roles de la clínica y la observación del proceso actual de recepción.

1) IDENTIFICAR PROBLEMA

1.1) LISTA DE PROBLEMAS

En una lluvia de ideas con el equipo se listaron los siguientes problemas:

- **P1.** Se programan dos citas para el mismo médico en el mismo horario.
- **P2.** Los pacientes quedan registrados dos veces o con datos incorrectos (CI, teléfono, nombre).
- **P3.** La recepcionista no conoce en tiempo real los horarios disponibles de cada médico.
- **P4.** Las cancelaciones no se registran y el horario sigue apareciendo ocupado.
- **P5.** No se registra cuando un paciente no se presenta a su cita.
- **P6.** Encontrar el historial de citas de un paciente en cuadernos toma demasiado tiempo.
- **P7.** Los reportes de atenciones se elaboran contando a mano y contienen errores.
- **P8.** La dirección no cuenta con información actualizada para decidir horarios y personal.
- **P9.** Se asignan citas a médicos que están de vacaciones o que ya no trabajan en la clínica.
- **P10.** Se anotan citas fuera del horario de atención del médico o en fechas ya pasadas.
- **P11.** El médico no conoce su agenda del día con anticipación.
- **P12.** El cuaderno de citas se pierde, se moja o se deteriora.
- **P13.** Las líneas telefónicas se saturan en horas pico y los pacientes no logran comunicarse.
- **P14.** Cualquier persona puede abrir y modificar las hojas de cálculo de citas.
- **P15.** No se sabe quién registró, modificó o canceló una cita.
- **P16.** Los pacientes olvidan su cita porque no reciben ningún recordatorio.
- **P17.** El médico no dispone de la información médica básica del paciente (alergias, antecedentes).
- **P18.** La letra ilegible en el cuaderno provoca errores en el nombre o la hora del paciente.
- **P19.** Cortes de energía eléctrica en el edificio de la clínica.
- **P20.** El estacionamiento de la clínica es pequeño.
- **P21.** Los pacientes llegan tarde por el tráfico de la ciudad.
- **P22.** La impresora de recepción se queda sin tinta.
- **P23.** El cobro de la consulta se calcula mal según la especialidad.

1.2) DEPURAR PROBLEMAS

Se descartaron los problemas que no corresponden al sistema de información o que están incluidos en otro problema:

Tabla 6. *Problemas depurados y motivo*

Problema	Motivo de la depuración
P18. La letra ilegible en el cuaderno provoca errores en el nombre o la hora del paciente.	Se integra en el problema de registro incorrecto de pacientes (P2 de la lista final).
P19. Cortes de energía eléctrica en el edificio de la clínica.	Es un problema de infraestructura eléctrica del edificio; no corresponde al sistema de información.
P20. El estacionamiento de la clínica es pequeño.	Es un problema de infraestructura física, ajeno a la gestión de citas.
P21. Los pacientes llegan tarde por el tráfico de la ciudad.	Es un factor externo que la clínica no controla; el sistema solo registra la asistencia.
P22. La impresora de recepción se queda sin tinta.	Es un problema de insumos y mantenimiento de equipos.
P23. El cobro de la consulta se calcula mal según la especialidad.	La facturación y los cobros están fuera del alcance del sistema.

1.3) LISTA FINAL DE PROBLEMAS

Después de la depuración, los problemas se renumeraron y quedó la siguiente lista final:

- **P1.** Duplicación de citas para un mismo médico y horario.
- **P2.** Registro duplicado o incorrecto de los datos de los pacientes.
- **P3.** Desconocimiento en tiempo real de los horarios disponibles de los médicos.
- **P4.** Cancelaciones no registradas que mantienen horarios bloqueados.
- **P5.** Falta de control de las inasistencias de los pacientes.
- **P6.** Demora en la búsqueda del historial de citas de un paciente.
- **P7.** Elaboración manual, lenta y con errores de los reportes de atenciones.
- **P8.** Falta de información actualizada para la dirección de la clínica.
- **P9.** Asignación de citas a médicos inactivos o de vacaciones.
- **P10.** Citas registradas fuera del horario de atención o en fechas pasadas.
- **P11.** El médico no conoce su agenda con anticipación.
- **P12.** Pérdida o deterioro de los registros físicos de citas.
- **P13.** Saturación de las llamadas telefónicas en horas pico.
- **P14.** Falta de control de acceso a la información de citas y pacientes.
- **P15.** Falta de trazabilidad de quién registró, modificó o canceló una cita.
- **P16.** Olvido de las citas por falta de recordatorios al paciente.
- **P17.** La información médica básica del paciente no está disponible para el médico.

1.4) PROPIETARIOS DE PROBLEMAS

Se identificó qué actor de la clínica es responsable de cada problema o se ve más afectado por él.

Tabla 7. *Tabla de propietarios y problemas*

Problema	Paciente	Recepción	Médico	Dirección	TI
P1. Duplicación de citas para un mismo médico y horario.		X			X
P2. Registro duplicado o incorrecto de los datos de los pacientes.	X	X			
P3. Desconocimiento en tiempo real de los horarios disponibles de los médicos.		X			X
P4. Cancelaciones no registradas que mantienen horarios bloqueados.		X			
P5. Falta de control de las inasistencias de los pacientes.		X			
P6. Demora en la búsqueda del historial de citas de un paciente.		X	X		
P7. Elaboración manual, lenta y con errores de los reportes de atenciones.				X	X
P8. Falta de información actualizada para la dirección de la clínica.				X	
P9. Asignación de citas a médicos inactivos o de vacaciones.				X	X
P10. Citas registradas fuera del horario de atención o en fechas pasadas.		X			X
P11. El médico no conoce su agenda con anticipación.			X		
P12. Pérdida o deterioro de los registros físicos de citas.		X		X	
P13. Saturación de las llamadas telefónicas en horas pico.	X	X			
P14. Falta de control de acceso a la información de citas y pacientes.				X	X
P15. Falta de trazabilidad de quién registró, modificó o canceló una cita.				X	X
P16. Olvido de las citas por falta de recordatorios al paciente.	X				
P17. La información médica básica del paciente no está disponible para el médico.			X		

1.5) ANÁLISIS DE PROBLEMAS

El siguiente grafo dirigido muestra las relaciones de causa y efecto entre los problemas de la lista final: una flecha de Pi a Pj indica que Pi provoca o agrava Pj. Todos los caminos terminan en el problema central, la deficiente gestión de citas médicas.

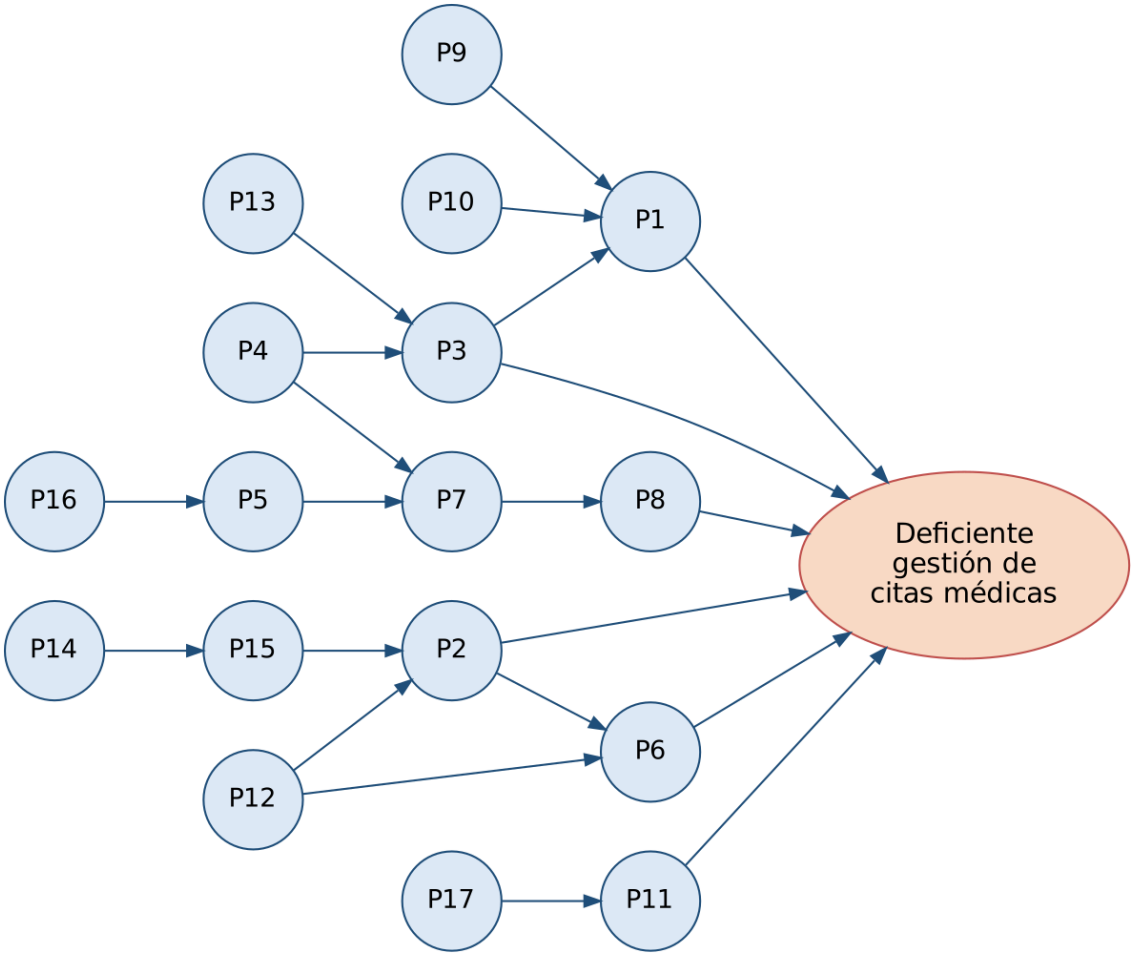


Figura 5. *Grafo dirigido de los problemas*

Del grafo se observa que la falta de disponibilidad en tiempo real (P3) y las cancelaciones no registradas (P4) son causas de varios problemas, ya que originan las citas duplicadas (P1) y los reportes incorrectos (P7). La pérdida de registros (P12) y la falta de trazabilidad (P15) afectan a la calidad de los datos de los pacientes (P2) y al historial (P6).

1.6) ESTIMACIÓN Y CUANTIFICACIÓN DE PROBLEMA

Se estimó el peso de cada problema sobre el problema central (en porcentaje, con un total de 100 %), según su frecuencia, el número de procesos que afecta y su impacto sobre el paciente. Son estimaciones del equipo para el caso de estudio.

Tabla 8. *Tabla de problemas y estimación*

Problema	**Estimación (%)**	**Justificación**
P1. Citas duplicadas	12	Es el problema más visible para el paciente y el que motiva el proyecto; afecta directamente la atención.

Problema	Estimación (%)	Justificación
P2. Datos de paciente erróneos	9	Los datos erróneos impiden contactar al paciente y generan registros duplicados.
P3. Sin disponibilidad real	10	Sin disponibilidad real no es posible programar citas correctamente; origina P1.
P4. Cancelaciones sin control	7	Deja horarios sin usar y distorsiona la disponibilidad y los reportes.
P5. Inasistencias sin registro	6	Sin registro de inasistencias no se puede medir ni corregir la pérdida de horarios.
P6. Historial lento	7	Retrasa la atención en recepción y deja al médico sin antecedentes de consultas.
P7. Reportes manuales	6	Consume horas de trabajo y produce información con errores.
P8. Sin información gerencial	5	Consecuencia de P7; afecta la planificación de horarios y personal.
P9. Médicos inactivos	5	Provoca citas que no pueden atenderse y deben reprogramarse.
P10. Citas fuera de horario	6	Genera citas imposibles de cumplir y contribuye a P1.
P11. Agenda médica no visible	5	El médico no se prepara para su jornada ni conoce a sus pacientes del día.
P12. Pérdida de registros	4	Poco frecuente, pero su impacto es alto cuando ocurre (pérdida total de información).
P13. Saturación telefónica	5	En horas pico los pacientes no logran comunicarse y se pierden reservas.
P14. Sin control de acceso	3	Riesgo de modificaciones indebidas de la información de los pacientes.
P15. Sin trazabilidad	3	Impide aclarar errores y responsabilidades en el registro de citas.
P16. Sin recordatorios	4	Aumenta las inasistencias (P5).
P17. Sin datos médicos	3	Afecta la calidad de la atención médica; el sistema la mejora parcialmente con el perfil del paciente.
Total	**100**	

1.7) ALTERNATIVAS DE CAMBIO

AC1. Agenda centralizada con validación de disponibilidad

- **Cambio sugerido:** Calcular los horarios disponibles a partir del horario de atención de cada médico y validar en el servidor y en la base de datos que no existan citas superpuestas, fuera de horario, en fechas pasadas o con médicos inactivos.
- **Problemas que resuelve:** P1, P3, P9, P10, P13
- **Justificación:** Elimina la causa principal de las citas duplicadas y permite atender más rápido por teléfono o en mostrador.

AC2. Registro único de pacientes

- **Cambio sugerido:** Registrar a cada paciente una sola vez con CI único, validación de datos y perfil con información médica y contacto de emergencia.
- **Problemas que resuelve:** P2, P12, P17

- **Justificación:** Evita duplicados y datos erróneos; la información queda respaldada en la base de datos y no en papel.

AC3. Control del ciclo de vida de la cita

- **Cambio sugerido:** Manejar los estados Programada, Confirmada, Atendida, Cancelada e Inasistencia con transiciones válidas, motivo de cancelación e historial de cambios.
- **Problemas que resuelve:** P4, P5, P6, P11, P16
- **Justificación:** Libera los horarios cancelados, registra las inasistencias y permite consultar el historial y la agenda del médico.

AC4. Reportes automáticos

- **Cambio sugerido:** Generar reportes por fecha, médico, especialidad y estado con totales y filtros, exportables a CSV o PDF.
- **Problemas que resuelve:** P7, P8
- **Justificación:** Proporciona a la dirección información confiable sin conteos manuales.

AC5. Seguridad y trazabilidad

- **Cambio sugerido:** Control de acceso por roles y permisos, contraseñas con hash, sesiones seguras y registro del usuario que realiza cada operación.
- **Problemas que resuelve:** P14, P15
- **Justificación:** Protege la información sensible de los pacientes y permite rastrear los cambios.

1.8) CONCLUSIÓN Y RECOMENDACIÓN

El diagnóstico permitió identificar 17 problemas en la gestión de citas de la Clínica Vida Salud, cuyas causas se concentran en la ausencia de una fuente única de información sobre horarios y citas y en la falta de reglas que impidan registrar citas inválidas. Las cinco alternativas de cambio se traducen en 17 casos de uso organizados en 6 paquetes y distribuidos en dos ciclos de desarrollo. Se concluye que un sistema web con validación de disponibilidad en el servidor y en la base de datos es una solución viable para eliminar las citas duplicadas y mejorar el control de las atenciones.

Se recomienda priorizar en el primer ciclo la agenda centralizada con validación de disponibilidad (AC1) y el registro único de pacientes (AC2), porque atacan los problemas de mayor peso (P1, P2 y P3), y completar en el segundo ciclo el control del ciclo de vida de la cita, los reportes y el portal del paciente.

1.9) DISEÑAR EL DIAGRAMA DE ISHIKAWA

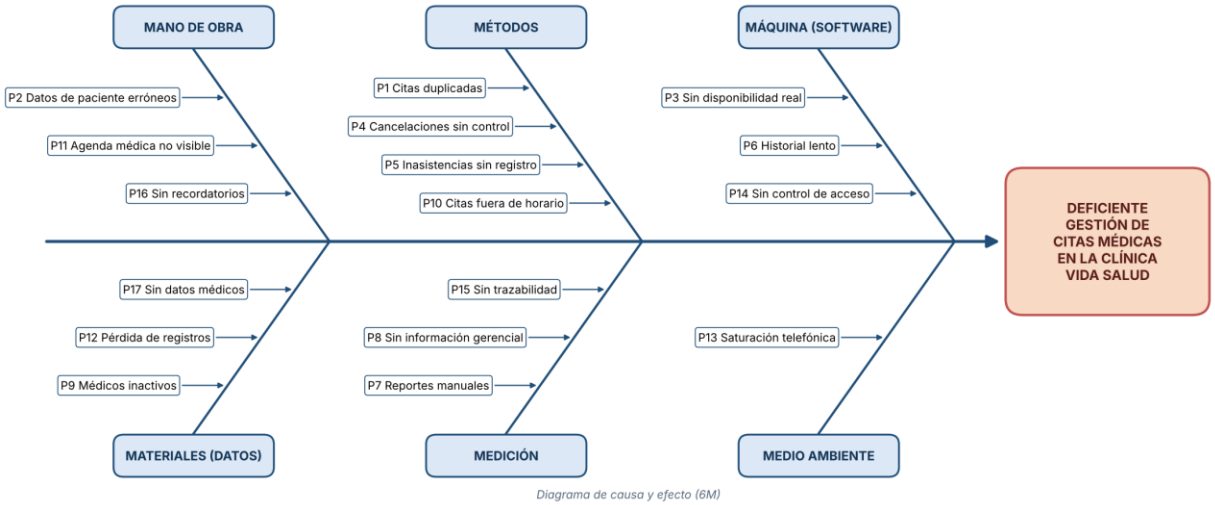


Figura 6. *Diagrama de Ishikawa (6M) de la deficiente gestión de citas médicas*

Categorías: Mano de obra, Métodos, Máquina, Materiales, Medición y Medio ambiente. Cada flecha corresponde a un problema de la lista final.

2) IDENTIFICAR LAS PRINCIPALES CATEGORÍAS

1. Mano de obra: Se refiere a las personas que intervienen en el proceso (recepcionista, médicos y pacientes). Errores de registro, olvidos y falta de información del personal detienen el proceso aunque las herramientas sean adecuadas.

2. Métodos: Son las reglas y procedimientos con los que se programa y controla una cita. Si el procedimiento permite anotar dos citas en el mismo horario o no exige registrar las cancelaciones, el error está en el método.

3. Máquina (software): Corresponde a las herramientas tecnológicas utilizadas. Hoy son hojas de cálculo y cuadernos que no calculan disponibilidad, no permiten búsquedas rápidas ni controlan el acceso.

4. Materiales (datos): En un sistema de información los materiales son los datos. Si la información de médicos, pacientes o citas está desactualizada, incompleta o se pierde, el resultado será incorrecto.

5. Medición: Se refiere a la capacidad de controlar y evaluar el proceso: reportes, indicadores y registro de quién hizo cada operación.

6. Medio ambiente: Son los factores del entorno que afectan el proceso, como la alta demanda de llamadas en horas pico.

3) IDENTIFICAR LAS CAUSAS

1. Mano de obra

- **P2:** Registro duplicado o incorrecto de los datos de los pacientes.
- **P11:** El médico no conoce su agenda con anticipación.
- **P16:** Olvido de las citas por falta de recordatorios al paciente.

2. Métodos

- **P1:** Duplicación de citas para un mismo médico y horario.
- **P4:** Cancelaciones no registradas que mantienen horarios bloqueados.
- **P5:** Falta de control de las inasistencias de los pacientes.
- **P10:** Citas registradas fuera del horario de atención o en fechas pasadas.

3. Máquina (software)

- **P3:** Desconocimiento en tiempo real de los horarios disponibles de los médicos.
- **P6:** Demora en la búsqueda del historial de citas de un paciente.
- **P14:** Falta de control de acceso a la información de citas y pacientes.

4. Materiales (datos)

- **P9:** Asignación de citas a médicos inactivos o de vacaciones.
- **P12:** Pérdida o deterioro de los registros físicos de citas.
- **P17:** La información médica básica del paciente no está disponible para el médico.

5. Medición

- **P7:** Elaboración manual, lenta y con errores de los reportes de atenciones.
- **P8:** Falta de información actualizada para la dirección de la clínica.
- **P15:** Falta de trazabilidad de quién registró, modificó o canceló una cita.

6. Medio ambiente

- **P13:** Saturación de las llamadas telefónicas en horas pico.

4) ANALIZAR Y DISCUTIR EL DIAGRAMA

El diagrama muestra que la mayor cantidad de causas se concentra en las categorías Métodos y Máquina: el problema no es la falta de compromiso del personal, sino que el procedimiento manual y las herramientas actuales no impiden los errores. Las citas duplicadas (P1), las citas fuera de horario (P10) y las cancelaciones sin control (P4) son consecuencia de un método que no valida la disponibilidad, mientras que la falta de disponibilidad en tiempo real (P3) y el historial lento (P6) se deben a que el cuaderno y las hojas de cálculo no permiten consultar ni calcular información.

En la categoría Materiales, la pérdida de registros (P12) y los datos desactualizados de los médicos (P9) muestran la fragilidad de mantener la información en papel. En Medición, la ausencia de reportes automáticos y de trazabilidad (P7, P8 y P15) deja a la dirección sin información para decidir. Por ello, la solución debe actuar sobre el método y la herramienta a la vez: un sistema web que centralice los datos, valide las reglas del negocio en el servidor y en la base de datos, y genere la información de control de forma automática.

CAPÍTULO 2: FLUJO DE TRABAJO: CAPTURA DE REQUISITOS

La captura de requisitos se realizó a partir del caso de estudio proporcionado por la dirección de la clínica, la descripción de los roles de usuario, la revisión del proceso actual y las alternativas de cambio obtenidas con el método de Ishikawa. Su propósito es guiar el desarrollo hacia el sistema correcto, describiendo los requisitos de forma que el cliente y los desarrolladores lleguen a un acuerdo.

2.1 Identificar actores y casos de uso

2.1.1 Actores

A1. Administrador: Configura y supervisa el sistema: registra y administra usuarios, asigna roles y permisos, registra médicos, especialidades y horarios de atención, activa o desactiva registros y consulta los reportes generales.

A2. Recepcionista: Gestiona la atención de los pacientes: registra y busca pacientes, consulta médicos y horarios disponibles, programa, reprograma y cancela citas, registra la asistencia o inasistencia y consulta citas por fecha y estado.

A3. Médico: Consulta su agenda de atención, visualiza los datos básicos del paciente, identifica las citas pendientes, atendidas y canceladas, y marca una cita como atendida cuando tiene ese permiso.

A4. Paciente: Accede al portal del paciente: registra y actualiza su perfil (datos personales, contacto, información médica y contacto de emergencia), reserva citas y da seguimiento a su estado.

Todos los actores son especializaciones de «Usuario del sistema», que es quien inicia y cierra sesión.

2.1.2 Casos de uso

Los casos de uso identificados se agrupan por paquete:

Paquete P1 Seguridad

- CU01. Iniciar y cerrar sesión
- CU02. Gestionar usuarios
- CU03. Gestionar roles y permisos

Paquete P2 Personas

- CU04. Gestionar pacientes
- CU05. Gestionar médicos
- CU06. Gestionar especialidades

Paquete P3 Agenda

- CU07. Gestionar horarios de atención
- CU08. Consultar disponibilidad de citas

Paquete P4 Citas

- CU09. Programar cita médica
- CU10. Reprogramar cita
- CU11. Cancelar cita
- CU12. Registrar asistencia de la cita
- CU13. Consultar agenda médica
- CU14. Consultar historial de citas del paciente

Paquete P5 Reportes

- CU15. Generar reportes

Paquete P6 Portal del paciente

- CU16. Gestionar perfil del paciente
- CU17. Reservar y dar seguimiento a mis citas

2.2 Priorizar casos de uso

Los casos de uso se priorizaron según su importancia para la arquitectura y para el negocio. La prioridad alta corresponde a los casos de uso sin los cuales el sistema no cumple su propósito (seguridad, registros base, horarios y programación de citas). Con esta prioridad se distribuyeron en dos ciclos: el Ciclo #1 (9 de 17 casos de uso, 53 %) entrega un sistema que ya permite programar citas sin conflictos, y el Ciclo #2 completa el ciclo de vida de la cita, las consultas, los reportes y el portal del paciente.

Tabla 9. *Priorización de casos de uso por ciclo*

ID	Caso de uso	Paquete	Prioridad
CICLO #1 — Seguridad, registros base, horarios y programación de citas			
CU01	Iniciar y cerrar sesión	Seguridad	Alta
CU02	Gestionar usuarios	Seguridad	Alta
CU03	Gestionar roles y permisos	Seguridad	Media
CU04	Gestionar pacientes	Personas	Alta
CU05	Gestionar médicos	Personas	Alta
CU06	Gestionar especialidades	Personas	Media
CU07	Gestionar horarios de atención	Agenda	Alta

ID	Caso de uso	Paquete	Prioridad
CU08	Consultar disponibilidad de citas	Agenda	Alta
CU09	Programar cita médica	Citas	Alta
CICLO #2 — Gestión de citas, consultas, reportes y portal del paciente			
CU10	Reprogramar cita	Citas	Alta
CU11	Cancelar cita	Citas	Alta
CU12	Registrar asistencia de la cita	Citas	Alta
CU13	Consultar agenda médica	Citas	Alta
CU14	Consultar historial de citas del paciente	Citas	Media
CU15	Generar reportes	Reportes	Media
CU16	Gestionar perfil del paciente	Portal del paciente	Media
CU17	Reservar y dar seguimiento a mis citas	Portal del paciente	Media

2.3 Detallar casos de uso

Para cada caso de uso se presenta su diagrama y su especificación: propósito, actores, actor iniciador, flujo principal, precondición, postcondición y excepciones.

CICLO # 1

CU01: Iniciar y cerrar sesión

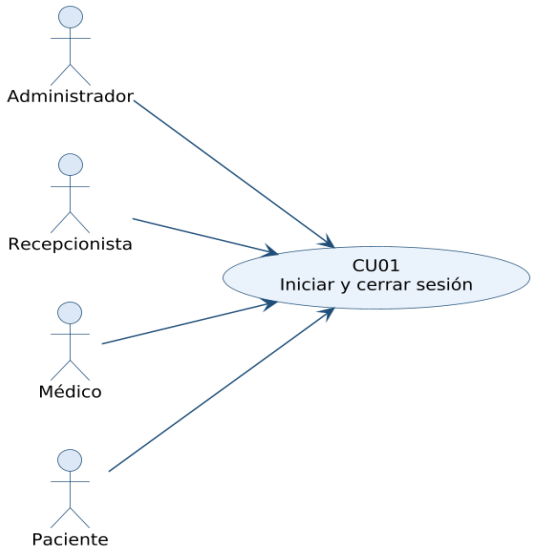


Figura 7. *Diagrama de caso de uso — CU01 Iniciar y cerrar sesión*

Tabla 10. *Especificación del caso de uso CU01*

Caso de uso	CU01: Iniciar y cerrar sesión
Propósito	Permitir que un usuario autorizado ingrese al sistema con su nombre de usuario o correo y su contraseña, acceda solo a las opciones de su rol y cierre su sesión de forma segura.
Actores	Administrador, Recepcionista, Médico, Paciente
Actor iniciador	Usuario del sistema (cualquier rol)
Precondición	El usuario tiene una cuenta registrada con estado «activo».
Flujo principal	1. El usuario ingresa a la página de inicio de sesión. 2. Introduce su nombre de usuario o correo electrónico y su contraseña. 3. El sistema valida los campos en el cliente (JavaScript) y en el servidor (PHP). 4. El sistema busca la cuenta y verifica la contraseña con password_verify() contra el hash almacenado. 5. El sistema regenera el identificador de sesión (session_regenerate_id), guarda el usuario y su rol en $_SESSION y registra el último acceso en la bitácora. 6. El sistema muestra el menú principal con las opciones permitidas para el rol. 7. Al terminar, el usuario selecciona «Cerrar sesión»; el sistema destruye la sesión y lo redirige al inicio de sesión.
Postcondición	El usuario accede únicamente a las funcionalidades de su rol; el acceso queda registrado en la bitácora.
Excepciones	• Usuario o contraseña incorrectos: se muestra un mensaje genérico, sin indicar cuál dato falló. • Cuenta inactiva o bloqueada (tras 5 intentos fallidos): se informa que el acceso no está permitido. • Acceso a una página interna sin sesión válida: el sistema redirige al inicio de sesión.

CU02: Gestionar usuarios

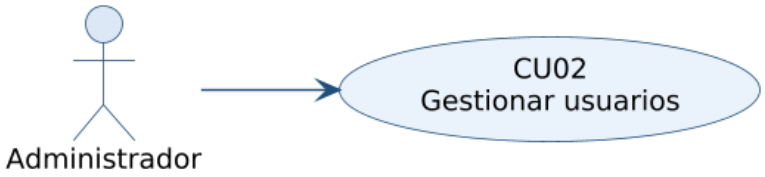


Figura 8. *Diagrama de caso de uso — CU02 Gestionar usuarios*

Tabla 11. *Especificación del caso de uso CU02*

Caso de uso	CU02: Gestionar usuarios
Propósito	Registrar, consultar, modificar, activar y desactivar las cuentas de usuario del personal y de los pacientes, asignándoles un rol.
Actores	Administrador
Actor iniciador	Administrador
Precondición	El administrador inició sesión (CU01).
Flujo principal	1. El administrador ingresa a la opción «Usuarios».

	2. El sistema lista los usuarios con su rol y estado, con búsqueda por nombre de usuario o correo. 3. El administrador elige «Nuevo usuario» e introduce nombre de usuario, correo, contraseña inicial y rol; si el rol es Médico o Paciente, vincula la cuenta con el registro correspondiente. 4. El sistema valida que el usuario y el correo no existan, genera el hash con password_hash() y guarda la cuenta. 5. Para modificar, el administrador selecciona un usuario, cambia sus datos o su rol y guarda. 6. Para desactivar o activar, el administrador cambia el estado; la cuenta no se elimina. 7. El sistema registra cada operación en la bitácora y muestra un mensaje de confirmación.
Postcondición	La cuenta queda registrada o actualizada con su rol; el cambio queda en la bitácora.
Excepciones	• Nombre de usuario o correo ya registrado. • Contraseña que no cumple la longitud mínima (8 caracteres, letras y números). • Intento de desactivar la propia cuenta del administrador en sesión.

CU03: Gestionar roles y permisos



Figura 9. *Diagrama de caso de uso — CU03 Gestionar roles y permisos*

Tabla 12. *Especificación del caso de uso CU03*

Caso de uso	CU03: Gestionar roles y permisos
Propósito	Definir qué operaciones puede realizar cada rol (Administrador, Recepcionista, Médico y Paciente), por ejemplo autorizar al médico a marcar citas como atendidas.
Actores	Administrador
Actor iniciador	Administrador
Precondición	El administrador inició sesión (CU01).
Flujo principal	1. El administrador ingresa a la opción «Roles y permisos». 2. El sistema muestra los roles y, para el rol seleccionado, la lista de permisos con casillas de verificación. 3. El administrador marca o desmarca los permisos del rol. 4. El sistema guarda la asignación en una transacción y registra el cambio en la bitácora. 5. Los nuevos permisos se aplican en el siguiente inicio de sesión de los usuarios del rol.
Postcondición	El rol queda con el conjunto de permisos actualizado.
Excepciones	• Intento de quitar al rol Administrador el permiso de gestionar roles (se impide quedar sin administrador). • Rol inactivo: no se le pueden asignar permisos.

CU04: Gestionar pacientes

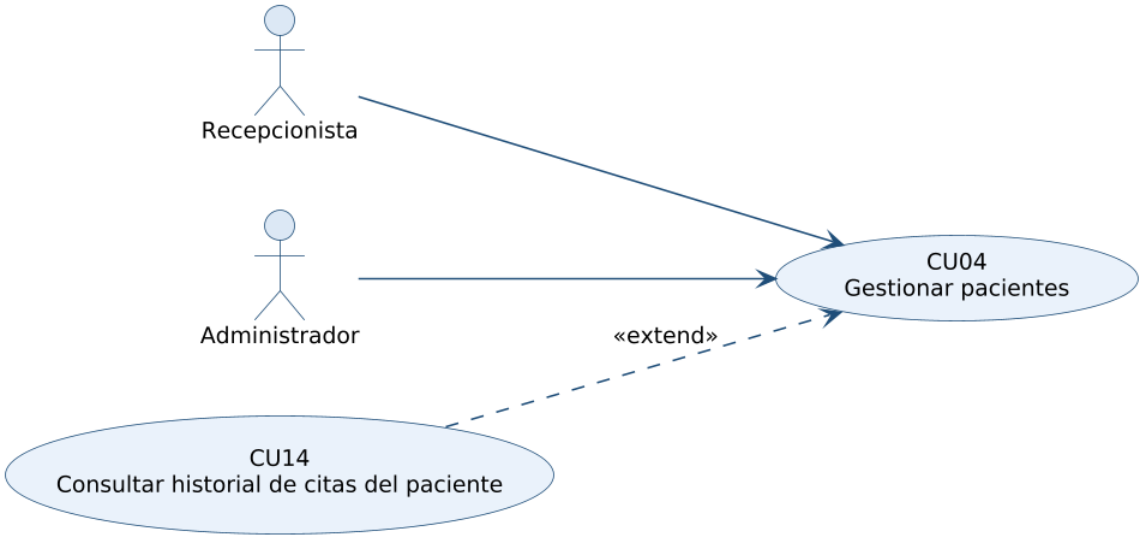


Figura 10. *Diagrama de caso de uso — CU04 Gestionar pacientes*

Tabla 13. *Especificación del caso de uso CU04*

Caso de uso	CU04: Gestionar pacientes
Propósito	Registrar, listar, buscar, modificar y desactivar pacientes, y consultar sus citas anteriores, sin perder el historial asociado.
Actores	Recepcionista, Administrador
Actor iniciador	Recepcionista
Precondición	La recepcionista o el administrador inició sesión (CU01) y tiene el permiso paciente.gestionar.
Flujo principal	1. La recepcionista ingresa a la opción «Pacientes». 2. El sistema lista los pacientes activos y permite buscar por CI, nombre o apellido. 3. Para registrar, la recepcionista selecciona «Nuevo paciente» e ingresa CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono, correo (opcional), dirección (opcional) y contacto de emergencia. 4. El sistema valida los datos en el cliente y en el servidor, verifica que el CI no esté registrado, genera el código (PAC-00001) y guarda el paciente. 5. Para modificar, selecciona un paciente, actualiza sus datos y guarda. 6. Para desactivar, selecciona «Desactivar»; el sistema cambia el estado a «inactivo» sin eliminar sus citas. 7. La recepcionista puede seleccionar «Ver citas» para consultar el historial del paciente (CU14).
Postcondición	El paciente queda registrado, actualizado o inactivo; su historial de citas se conserva.
Excepciones	• CI ya registrado: el sistema rechaza el duplicado (RN01). • Campos obligatorios vacíos o con formato inválido (teléfono, correo, fecha de nacimiento futura). • Paciente no encontrado en la búsqueda.
Extendido por	CU14 Consultar historial de citas del paciente

CU05: Gestionar médicos

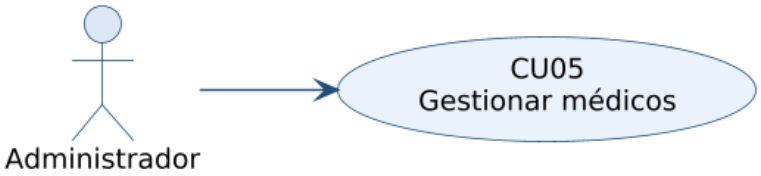


Figura 11. *Diagrama de caso de uso — CU05 Gestionar médicos*

Tabla 14. *Especificación del caso de uso CU05*

Caso de uso	CU05: Gestionar médicos
Propósito	Registrar, consultar, buscar, modificar, activar y desactivar médicos con sus especialidades, conservando sus citas históricas.
Actores	Administrador
Actor iniciador	Administrador
Precondición	El administrador inició sesión (CU01). Existen especialidades activas (CU06).
Flujo principal	1. El administrador ingresa a la opción «Médicos». 2. El sistema lista los médicos con su especialidad y estado; permite buscar por nombre o especialidad. 3. Para registrar, el administrador ingresa número de registro profesional, nombres, apellidos, teléfono, correo y selecciona una o varias especialidades. 4. El sistema valida que el registro profesional no exista, genera el código (MED-0001) y guarda el médico y sus especialidades en una transacción. 5. Opcionalmente crea la cuenta de usuario con rol Médico (CU02). 6. Para modificar, actualiza datos o especialidades y guarda. 7. Para desactivar, cambia el estado a «inactivo»; el médico deja de recibir nuevas citas pero sus citas históricas se conservan.
Postcondición	El médico queda registrado o actualizado con sus especialidades.
Excepciones	• Registro profesional duplicado. • No se seleccionó ninguna especialidad. • Desactivar un médico con citas futuras programadas: el sistema advierte y lista las citas a reprogramar.

CU06: Gestionar especialidades

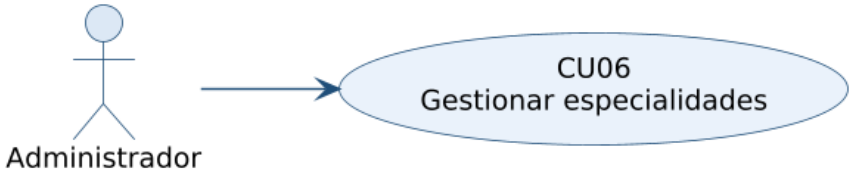


Figura 12. *Diagrama de caso de uso — CU06 Gestionar especialidades*

Tabla 15. *Especificación del caso de uso CU06*

Caso de uso	CU06: Gestionar especialidades
Propósito	Administrar las especialidades que ofrece la clínica (medicina general, pediatría, ginecología, odontología, cardiología, etc.).
Actores	Administrador
Actor iniciador	Administrador
Precondición	El administrador inició sesión (CU01).
Flujo principal	1. El administrador ingresa a la opción «Especialidades». 2. El sistema lista las especialidades con su estado. 3. El administrador registra una especialidad con nombre y descripción opcional. 4. El sistema normaliza el nombre (sin espacios extremos y sin distinguir mayúsculas), verifica que no exista y guarda. 5. El administrador puede modificar la descripción o activar/desactivar la especialidad.
Postcondición	La especialidad queda disponible para asignarla a médicos y programar citas.
Excepciones	• Nombre normalizado duplicado (p. ej. «PEDIATRÍA» ya existe como «Pediatría»). • Nombre vacío. • Desactivar una especialidad con médicos activos: se solicita confirmación.

CU07: Gestionar horarios de atención



Figura 13. *Diagrama de caso de uso — CU07 Gestionar horarios de atención*

Tabla 16. *Especificación del caso de uso CU07*

Caso de uso	CU07: Gestionar horarios de atención
Propósito	Registrar los bloques de atención semanales de cada médico (día, hora inicial, hora final y duración de la cita) y las fechas bloqueadas o feriados.
Actores	Administrador
Actor iniciador	Administrador
Precondición	El administrador inició sesión (CU01). El médico está registrado y activo (CU05).
Flujo principal	1. El administrador ingresa a «Horarios de atención» y selecciona un médico activo. 2. El sistema muestra los bloques semanales registrados del médico. 3. El administrador agrega un bloque indicando día de la semana, hora inicial, hora final y duración de la cita (p. ej. 30 minutos). 4. El sistema valida que la hora final sea posterior a la inicial y que el bloque no se superponga con otro del mismo médico y día, y lo guarda. 5. Opcionalmente, el administrador registra una fecha bloqueada (vacación o feriado) para el médico o para toda la clínica. 6. El sistema muestra la vista previa de los intervalos que se ofrecerán.

Postcondición	El horario queda registrado y se usa para calcular la disponibilidad (CU08).
Excepciones	• Hora final menor o igual a la hora inicial. • Bloque superpuesto con otro horario del mismo médico. • Duración no admitida (valores permitidos: 15, 20, 30, 45 o 60 minutos).

CU08: Consultar disponibilidad de citas

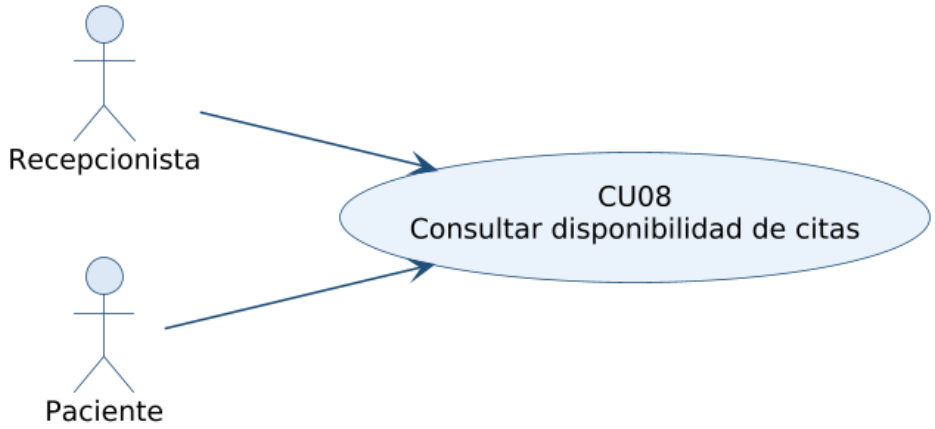


Figura 14. *Diagrama de caso de uso — CU08 Consultar disponibilidad de citas*

Tabla 17. *Especificación del caso de uso CU08*

Caso de uso	CU08: Consultar disponibilidad de citas
Propósito	Mostrar los horarios libres de un médico en una fecha, calculados a partir de su horario de atención, la duración configurada y las citas ya ocupadas.
Actores	Recepcionista, Paciente
Actor iniciador	Recepcionista o Paciente
Precondición	El usuario inició sesión (CU01). Existen médicos con horarios registrados (CU07).
Flujo principal	1. El usuario selecciona una especialidad. 2. El sistema lista los médicos activos de esa especialidad. 3. El usuario selecciona un médico y una fecha (no pasada). 4. El sistema obtiene el horario de atención del día de la semana, divide el bloque en intervalos según la duración configurada y descarta los intervalos pasados, bloqueados u ocupados por citas no canceladas. 5. El sistema muestra únicamente los horarios disponibles.
Postcondición	El usuario conoce los horarios disponibles para programar una cita (CU09 / CU17).
Excepciones	• Fecha pasada: el sistema no la permite. • El médico no atiende ese día o la fecha está bloqueada: se informa «Sin horarios disponibles». • Todos los intervalos están ocupados.

CU09: Programar cita médica

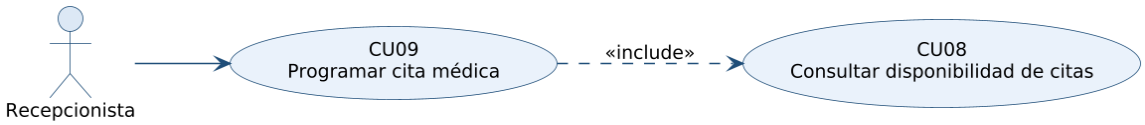


Figura 15. *Diagrama de caso de uso — CU09 Programar cita médica*

Tabla 18. *Especificación del caso de uso CU09*

Caso de uso	CU09: Programar cita médica
Propósito	Registrar una cita para un paciente con un médico de una especialidad en una fecha y hora disponibles, impidiendo citas duplicadas o superpuestas.
Actores	Recepcionista
Actor iniciador	Recepcionista
Precondición	La recepcionista inició sesión (CU01). El paciente está registrado y activo (CU04) y el médico tiene horario (CU07).
Flujo principal	1. La recepcionista ingresa a «Programar cita». 2. Busca y selecciona al paciente por CI o nombre (CU04). 3. Selecciona la especialidad, el médico y la fecha; el sistema muestra solo los horarios disponibles (CU08). 4. Selecciona la hora disponible, la modalidad e ingresa un motivo breve (opcional). 5. Al guardar, el sistema inicia una transacción y vuelve a verificar en el servidor que el paciente y el médico estén activos, que la hora esté dentro del horario de atención, que no sea una fecha pasada y que no exista otra cita superpuesta. 6. El sistema genera el código único (CM-0001), registra la fecha de creación y el estado «Programada», y confirma la transacción. 7. El sistema muestra el comprobante de la cita (código, paciente, especialidad, médico, fecha, horario y motivo).
Postcondición	La cita queda registrada en estado «Programada» y el intervalo deja de estar disponible.
Excepciones	• Otro usuario ocupó el horario mientras se completaba el formulario: el sistema rechaza la operación y muestra «El horario ya no está disponible» (RN07, RN13). • Hora fuera del horario de atención o fecha pasada (RN05, RN06). • Médico o paciente inactivo (RN04).
Incluye	CU08 Consultar disponibilidad de citas

CICLO # 2

CU10: Reprogramar cita

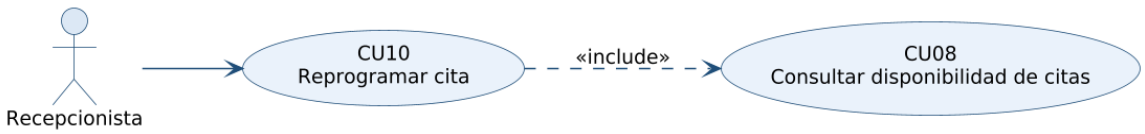


Figura 16. *Diagrama de caso de uso — CU10 Reprogramar cita*

Tabla 19. *Especificación del caso de uso CU10*

Caso de uso	CU10: Reprogramar cita
Propósito	Cambiar la fecha y hora de una cita Programada o Confirmada a un nuevo horario disponible del médico, manteniendo la integridad y la trazabilidad.
Actores	Recepcionista
Actor iniciador	Recepcionista
Precondición	La recepcionista inició sesión (CU01). La cita existe en estado Programada o Confirmada.
Flujo principal	1. La recepcionista busca la cita por código o por paciente. 2. El sistema recupera los datos de la cita y verifica que su estado sea «Programada» o «Confirmada». 3. La recepcionista selecciona una nueva fecha; el sistema muestra los horarios disponibles del médico (CU08). 4. La recepcionista elige el nuevo horario y confirma. 5. El sistema, dentro de una transacción, verifica nuevamente la disponibilidad, actualiza la fecha y hora y registra en el historial la fecha y hora anteriores. 6. El sistema muestra el comprobante actualizado.
Postcondición	La cita queda con la nueva fecha y hora; el horario anterior se libera y el cambio queda en el historial.
Excepciones	• Cita Atendida, Cancelada o con Inasistencia: no se puede reprogramar directamente (política: registrar una nueva cita). • El nuevo horario fue ocupado por otra cita. • Nueva fecha pasada o fuera del horario de atención.
Incluye	CU08 Consultar disponibilidad de citas

CU11: Cancelar cita

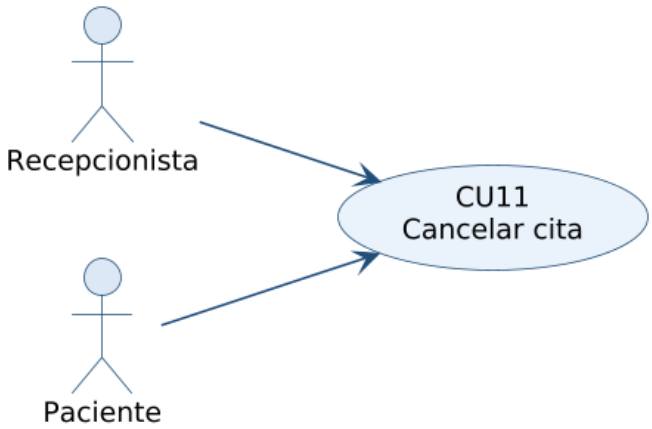


Figura 17. *Diagrama de caso de uso — CU11 Cancelar cita*

Tabla 20. *Especificación del caso de uso CU11*

Caso de uso	CU11: Cancelar cita
Propósito	Cancelar una cita Programada o Confirmada registrando el motivo, sin eliminar físicamente el registro y liberando el horario.

Actores	Recepcionista, Paciente
Actor iniciador	Recepcionista (o el Paciente desde el portal)
Precondición	El usuario inició sesión (CU01). La cita existe en estado Programada o Confirmada.
Flujo principal	1. El usuario busca la cita por código o paciente. 2. El sistema muestra los datos y verifica que el estado sea «Programada» o «Confirmada». 3. El usuario selecciona «Cancelar» e ingresa el motivo de cancelación. 4. El sistema solicita confirmación. 5. El sistema cambia el estado a «Cancelada», guarda el motivo y registra el cambio en el historial. 6. El intervalo queda nuevamente disponible y la cita permanece en las consultas históricas.
Postcondición	La cita queda «Cancelada», deja de ocupar el horario (RN09) y se conserva en el historial.
Excepciones	• Cita Atendida o con Inasistencia: no puede cancelarse como si nunca hubiera ocurrido. • Motivo de cancelación vacío. • El paciente intenta cancelar una cita que no le pertenece.

CU12: Registrar asistencia de la cita

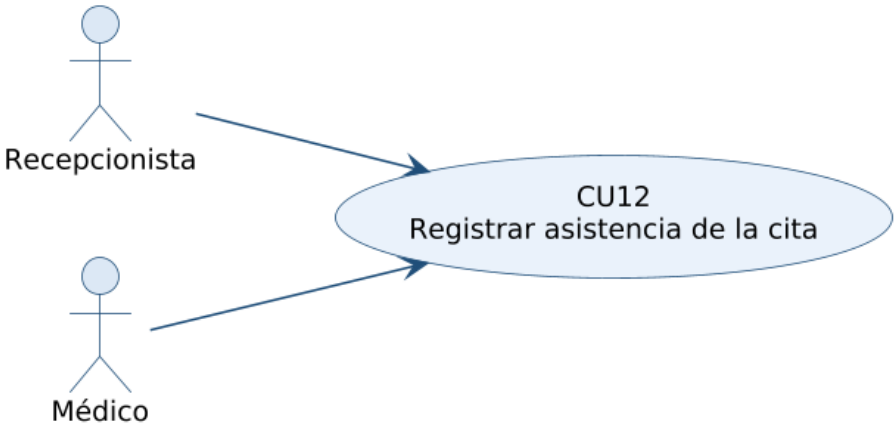


Figura 18. *Diagrama de caso de uso — CU12 Registrar asistencia de la cita*

Tabla 21. *Especificación del caso de uso CU12*

Caso de uso	CU12: Registrar asistencia de la cita
Propósito	Controlar los estados de la cita: confirmar la asistencia, marcarla como Atendida o registrar la Inasistencia del paciente, aplicando solo transiciones válidas.
Actores	Recepcionista, Médico
Actor iniciador	Recepcionista o Médico
Precondición	El usuario inició sesión (CU01) y tiene el permiso correspondiente (cita.asistencia o cita.atender).
Flujo principal	1. El usuario abre la agenda del día (CU13) y selecciona una cita. 2. El sistema muestra el estado actual y solo las acciones válidas para ese estado. 3. La recepcionista confirma la llegada del paciente («Confirmada») o registra «Inasistencia» si no se presentó.

	4. El médico, si tiene el permiso cita.atender, marca la cita como «Atendida» al finalizar la consulta. 5. El sistema valida la transición, actualiza el estado y registra el cambio en el historial con el usuario y la fecha.
Postcondición	La cita queda en el nuevo estado y el cambio es trazable.
Excepciones	• Transición no permitida (p. ej. Cancelada → Atendida) sin un procedimiento autorizado. • Marcar Atendida o Inasistencia una cita cuya hora aún no llegó. • Médico sin permiso cita.atender.

CU13: Consultar agenda médica

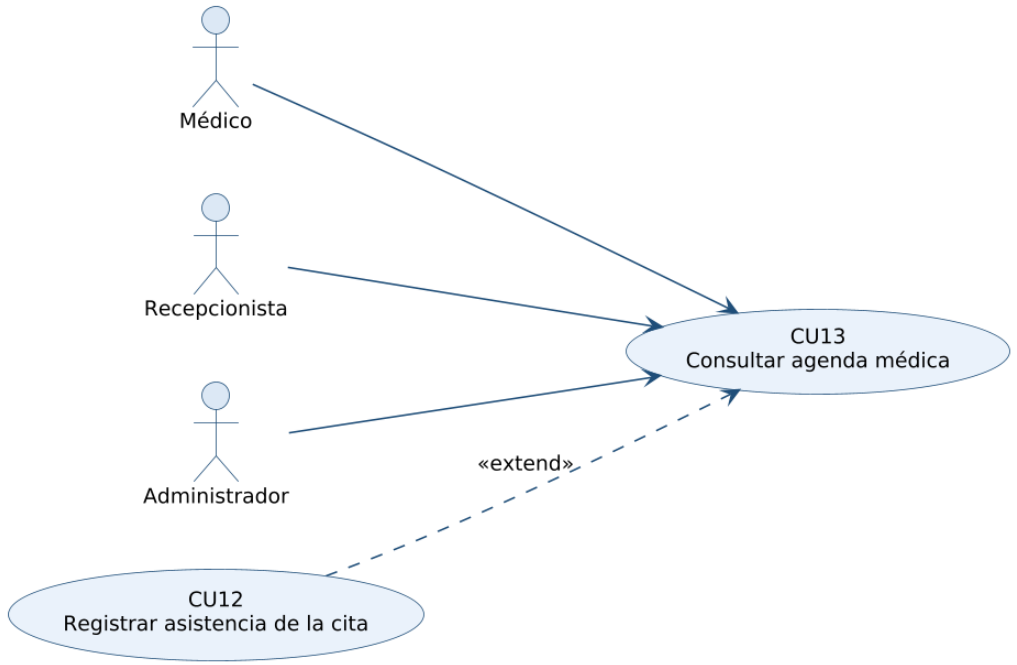


Figura 19. *Diagrama de caso de uso — CU13 Consultar agenda médica*

Tabla 22. *Especificación del caso de uso CU13*

Caso de uso	CU13: Consultar agenda médica
Propósito	Consultar la agenda de un médico para una fecha, con hora, código, paciente, especialidad, estado y motivo (si el usuario tiene permiso), filtrando por médico, fecha y estado.
Actores	Médico, Recepcionista, Administrador
Actor iniciador	Médico
Precondición	El usuario inició sesión (CU01) y tiene el permiso agenda.consultar.
Flujo principal	1. El usuario ingresa a «Agenda médica». 2. Si el usuario es Médico, el sistema fija su propio médico; la recepcionista o el administrador seleccionan el médico. 3. El usuario selecciona la fecha y, opcionalmente, un estado. 4. El sistema consulta las citas y las muestra ordenadas por hora, con colores por estado (pendiente, atendida, cancelada).

	5. El usuario puede seleccionar una cita para ver los datos básicos del paciente o registrar su asistencia (CU12).
Postcondición	Se muestra la agenda filtrada; no se modifica información.
Excepciones	• No hay citas para los filtros seleccionados. • Un médico intenta ver la agenda de otro médico: se deniega.
Extendido por	CU12 Registrar asistencia de la cita

CU14: Consultar historial de citas del paciente

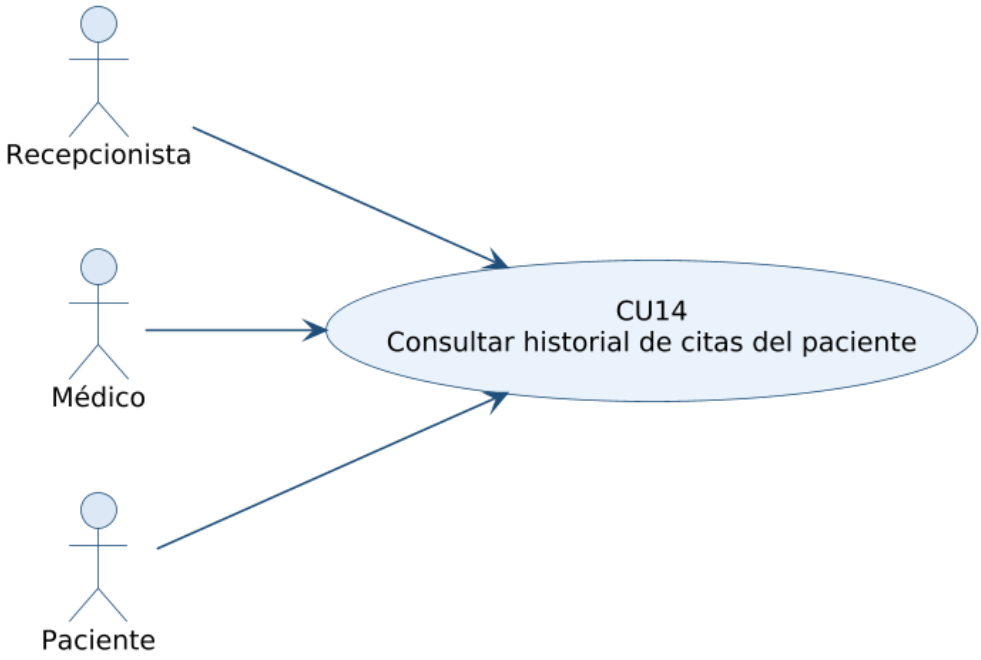


Figura 20. *Diagrama de caso de uso — CU14 Consultar historial de citas del paciente*

Tabla 23. *Especificación del caso de uso CU14*

Caso de uso	CU14: Consultar historial de citas del paciente
Propósito	Visualizar todas las citas de un paciente (pasadas, futuras y canceladas) con fecha y hora, médico, especialidad, estado y fecha de registro, sin perder trazabilidad.
Actores	Recepcionista, Médico, Paciente
Actor iniciador	Recepcionista
Precondición	El usuario inició sesión (CU01).
Flujo principal	1. El usuario busca al paciente por CI o nombre (el paciente ve directamente el suyo). 2. El sistema muestra los datos básicos del paciente. 3. El sistema lista su historial de citas ordenado de la más reciente a la más antigua. 4. El usuario puede filtrar por estado o rango de fechas y ver los cambios de estado de una cita.
Postcondición	Se muestra el historial completo del paciente.

Excepciones	• El paciente no tiene citas registradas. • Un paciente intenta consultar el historial de otro paciente: se deniega.

CU15: Generar reportes

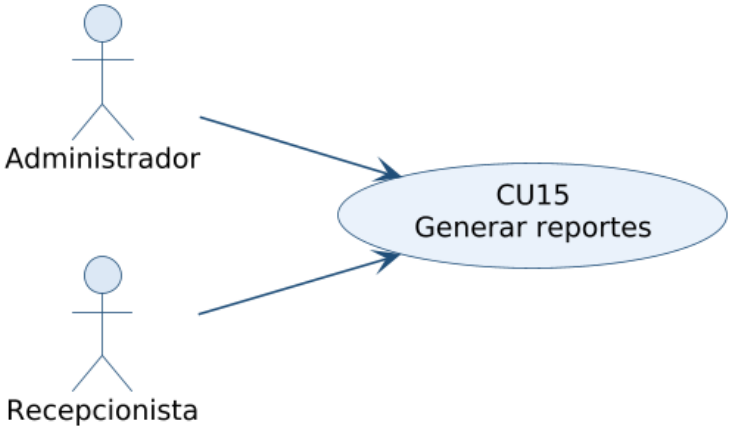


Figura 21. *Diagrama de caso de uso — CU15 Generar reportes*

Tabla 24. *Especificación del caso de uso CU15*

Caso de uso	CU15: Generar reportes
Propósito	Generar los reportes administrativos: citas por fecha, por médico, por especialidad y por estado; pacientes registrados; cantidad de citas atendidas, canceladas e inasistencias en un período; y agenda diaria de cada médico.
Actores	Administrador, Recepcionista
Actor iniciador	Administrador
Precondición	El usuario inició sesión (CU01) y tiene el permiso reporte.generar.
Flujo principal	1. El usuario ingresa a «Reportes» y elige el tipo de reporte. 2. Selecciona los filtros: fecha o rango de fechas, médico, especialidad y/o estado. 3. El sistema valida que la fecha inicial no sea mayor que la final y ejecuta la consulta parametrizada. 4. El sistema muestra el reporte en una tabla con totales. 5. Opcionalmente, el usuario exporta el reporte a CSV o PDF, o lo imprime.
Postcondición	Se obtiene el reporte solicitado con sus totales.
Excepciones	• Rango de fechas inválido. • No existen datos para los filtros seleccionados.

CU16: Gestionar perfil del paciente



Figura 22. *Diagrama de caso de uso — CU16 Gestionar perfil del paciente*

Tabla 25. *Especificación del caso de uso CU16*

Caso de uso	CU16: Gestionar perfil del paciente
Propósito	Permitir al paciente registrarse en el portal y mantener actualizado su perfil: datos personales, contacto, información médica (tipo de sangre, alergias, enfermedades crónicas, medicamentos, antecedentes), contacto de emergencia y preferencias de la cuenta.
Actores	Paciente
Actor iniciador	Paciente
Precondición	El paciente tiene un CI válido; para editar, inició sesión (CU01).
Flujo principal	1. El paciente ingresa al portal y elige «Crear cuenta» o inicia sesión (CU01). 2. Al crear la cuenta, ingresa su CI; si ya está registrado por recepción, el sistema vincula la cuenta a su registro; si no, crea el paciente. 3. El paciente abre «Mi perfil»; el sistema muestra las secciones del perfil. 4. El paciente actualiza sus datos personales, de contacto, información médica y contacto de emergencia. 5. El paciente cambia su contraseña o sus preferencias de notificaciones y privacidad. 6. El sistema valida y guarda los cambios.
Postcondición	El perfil del paciente queda actualizado.
Excepciones	• El CI ya tiene una cuenta de portal vinculada. • Datos con formato inválido. • Contraseña actual incorrecta al cambiarla.

CU17: Reservar y dar seguimiento a mis citas

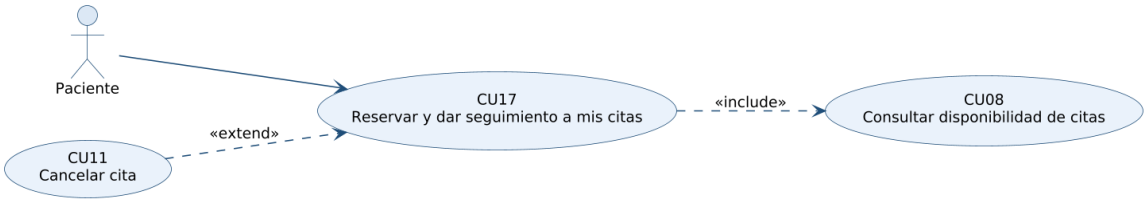


Figura 23. *Diagrama de caso de uso — CU17 Reservar y dar seguimiento a mis citas*

Tabla 26. *Especificación del caso de uso CU17*

Caso de uso	CU17: Reservar y dar seguimiento a mis citas
Propósito	Permitir que el paciente reserve una cita en línea buscando médicos por especialidad y disponibilidad, y dé seguimiento a sus citas pendientes, confirmadas, completadas y canceladas.

Actores	Paciente
Actor iniciador	Paciente
Precondición	El paciente inició sesión en el portal (CU01) y su registro está activo.
Flujo principal	1. El paciente ingresa a «Reservar cita». 2. Selecciona especialidad, médico y fecha; el sistema muestra los horarios disponibles (CU08). 3. Elige un horario, la modalidad y un motivo breve, y confirma. 4. El sistema registra la cita con las mismas validaciones del servidor que CU09. 5. En «Mis citas», el sistema lista sus citas por estado y muestra un recordatorio de las próximas consultas. 6. Desde «Mis citas» el paciente puede cancelar una cita programada (CU11).
Postcondición	La cita queda «Programada» a nombre del paciente; el paciente ve el estado de todas sus citas.
Excepciones	• El horario fue ocupado por otra reserva. • El paciente ya tiene una cita pendiente con el mismo médico en la misma fecha.
Incluye	CU08 Consultar disponibilidad de citas
Extendido por	CU11 Cancelar cita

2.4 Prototipar la interfaz de usuario

Los prototipos muestran la distribución de las pantallas principales: un menú lateral con las opciones permitidas para el rol, el encabezado con el usuario en sesión y el contenido del caso de uso. Se incluyen ejemplos de mensajes de validación (CI duplicado, credenciales incorrectas) y los colores de estado de las citas. Los datos que aparecen son ficticios.

Ciclo #1

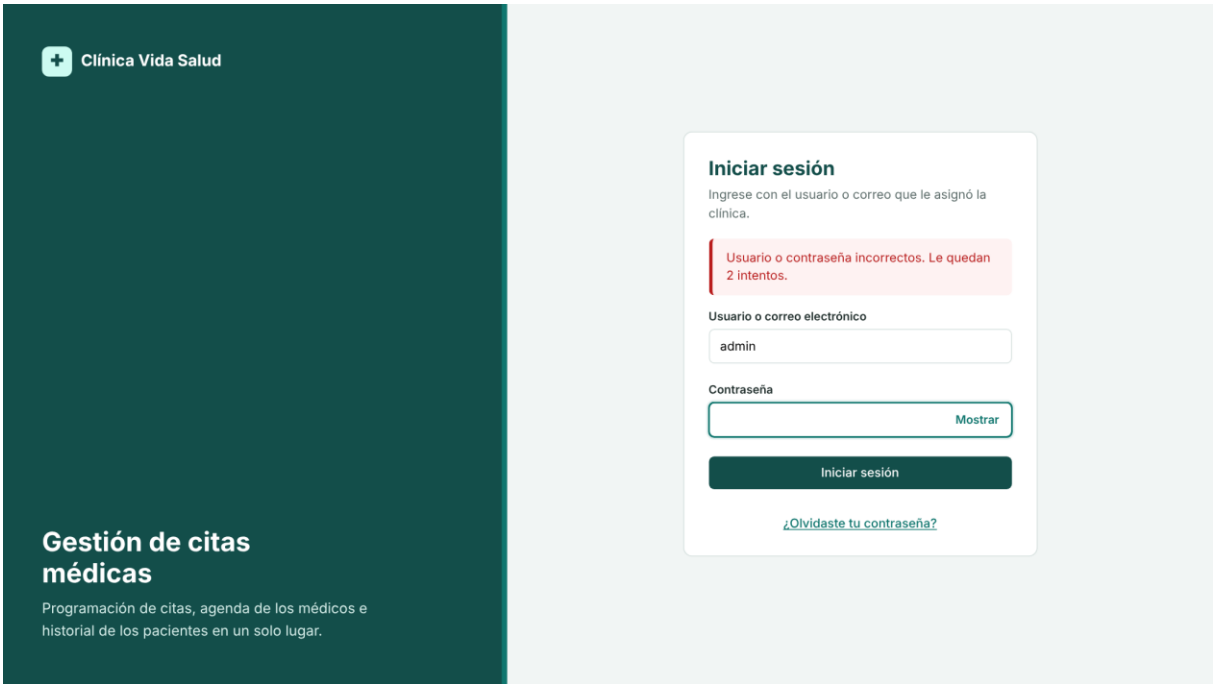


Figura 24. *Prototipo — CU01 Iniciar sesión (mensaje de credenciales incorrectas)*

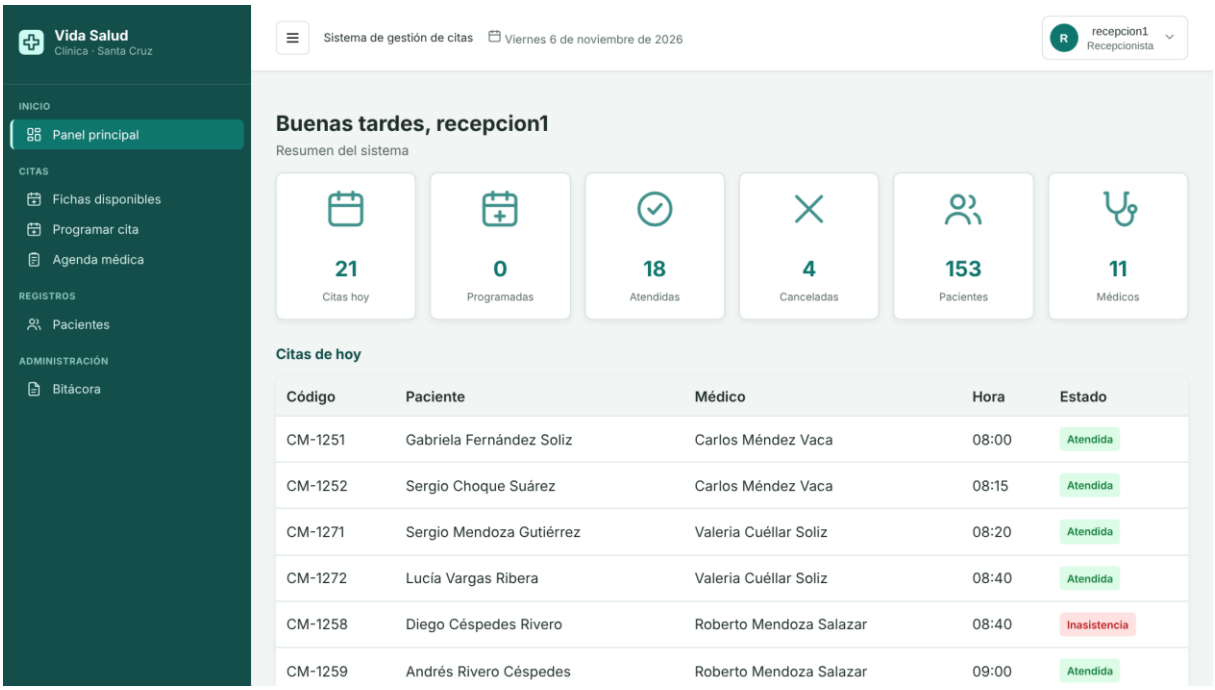


Figura 25. *Prototipo — Panel principal (resumen del día)*

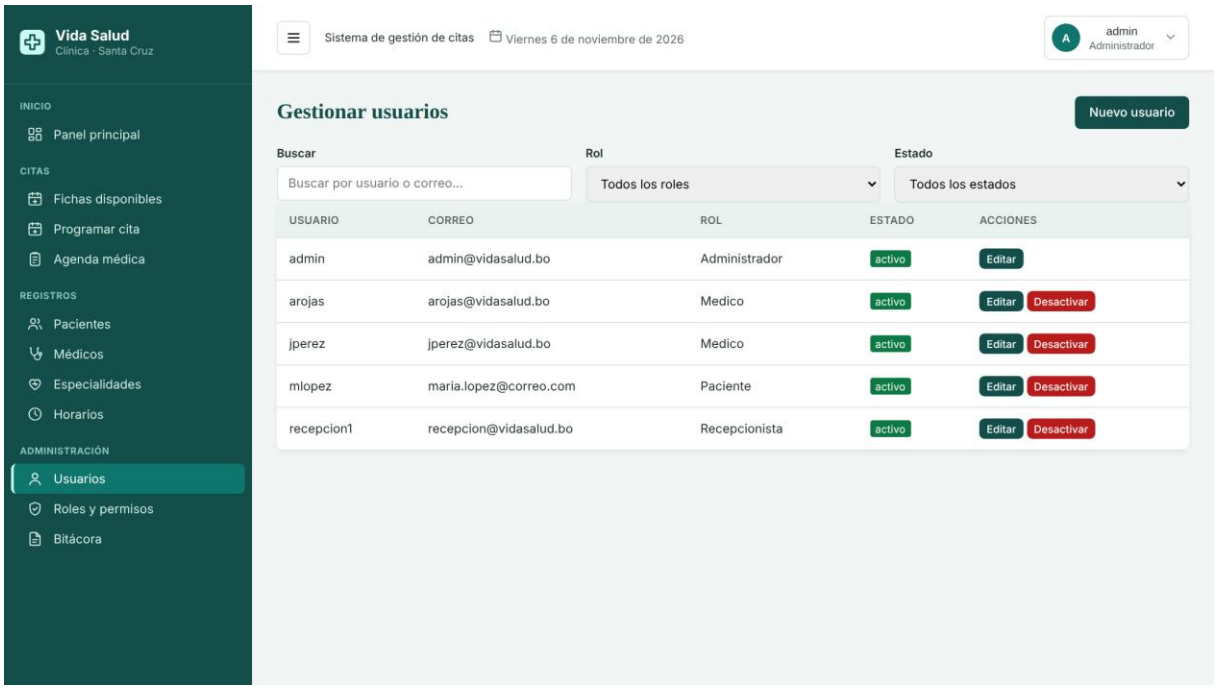


Figura 26. *Prototipo — CU02 Gestionar usuarios*

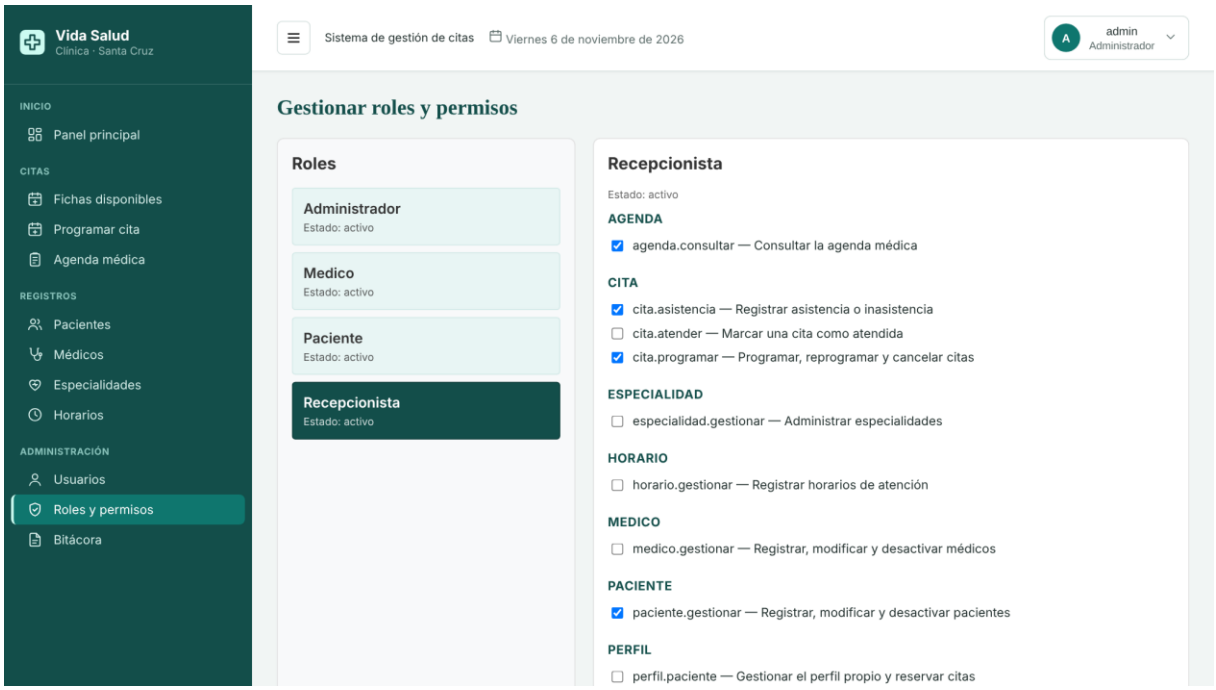


Figura 27. *Prototipo — CU03 Gestionar roles y permisos*

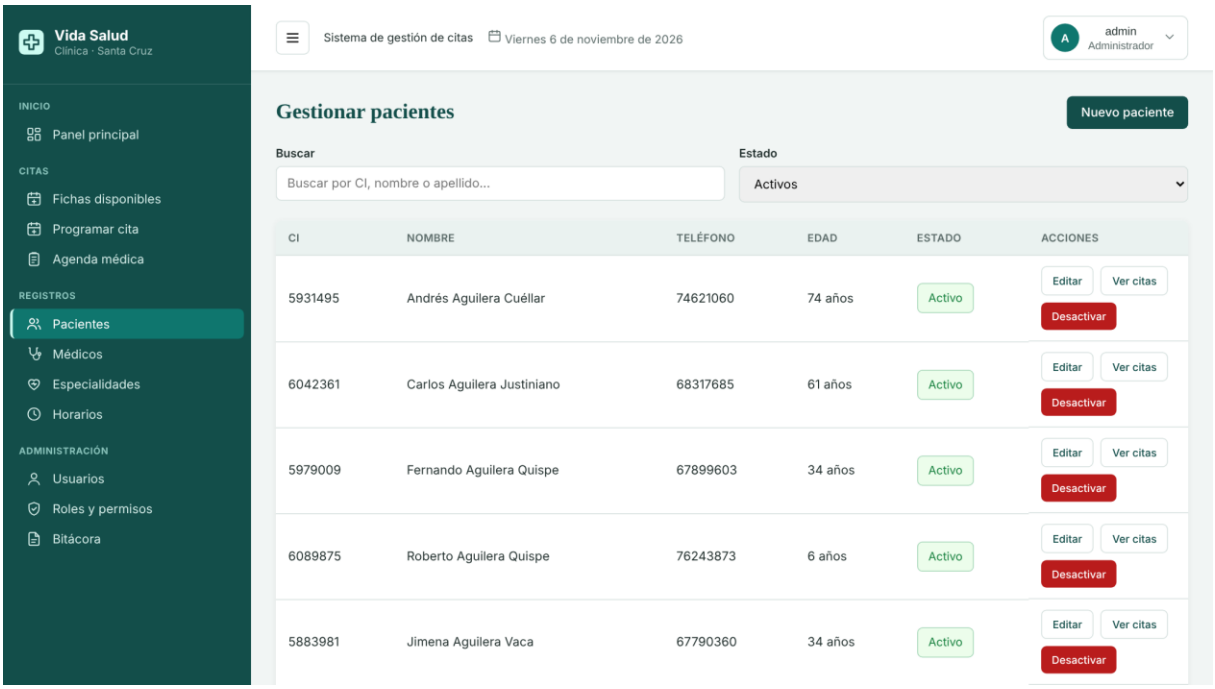


Figura 28. *Prototipo — CU04 Gestionar pacientes*

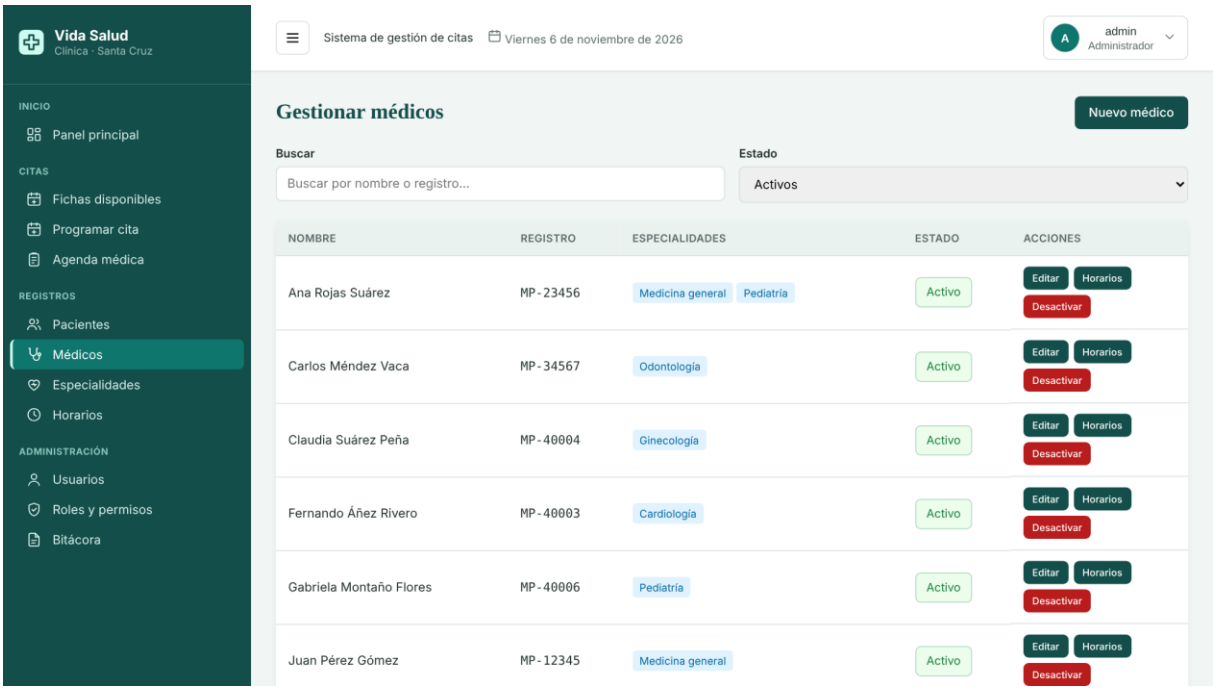


Figura 29. *Prototipo — CU05 Gestionar médicos*



Figura 30. *Prototipo — CU06 Gestionar especialidades*

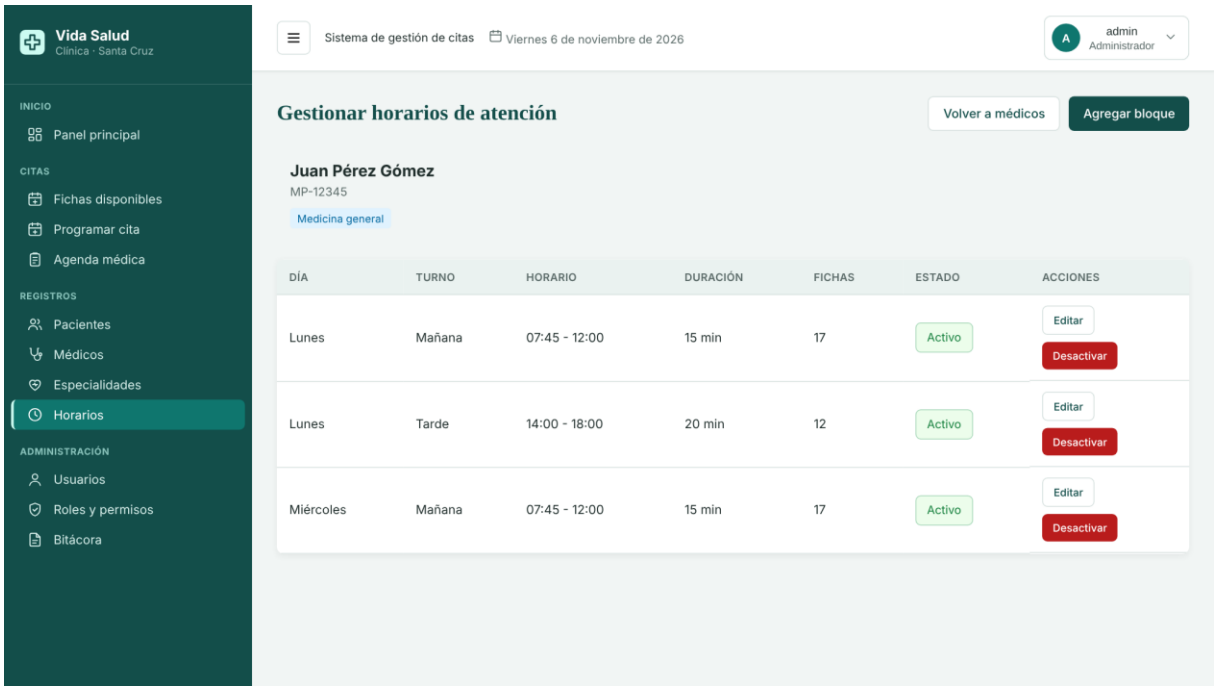


Figura 31. *Prototipo — CU07 Gestionar horarios de atención*

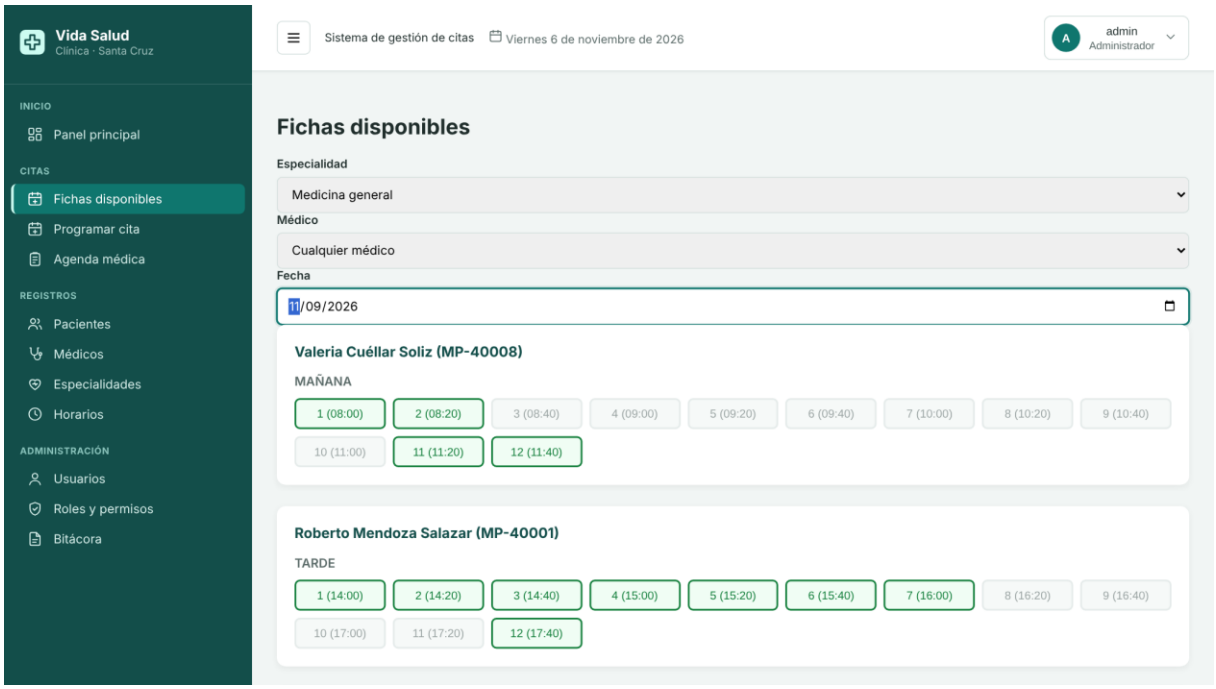


Figura 32. *Prototipo — CU08 Consultar fichas disponibles*

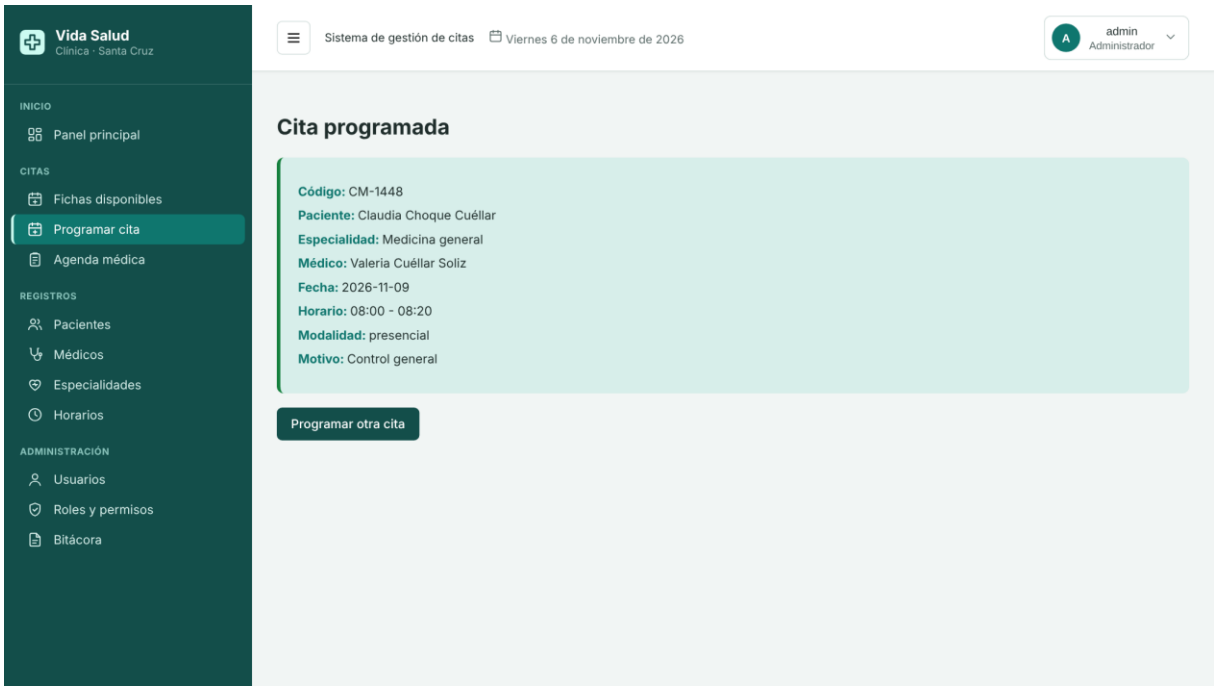


Figura 33. *Prototipo — CU09 Programar cita médica (comprobante)*

Ciclo #2

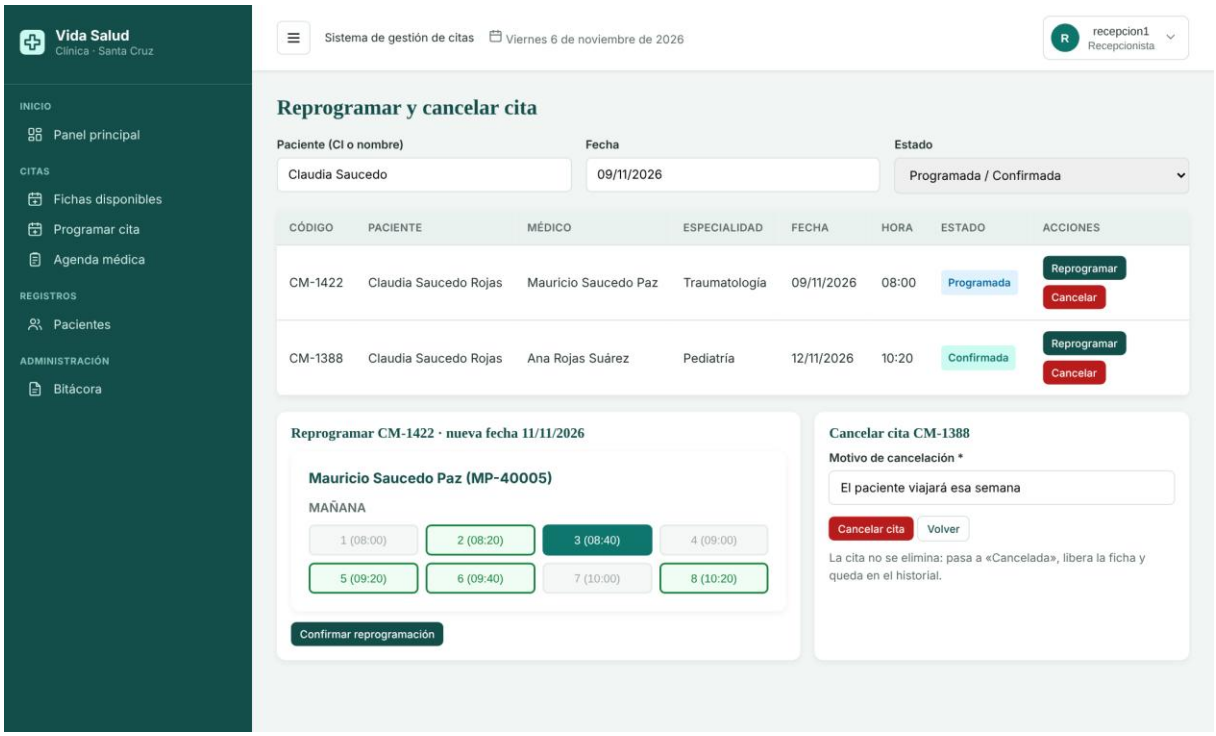


Figura 34. *Prototipo — CU10 Reprogramar y CU11 Cancelar cita*

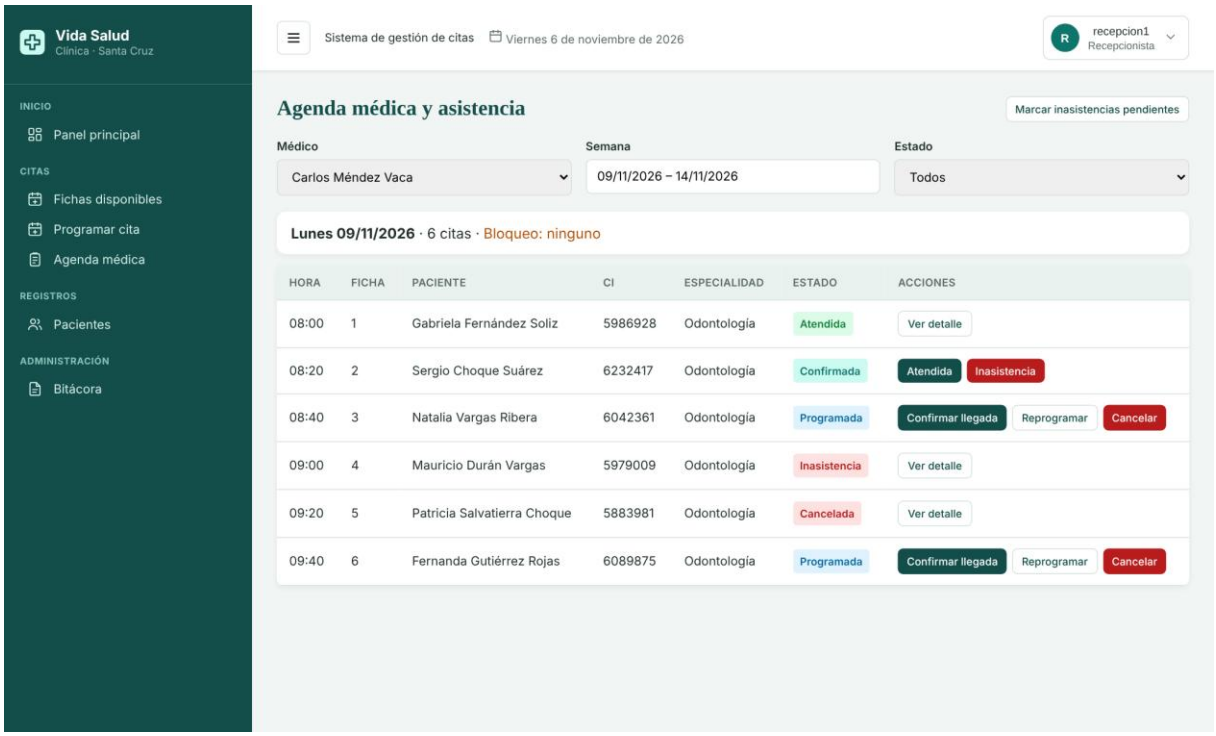


Figura 35. *Prototipo — CU13 Agenda médica y CU12 Registrar asistencia*

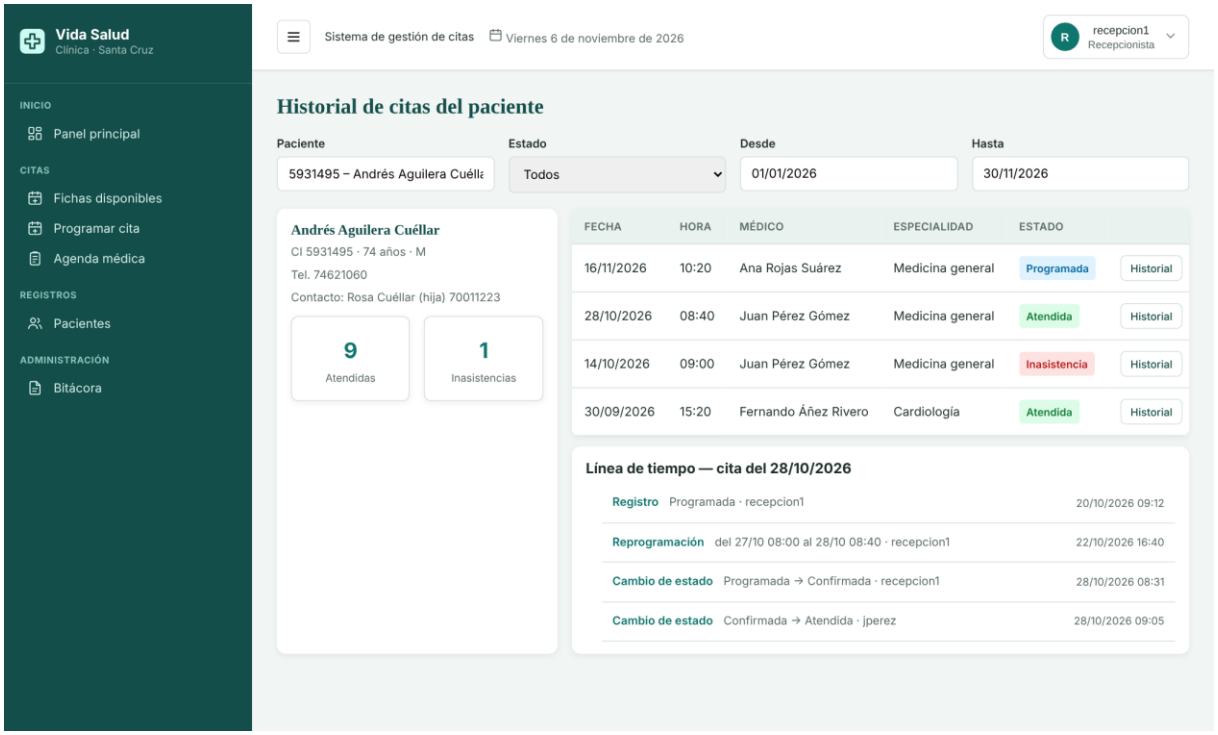


Figura 36. *Prototipo — CU14 Historial de citas del paciente*

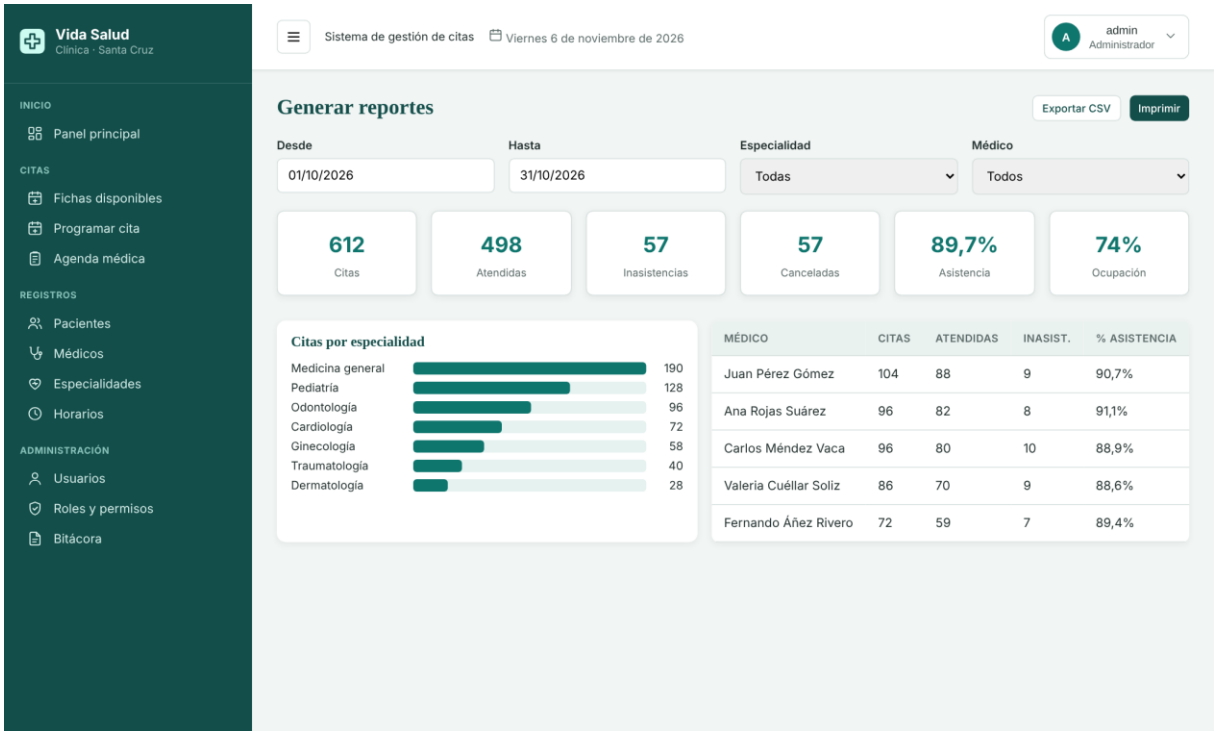


Figura 37. *Prototipo — CU15 Generar reportes*

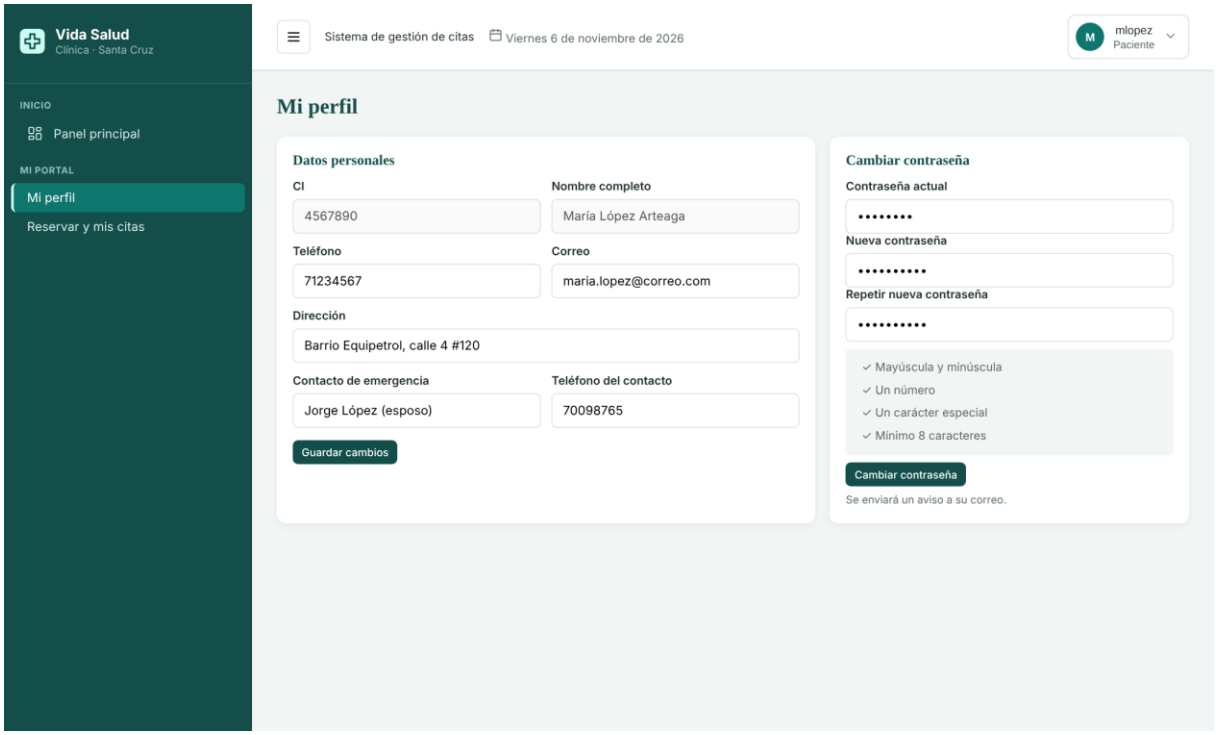


Figura 38. *Prototipo — CU16 Gestionar perfil del paciente*

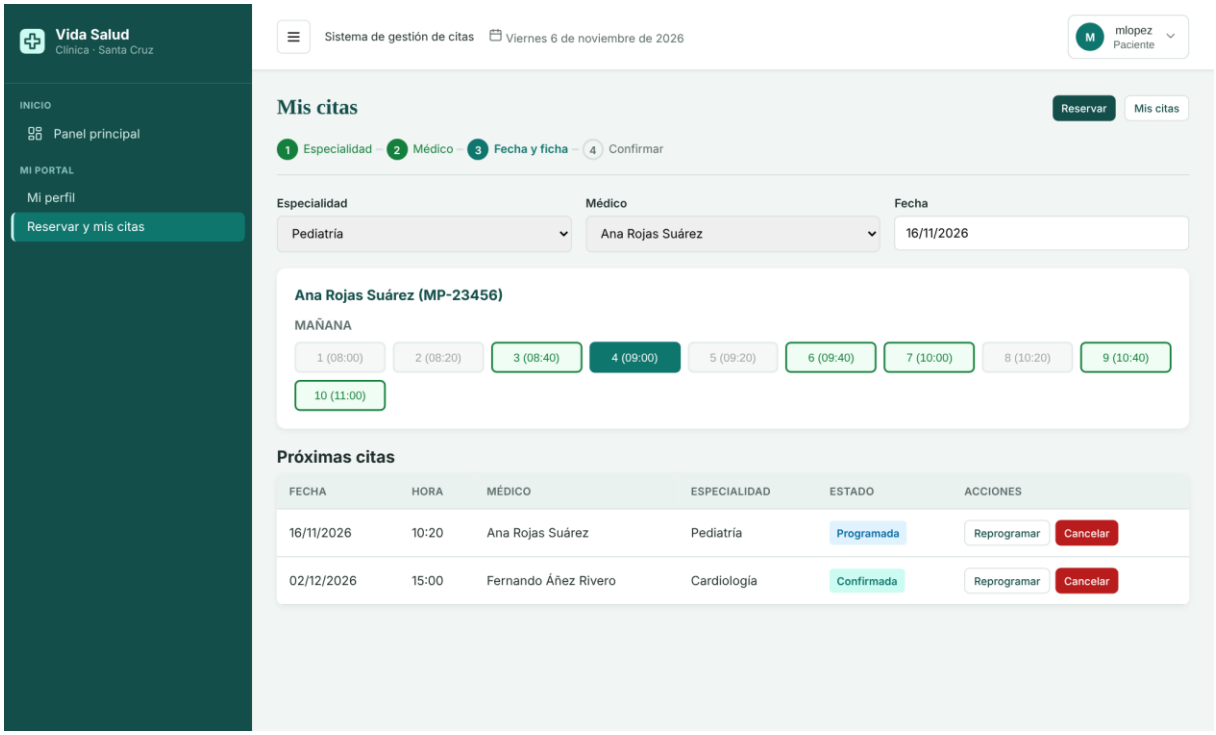


Figura 39. *Prototipo — CU17 Reservar y dar seguimiento a mis citas*

2.5 Estructurar el modelo de casos de uso

El modelo de casos de uso se estructura por ciclo, mostrando las relaciones de inclusión (<<include>>) y extensión (<<extend>>). CU09, CU10 y CU17 incluyen CU08 porque siempre necesitan calcular la disponibilidad; CU14 extiende CU04 (desde la ficha del paciente se puede ver su historial); CU12 extiende CU13 (desde la agenda se registra la asistencia); y CU11 extiende CU17 (desde «Mis citas» el paciente puede cancelar).

Ciclo #1: Seguridad, registros base, horarios y programación de citas

CU01, CU02 y CU03 (Seguridad); CU04, CU05 y CU06 (Personas); CU07 y CU08 (Agenda); CU09 (Citas). El administrador configura usuarios, roles, médicos, especialidades y horarios; la recepcionista registra pacientes, consulta la disponibilidad y programa citas.

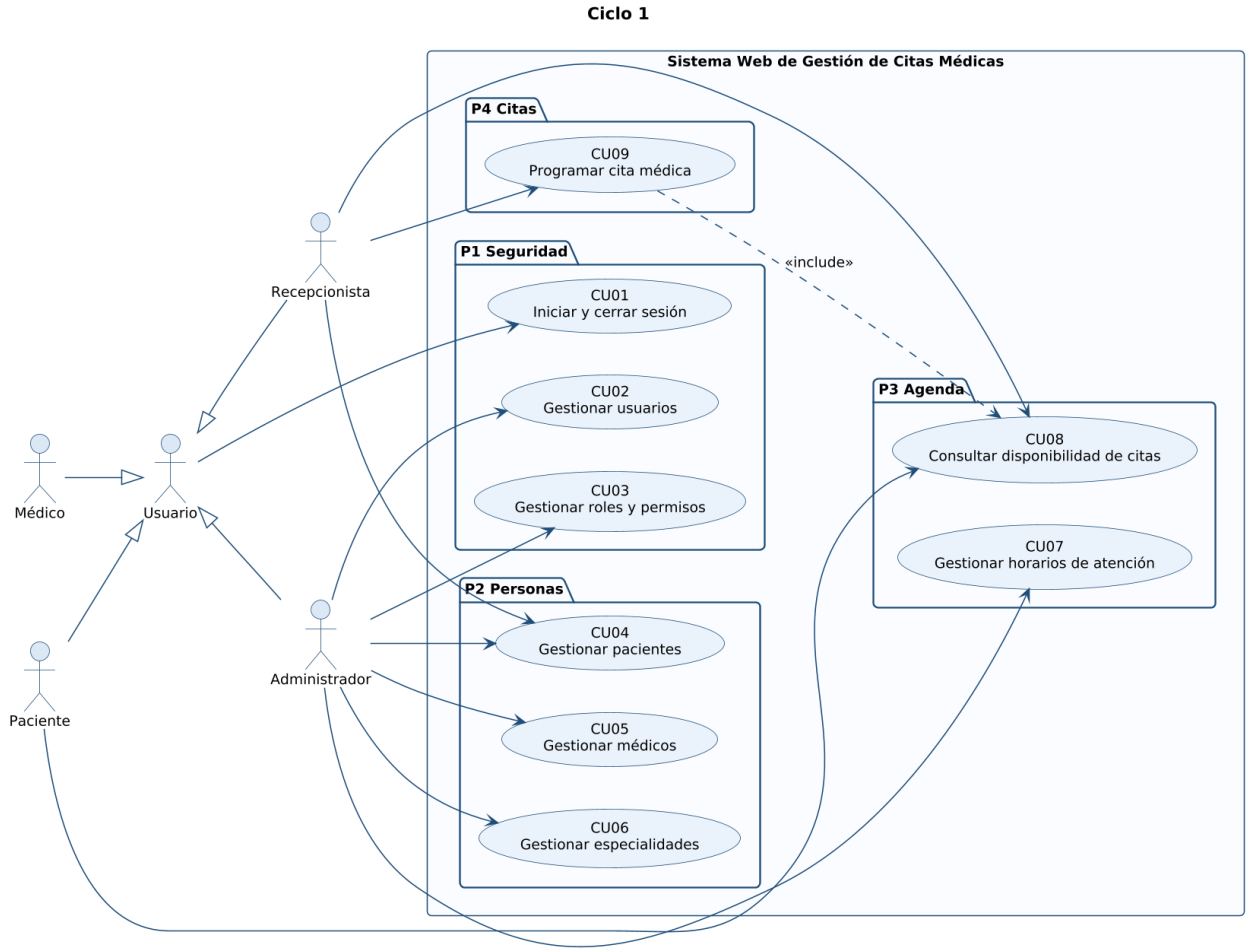


Figura 40. *Diagrama de casos de uso — Ciclo #1*

Ciclo #2: Gestión de citas, consultas, reportes y portal del paciente

El segundo ciclo completa el seguimiento de las citas y agrega las consultas, los reportes y el portal del paciente: CU10 a CU14 (paquete Citas), CU15 (Reportes), CU16 y CU17 (Portal del paciente). La recepcionista reprograma, cancela y registra la asistencia; el médico consulta su agenda y marca las citas atendidas; el administrador genera reportes; y el paciente gestiona su perfil y reserva sus citas en línea.

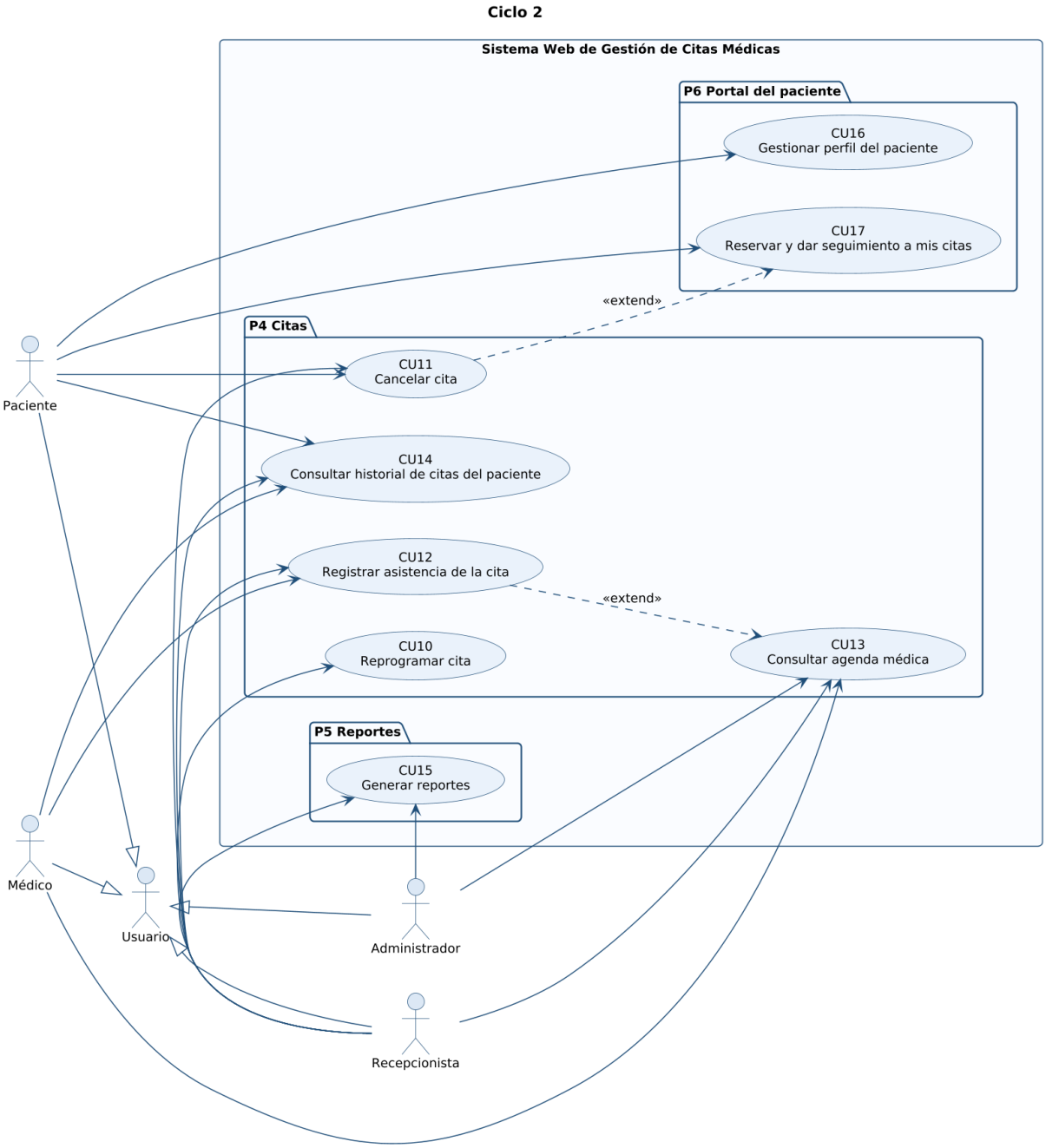


Figura 41. *Diagrama de casos de uso — Ciclo #2*

Escenario completo

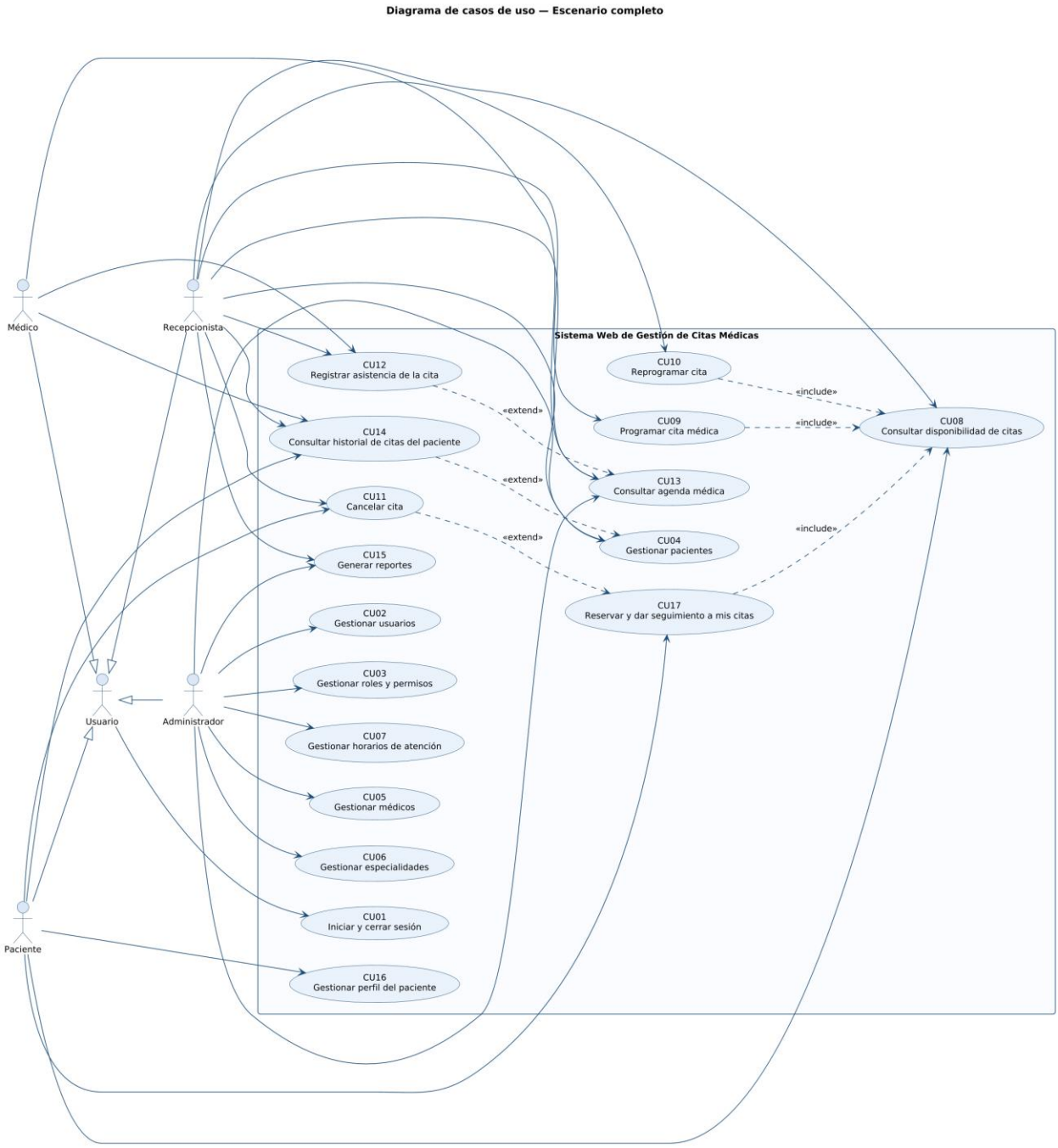


Figura 42. *Diagrama de casos de uso — escenario completo (17 casos de uso, 4 actores)*

CAPÍTULO 3: FLUJO DE TRABAJO: ANÁLISIS

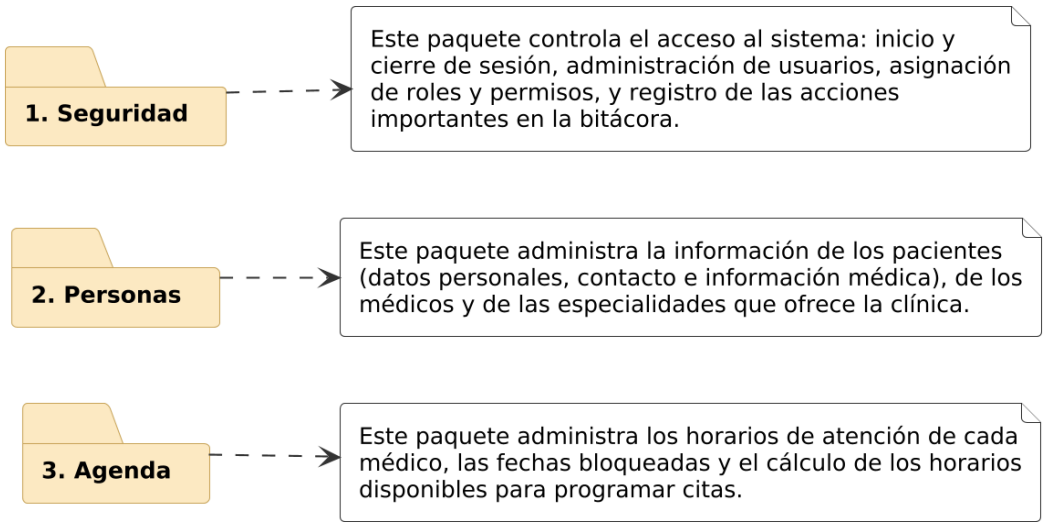
En el análisis se refinan los casos de uso capturados en un modelo de análisis formado por clases de interfaz (IU), control (CTR) y entidad (CE), organizadas en los seis paquetes del sistema. Esta misma división se mantiene en el diseño y en la implementación.

3.1 Análisis de arquitectura

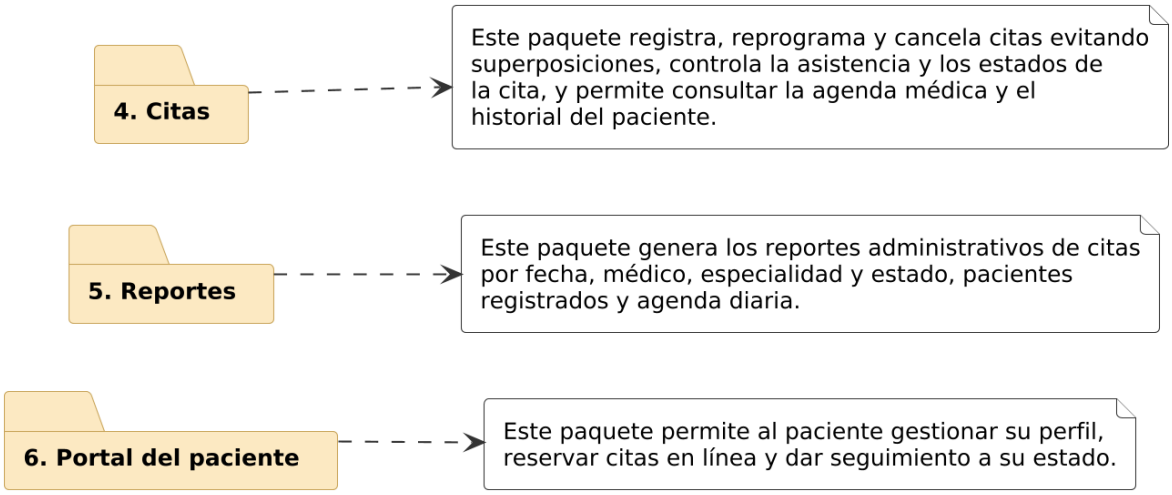
El análisis de la arquitectura identifica los paquetes de análisis del sistema, asigna a cada paquete los casos de uso que agrupa y muestra, para cada paquete, la vista de casos de uso con los actores que intervienen. Los paquetes se definieron agrupando los casos de uso que trabajan sobre las mismas clases entidad y responden a un mismo proceso del negocio.

3.1.1 Identificar paquetes

El sistema se organiza en seis paquetes de análisis. Cada paquete se acompaña de una nota que describe su responsabilidad.



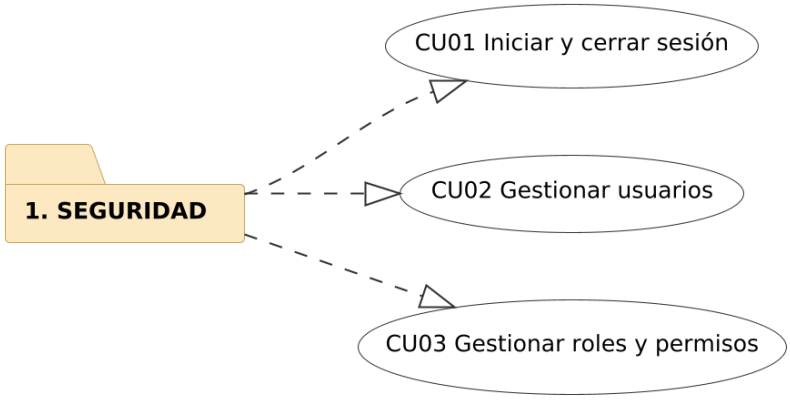
Identificación de paquetes (1 – 3)



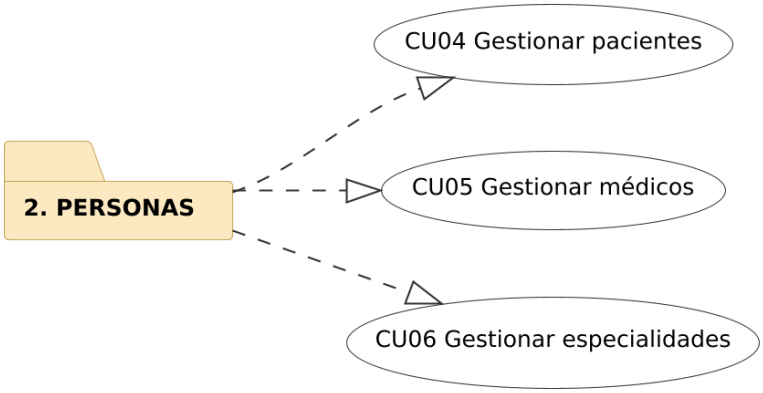
Identificación de paquetes (4 – 6)

3.1.2 Relacionar paquetes y casos de uso

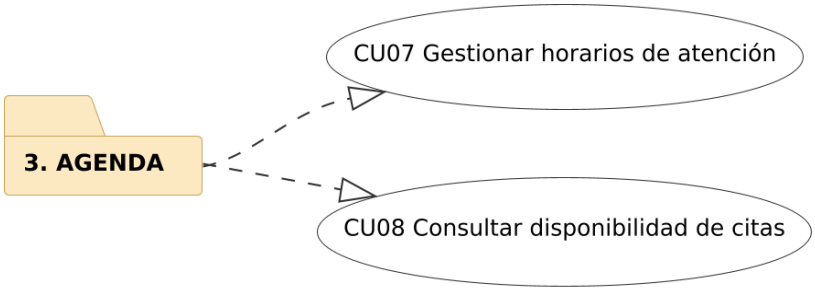
Cada paquete se relaciona con los casos de uso que contiene. Un caso de uso pertenece a un solo paquete.



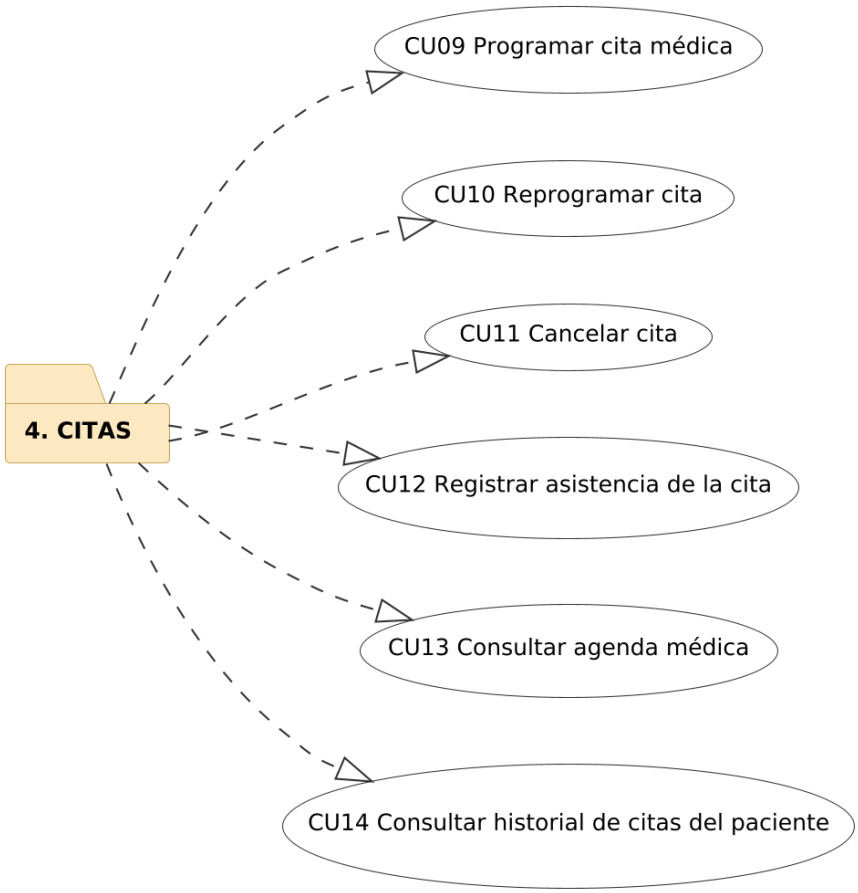
Paquete 1. Seguridad y sus casos de uso



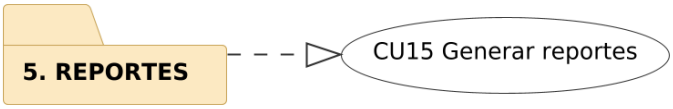
Paquete 2. Personas y sus casos de uso



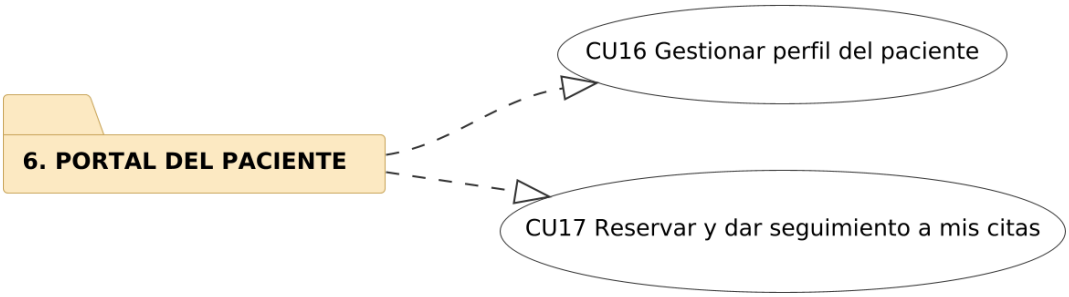
Paquete 3. Agenda y sus casos de uso



Paquete 4. Citas y sus casos de uso



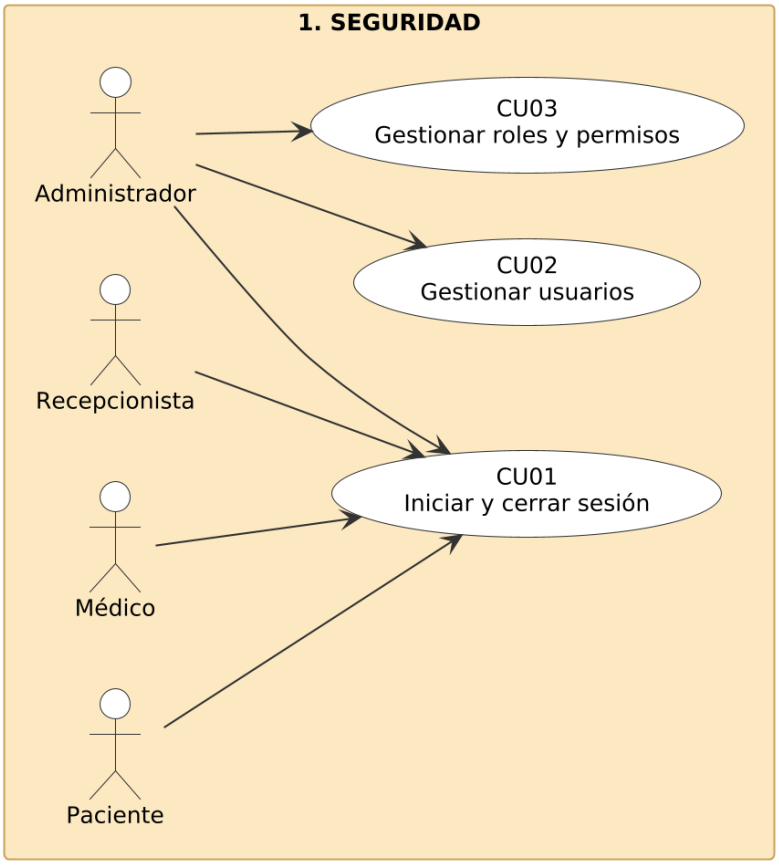
Paquete 5. Reportes y sus casos de uso



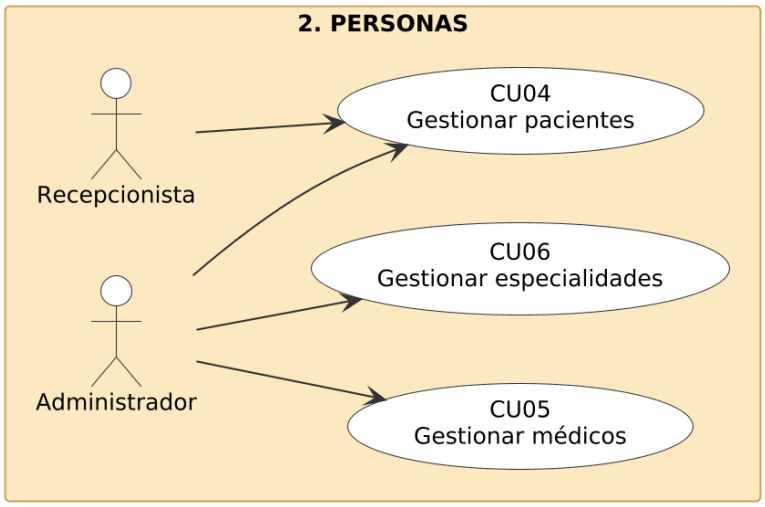
Paquete 6. Portal del paciente y sus casos de uso

3.1.3 Vista de casos de uso

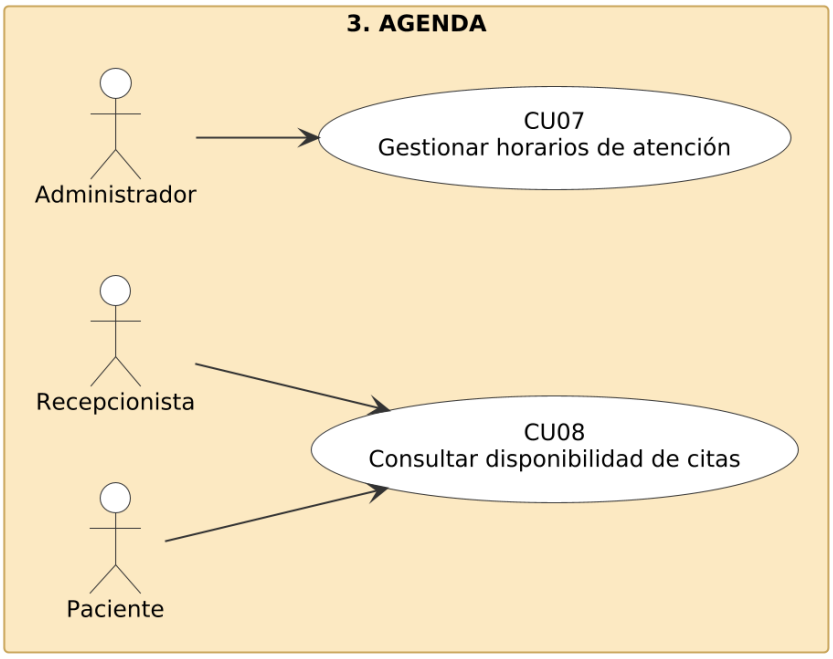
La vista de casos de uso muestra, dentro de cada paquete, los actores que inician cada caso de uso.



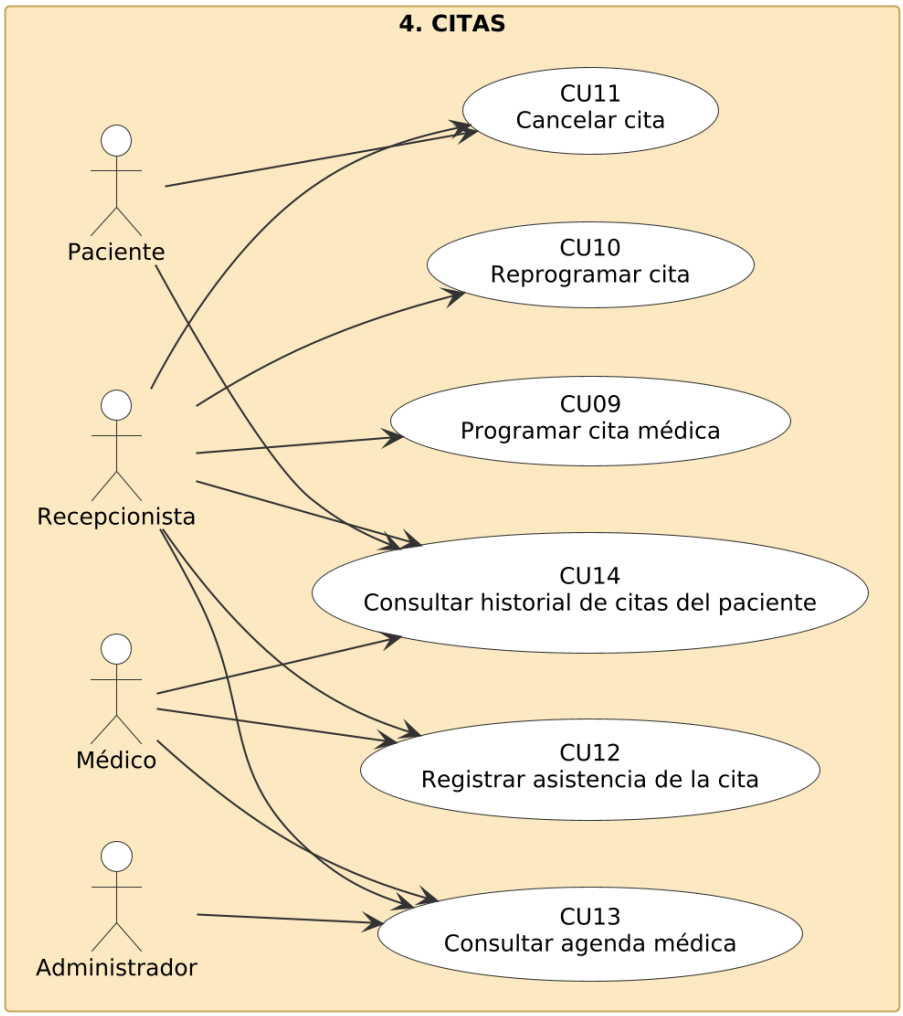
Vista de casos de uso — Paquete 1. Seguridad



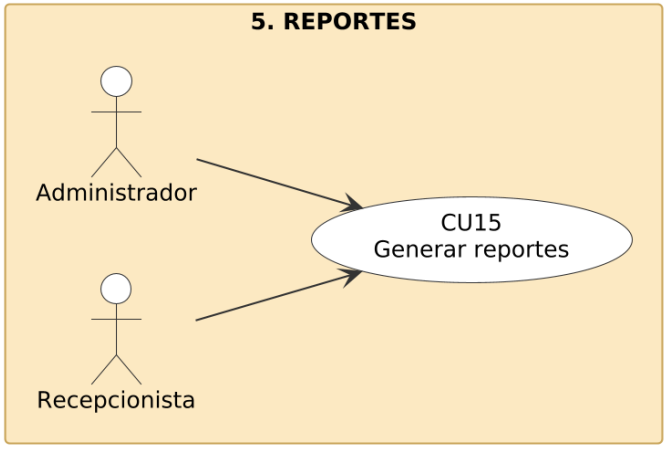
Vista de casos de uso — Paquete 2. Personas



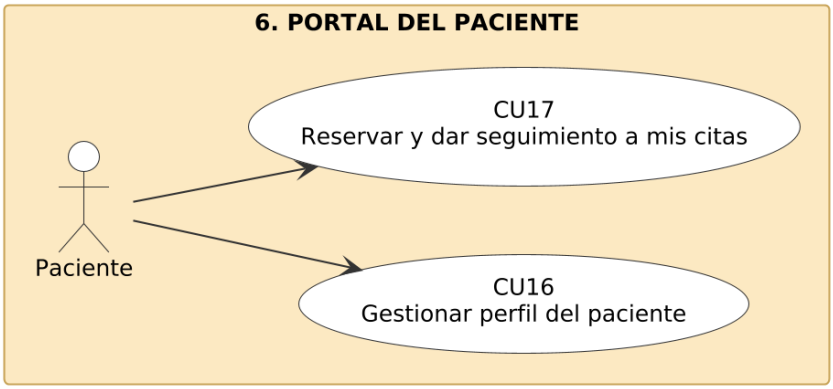
Vista de casos de uso — Paquete 3. Agenda



Vista de casos de uso — Paquete 4. Citas



Vista de casos de uso — Paquete 5. Reportes



Vista de casos de uso — Paquete 6. Portal del paciente

3.2 Analizar casos de uso

Para cada caso de uso se elaboró un diagrama de comunicación con las clases de análisis que participan y los mensajes numerados en el orden en que ocurren: los números enteros (1, 2, 3…) son acciones del actor y los decimales (1.1, 1.2…) los mensajes internos que provoca cada acción. Se respetan las reglas: el actor solo se comunica con la interfaz; la interfaz con el control; el control con las entidades; y no hay diálogo directo entre entidades. Los nombres coinciden con el código del sistema: cada interfaz IU_X se implementa en la pantalla CUnnX.js del frontend, cada control CTR_X en la clase PHP CunnX (sus mensajes son los métodos públicos de esa clase) y cada entidad en su tabla de PostgreSQL y en las funciones de repository.php; los atributos de las entidades son las columnas de la tabla. Los mismos objetos, mensajes y números se repiten en los diagramas de clases de análisis y de secuencia.

Ciclo #1

CU01: Iniciar y cerrar sesión

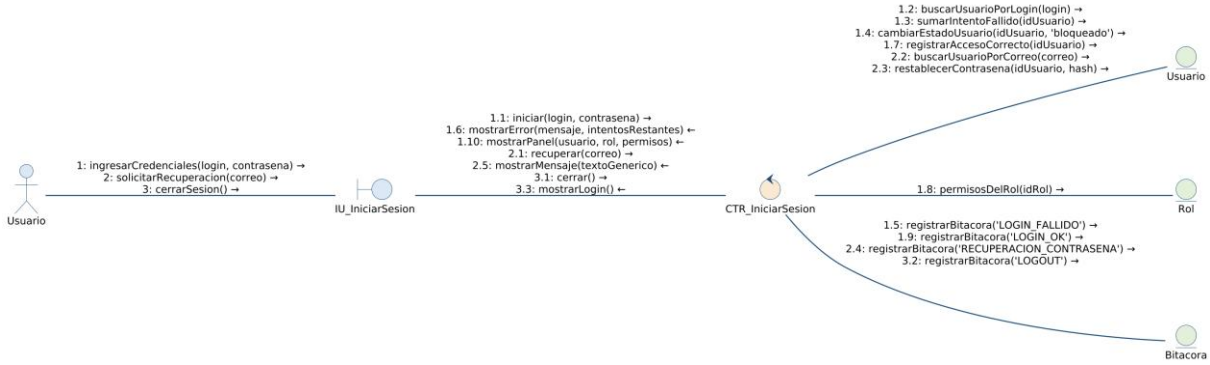


Figura 44. *Diagrama de comunicación — CU01 Iniciar y cerrar sesión*

CU02: Gestionar usuarios

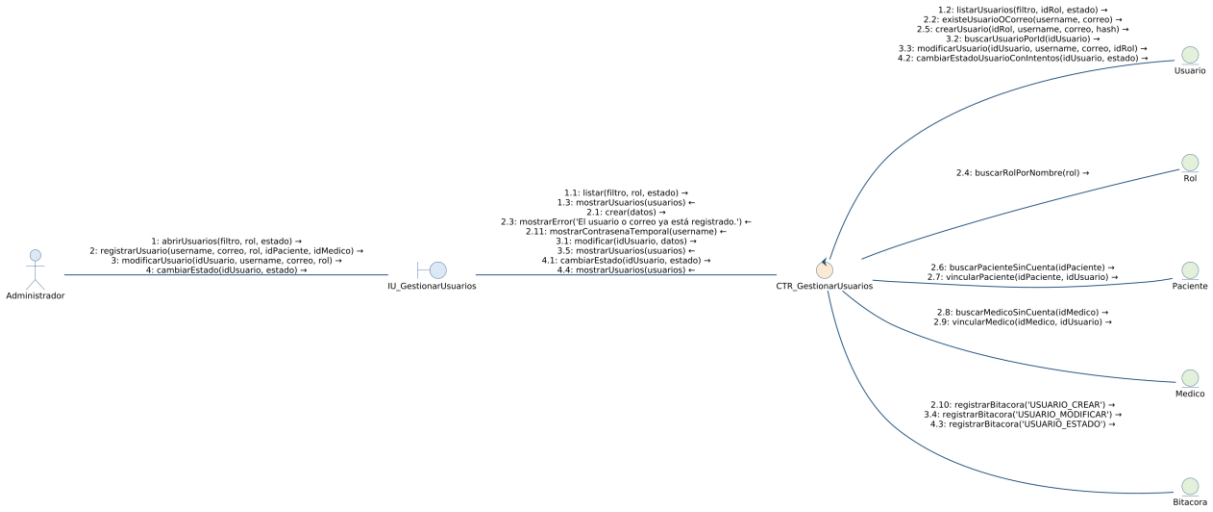


Figura 45. *Diagrama de comunicación — CU02 Gestionar usuarios*

CU03: Gestionar roles y permisos

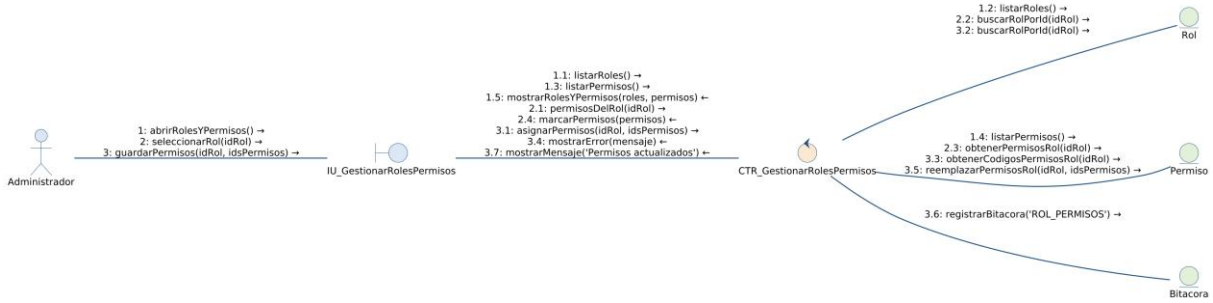


Figura 46. *Diagrama de comunicación — CU03 Gestionar roles y permisos*

CU04: Gestionar pacientes



Figura 47. *Diagrama de comunicación — CU04 Gestionar pacientes*

CU05: Gestionar médicos

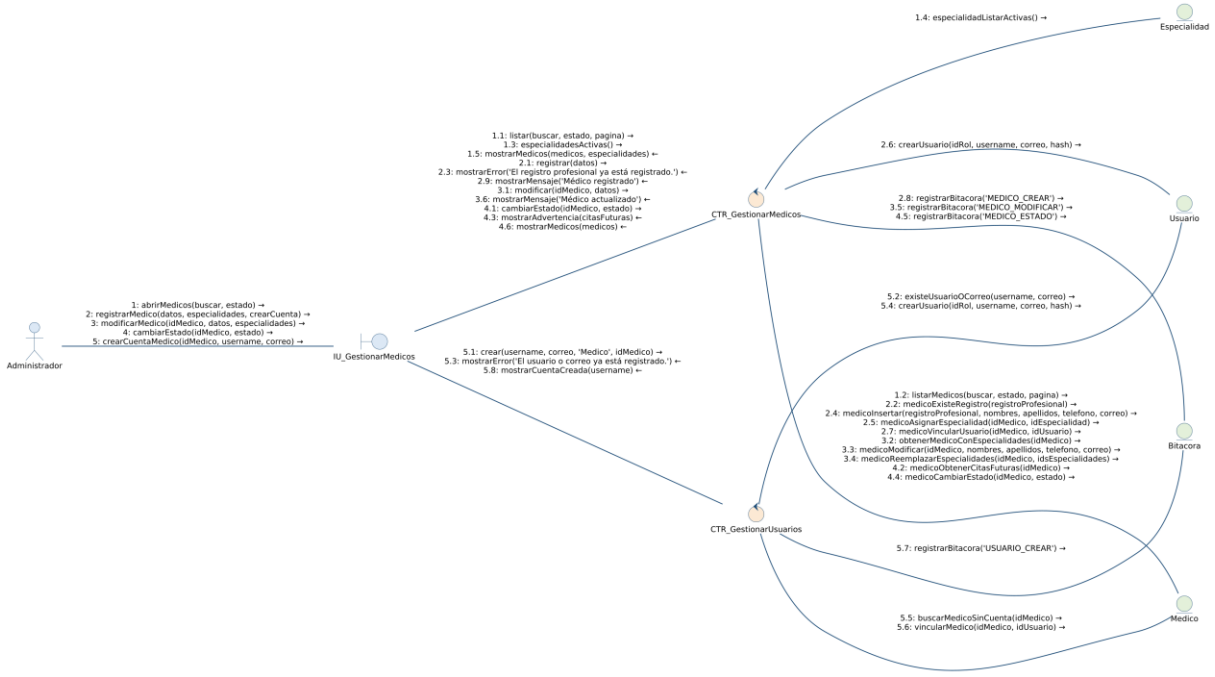


Figura 48. *Diagrama de comunicación — CU05 Gestionar médicos*

CU06: Gestionar especialidades



Figura 49. *Diagrama de comunicación — CU06 Gestionar especialidades*

CU07: Gestionar horarios de atención



Figura 50. *Diagrama de comunicación — CU07 Gestionar horarios de atención*

CU08: Consultar disponibilidad de citas



Figura 51. *Diagrama de comunicación — CU08 Consultar disponibilidad de citas*

CU09: Programar cita médica

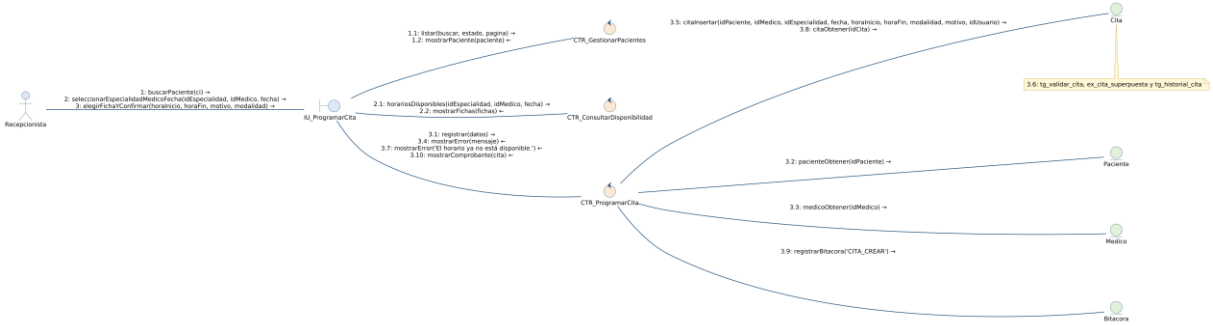


Figura 52. *Diagrama de comunicación — CU09 Programar cita médica*

Ciclo #2

CU10: Reprogramar cita

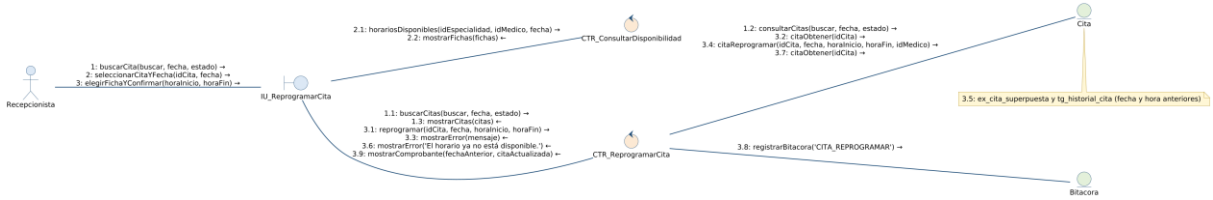


Figura 53. *Diagrama de comunicación — CU10 Reprogramar cita*

CU11: Cancelar cita

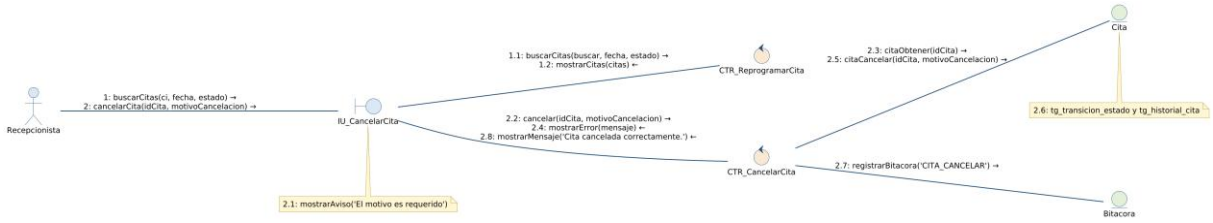


Figura 54. *Diagrama de comunicación — CU11 Cancelar cita*

CU12: Registrar asistencia de la cita

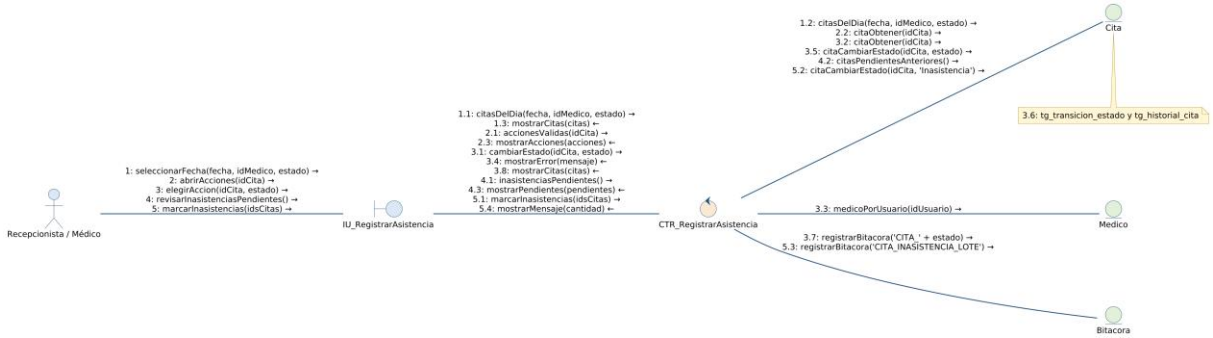


Figura 55. *Diagrama de comunicación — CU12 Registrar asistencia de la cita*

CU13: Consultar agenda médica



Figura 56. *Diagrama de comunicación — CU13 Consultar agenda médica*

CU14: Consultar historial de citas del paciente

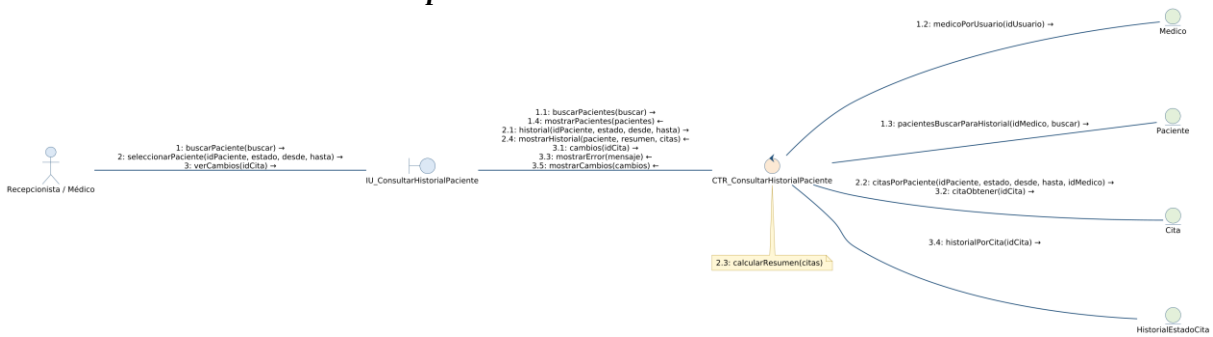


Figura 57. *Diagrama de comunicación — CU14 Consultar historial de citas del paciente*

CU15: Generar reportes

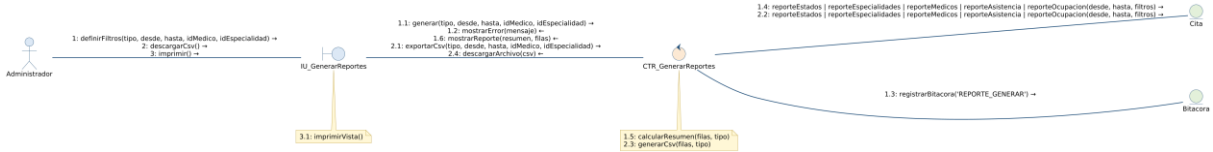


Figura 58. *Diagrama de comunicación — CU15 Generar reportes*

CU16: Gestionar perfil del paciente

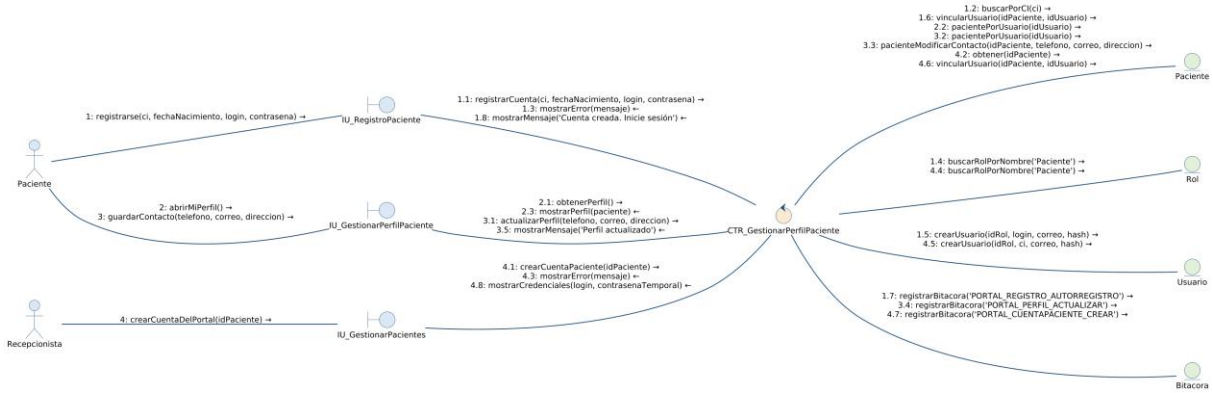


Figura 59. *Diagrama de comunicación — CU16 Gestionar perfil del paciente*

CU17: Reservar y dar seguimiento a mis citas

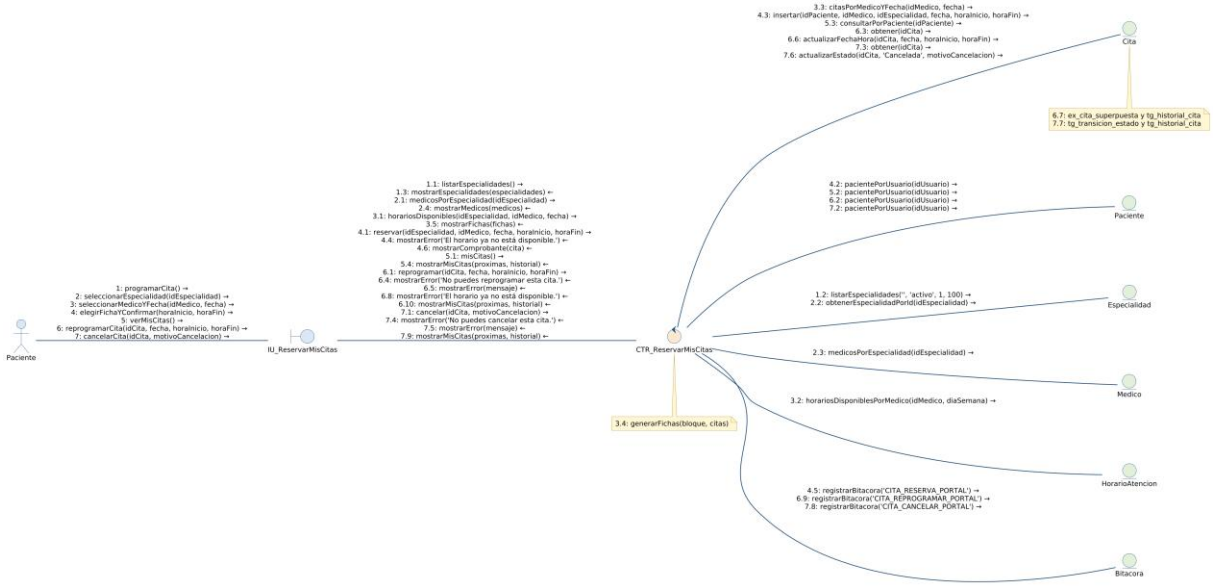


Figura 60. *Diagrama de comunicación — CU17 Reservar y dar seguimiento a mis citas*

3.3 Análisis de clases

Para cada caso de uso se identifican la clase de interfaz, la clase de control y las clases entidad que lee o modifica. Las responsabilidades (operaciones) de cada clase se obtienen de los mensajes de su diagrama de comunicación.

Tabla 27. *Clases de análisis del paquete Seguridad*

CU	Interfaz (IU)	Control (CTR)	Entidades (CE)
CU01	IU_IniciarSesion	CTR_IniciarSesion	Usuario, Rol, Bitacora
CU02	IU_GestionarUsuarios	CTR_GestionarUsuarios	Usuario, Rol, Paciente, Medico, Bitacora
CU03	IU_GestionarRolesPermisos	CTR_GestionarRolesPermisos	Rol, Permiso, Bitacora

Tabla 28. *Clases de análisis del paquete Personas*

CU	Interfaz (IU)	Control (CTR)	Entidades (CE)
CU04	IU_GestionarPacientes	CTR_GestionarPacientes	Paciente, ContactoEmergencia, Bitacora
CU05	IU_GestionarMedicos	CTR_GestionarMedicos (usa CTR_GestionarUsuarios)	Medico, Especialidad, Usuario, Bitacora
CU06	IU_GestionarEspecialidades	CTR_GestionarEspecialidades	Especialidad, Medico, Bitacora

Tabla 29. *Clases de análisis del paquete Agenda*

CU	Interfaz (IU)	Control (CTR)	Entidades (CE)
CU07	IU_GestionarHorarios	CTR_GestionarHorarios	Medico, HorarioAtencion, Bitacora
CU08	IU_ConsultarDisponibilidad	CTR_ConsultarDisponibilidad	Especialidad, Medico, HorarioAtencion, Cita

Tabla 30. *Clases de análisis del paquete Citas*

CU	Interfaz (IU)	Control (CTR)	Entidades (CE)
CU09	IU_ProgramarCita	CTR_ProgramarCita (usa CTR_GestionarPacientes, CTR_ConsultarDisponibilidad)	Paciente, Medico, Cita, Bitacora
CU10	IU_ReprogramarCita	CTR_ReprogramarCita (usa CTR_ConsultarDisponibilidad)	Cita, Bitacora
CU11	IU_CancelarCita	CTR_CancelarCita (usa CTR_ReprogramarCita)	Cita, Bitacora
CU12	IU_RegistrarAsistencia	CTR_RegistrarAsistencia	Cita, Medico, Bitacora
CU13	IU_ConsultarAgendaMedica	CTR_ConsultarAgendaMedica	Medico, Cita, BloqueoAgenda
CU14	IU_ConsultarHistorialPaciente	CTR_ConsultarHistorialPaciente	Medico, Paciente, Cita, HistorialEstadoCita

Tabla 31. *Clases de análisis del paquete Reportes*

CU	Interfaz (IU)	Control (CTR)	Entidades (CE)
CU15	IU_GenerarReportes	CTR_GenerarReportes	Cita, Bitacora

Tabla 32. *Clases de análisis del paquete Portal del paciente*

CU	Interfaz (IU)	Control (CTR)	Entidades (CE)
CU16	IU_RegistroPaciente, IU_GestionarPerfilPaciente, IU_GestionarPacientes	CTR_GestionarPerfilPaciente	Paciente, Rol, Usuario, Bitacora
CU17	IU_ReservarMisCitas	CTR_ReservarMisCitas	Paciente, Especialidad, Medico, HorarioAtencion, Cita, Bitacora

Ciclo #1

CU01: Iniciar y cerrar sesión

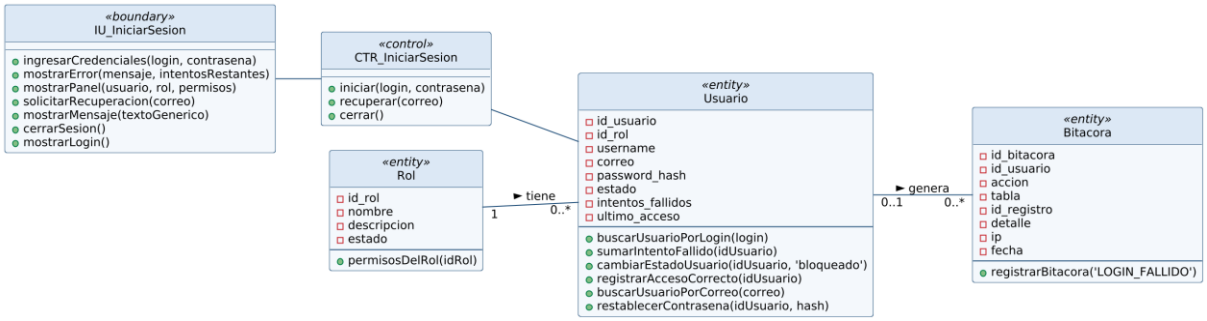


Figura 61. *Análisis de clases — CU01 Iniciar y cerrar sesión*

CU02: Gestionar usuarios

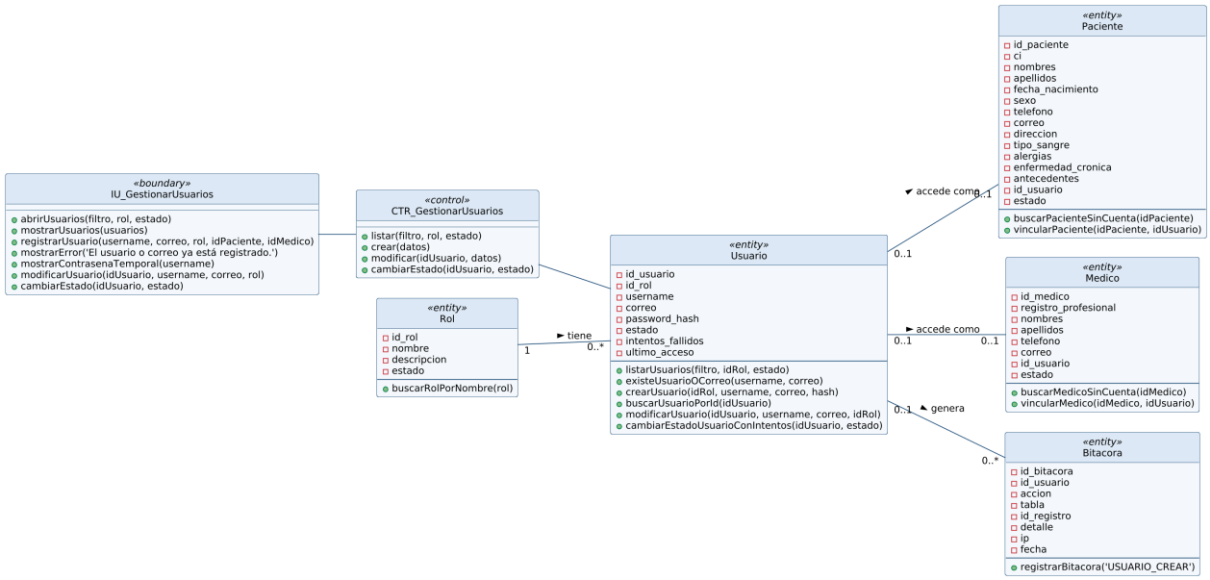


Figura 62. *Análisis de clases — CU02 Gestionar usuarios*

CU03: Gestionar roles y permisos

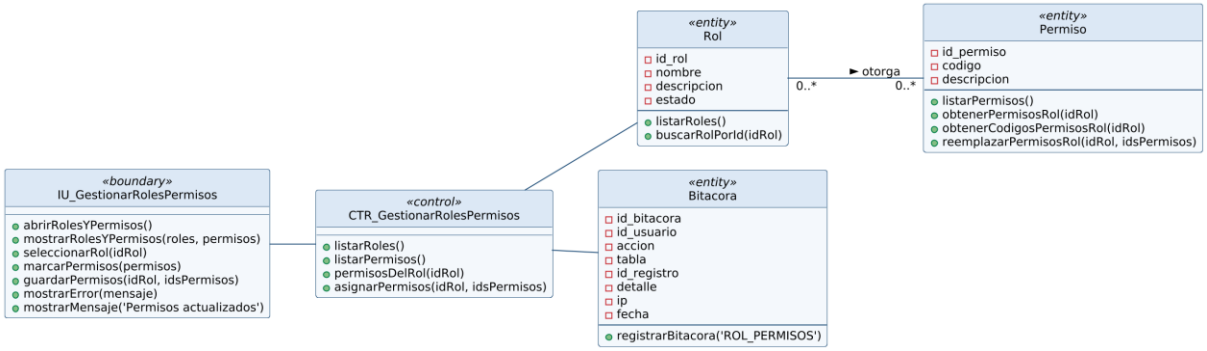


Figura 63. *Análisis de clases — CU03 Gestionar roles y permisos*

CU04: Gestionar pacientes



Figura 64. *Análisis de clases — CU04 Gestionar pacientes*

CU05: Gestionar médicos

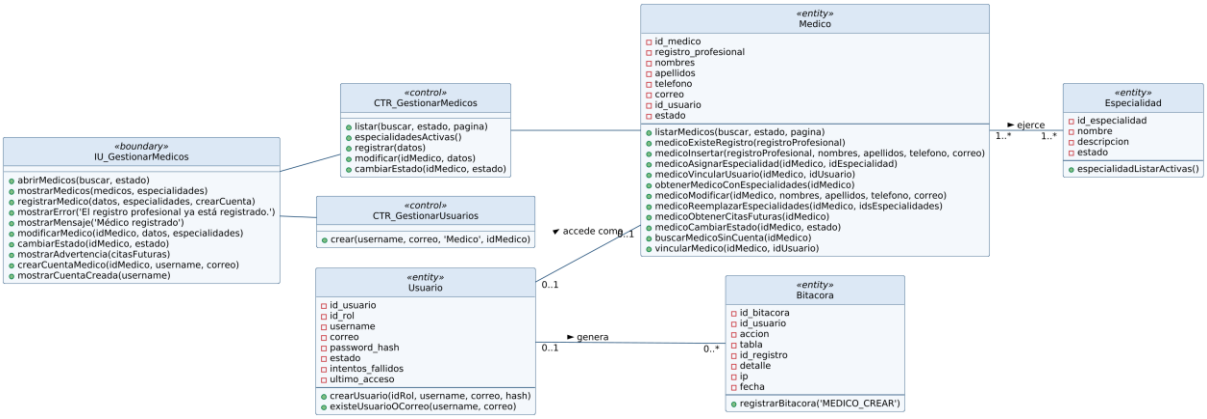


Figura 65. *Análisis de clases — CU05 Gestionar médicos*

CU06: Gestionar especialidades

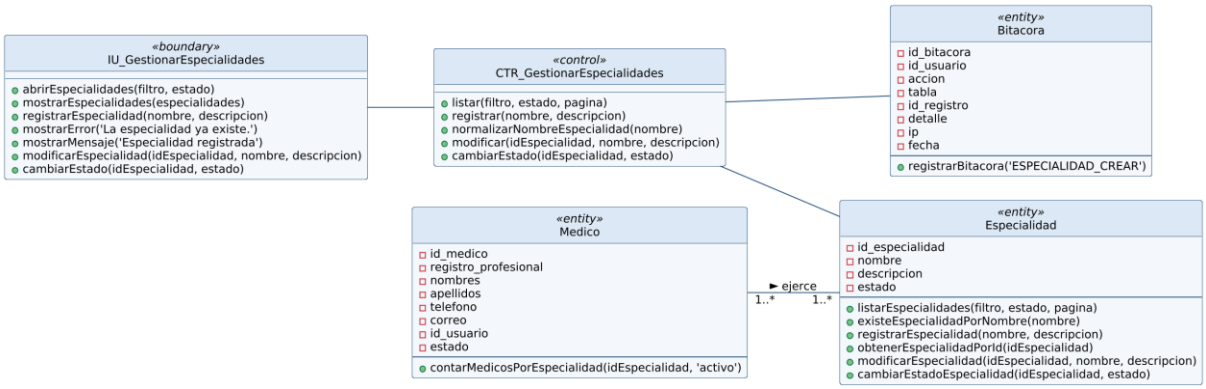


Figura 66. *Análisis de clases — CU06 Gestionar especialidades*

CU07: Gestionar horarios de atención

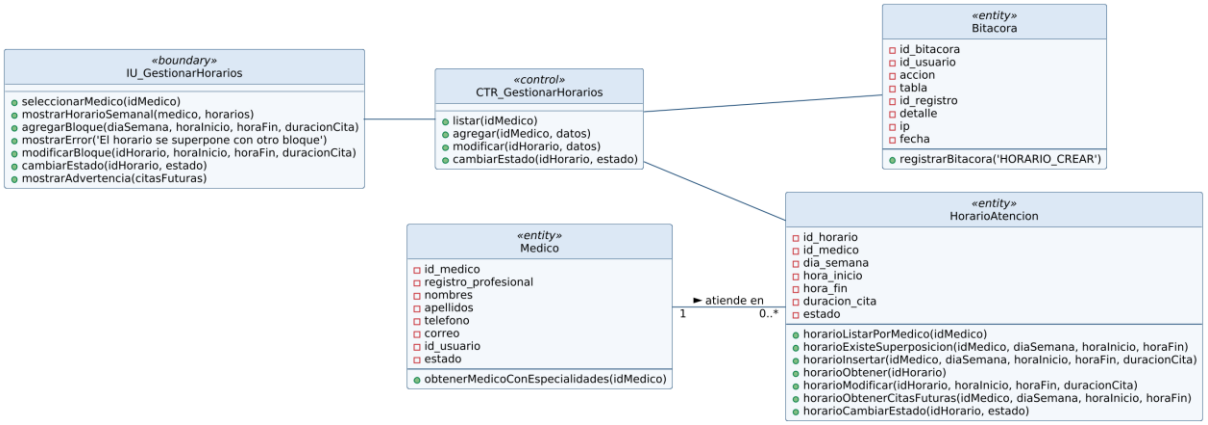


Figura 67. *Análisis de clases — CU07 Gestionar horarios de atención*

CU08: Consultar disponibilidad de citas



Figura 68. *Análisis de clases — CU08 Consultar disponibilidad de citas*

CU09: Programar cita médica

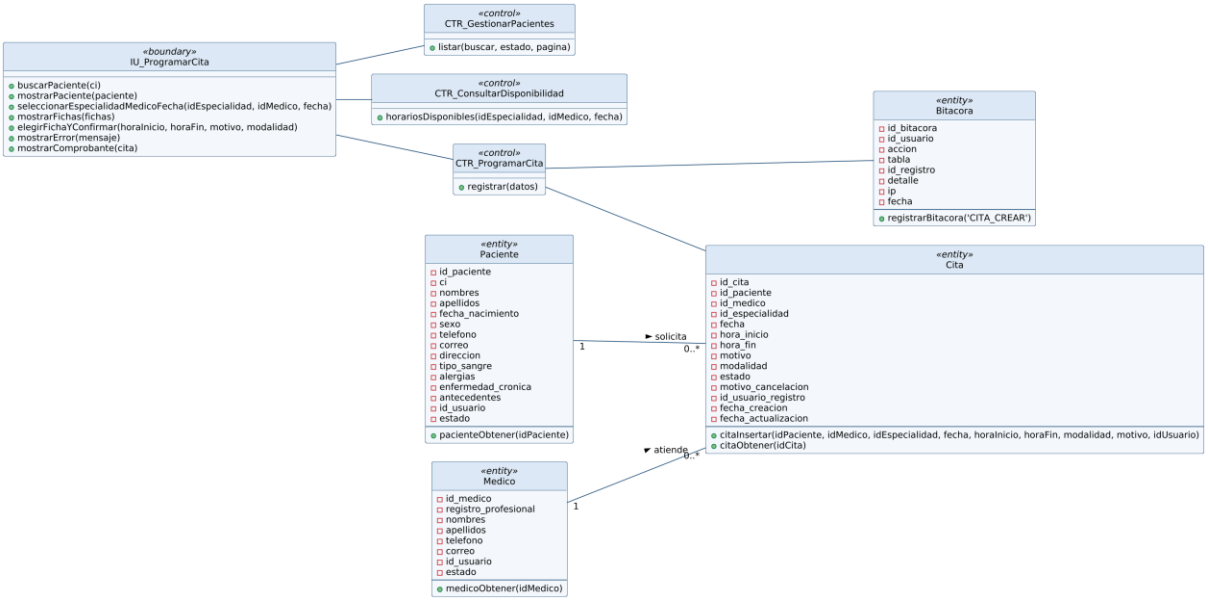


Figura 69. *Análisis de clases — CU09 Programar cita médica*

Ciclo #2

CU10: Reprogramar cita

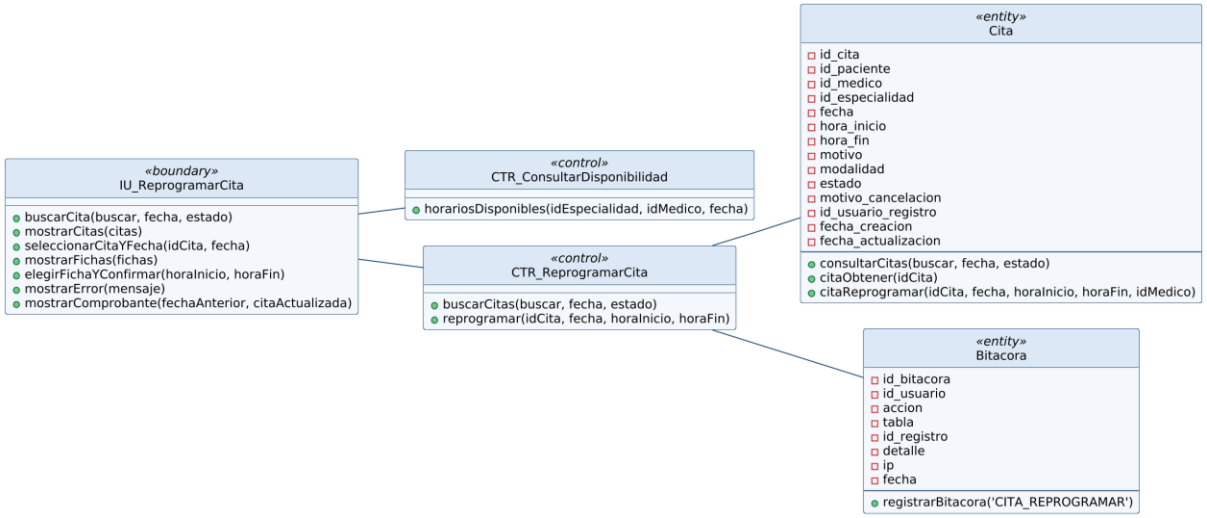


Figura 70. *Análisis de clases — CU10 Reprogramar cita*

CU11: Cancelar cita

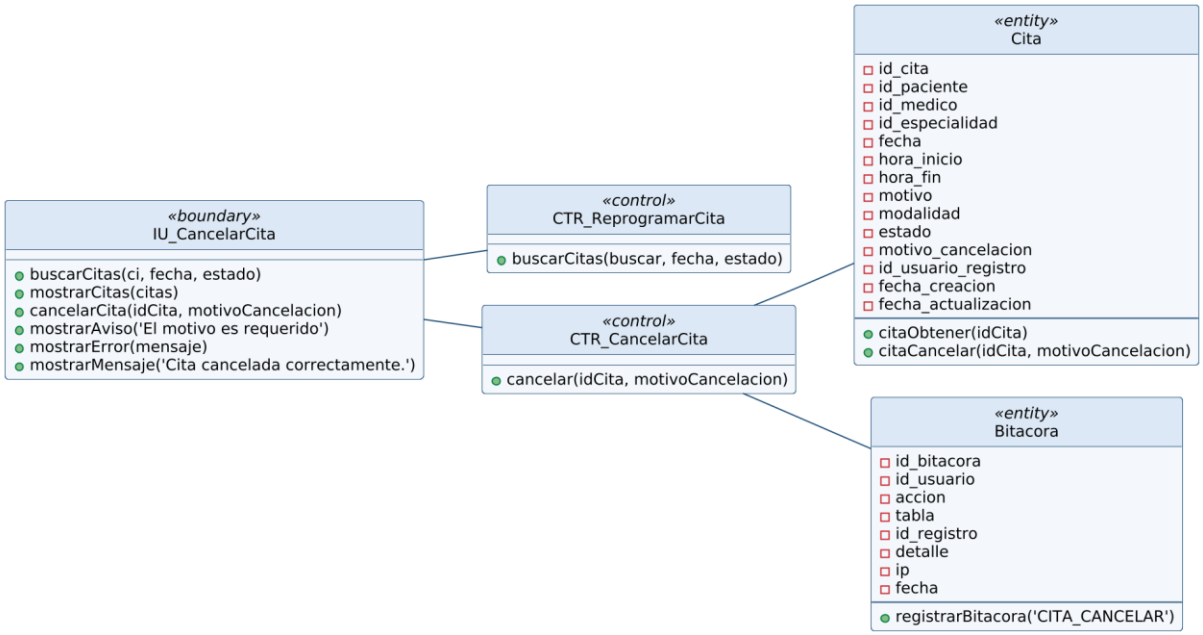


Figura 71. *Análisis de clases — CU11 Cancelar cita*

CU12: Registrar asistencia de la cita

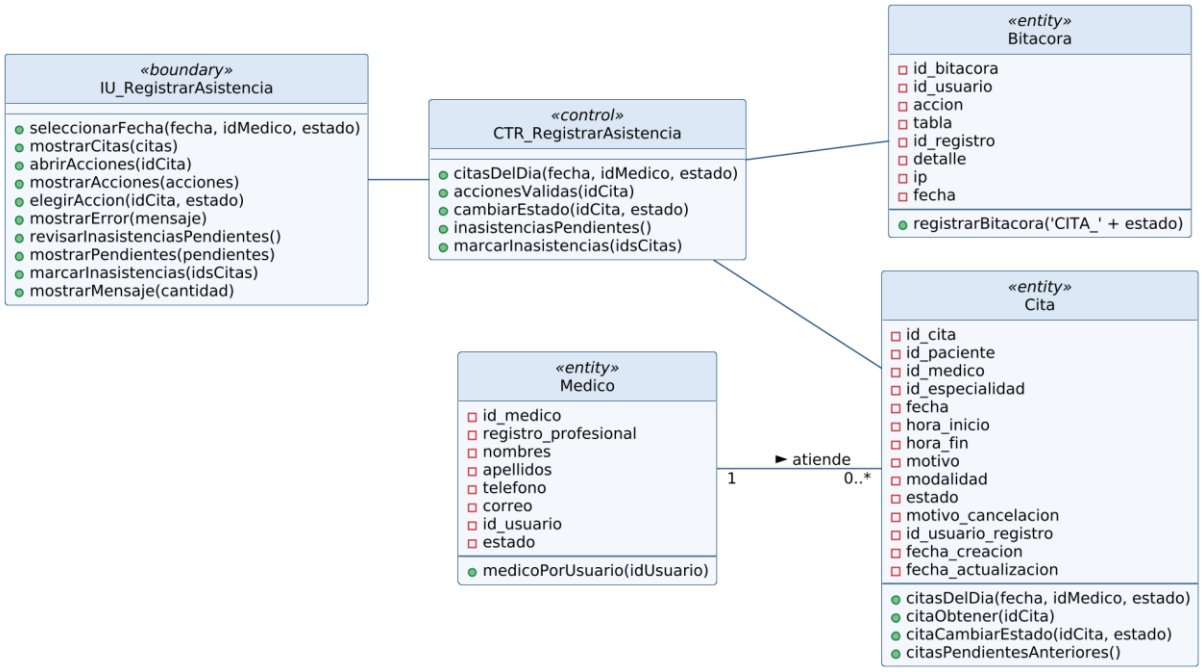


Figura 72. *Análisis de clases — CU12 Registrar asistencia de la cita*

CU13: Consultar agenda médica

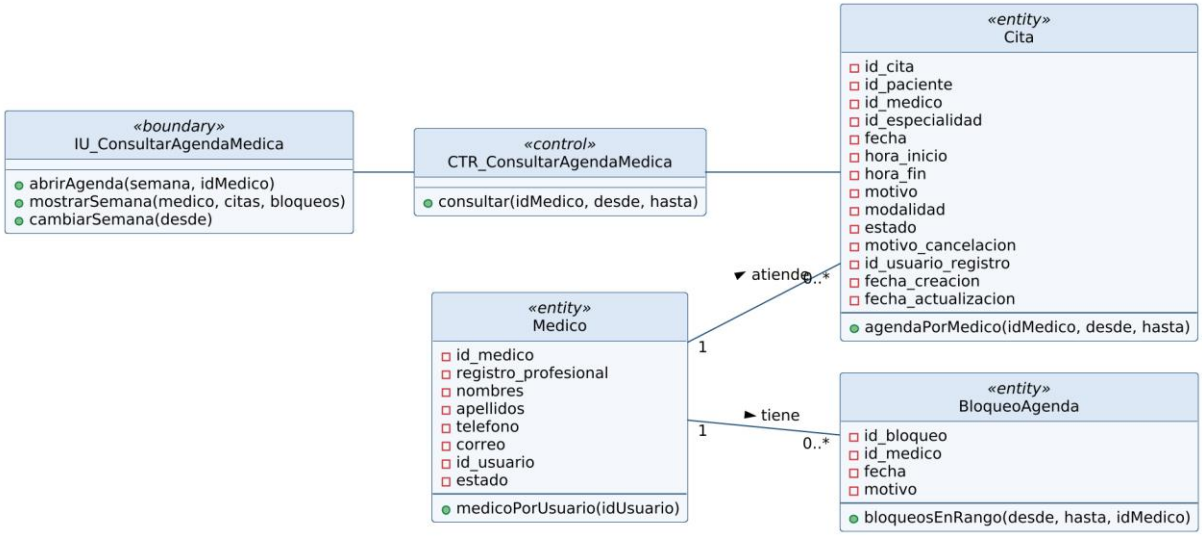


Figura 73. *Análisis de clases — CU13 Consultar agenda médica*

CU14: Consultar historial de citas del paciente

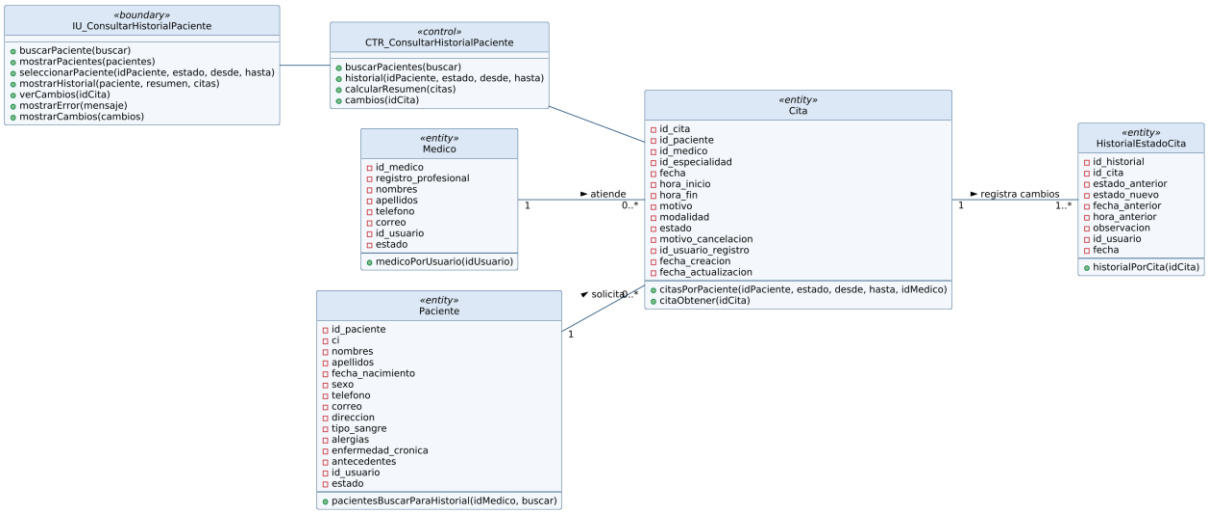


Figura 74. *Análisis de clases — CU14 Consultar historial de citas del paciente*

CU15: Generar reportes

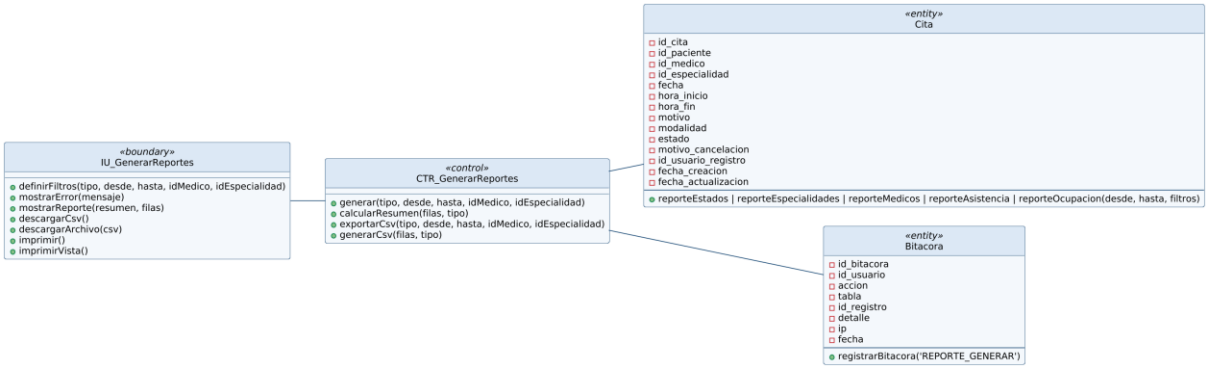


Figura 75. *Análisis de clases — CU15 Generar reportes*

CU16: Gestionar perfil del paciente

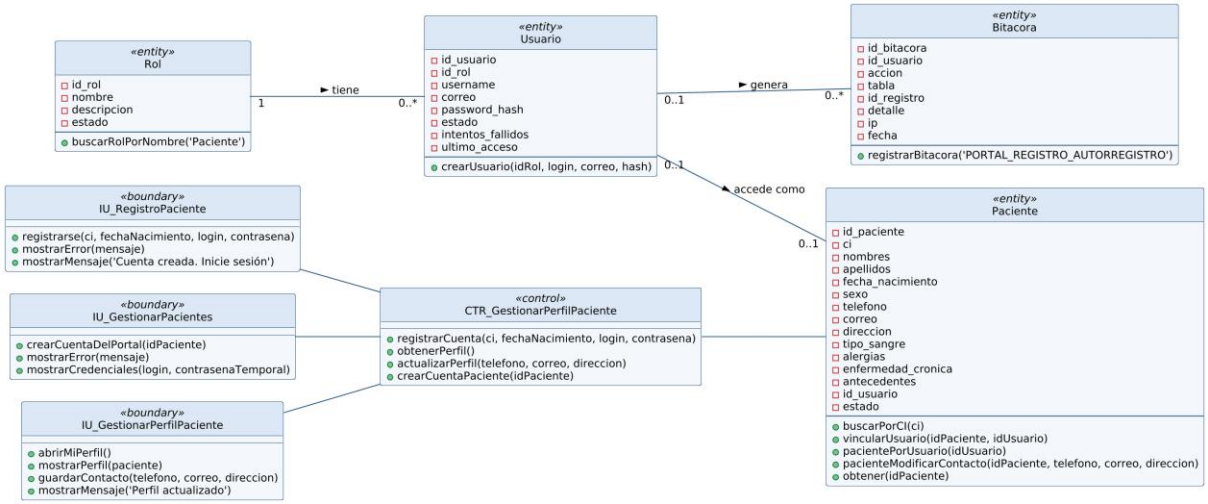


Figura 76. *Análisis de clases — CU16 Gestionar perfil del paciente*

CU17: Reservar y dar seguimiento a mis citas

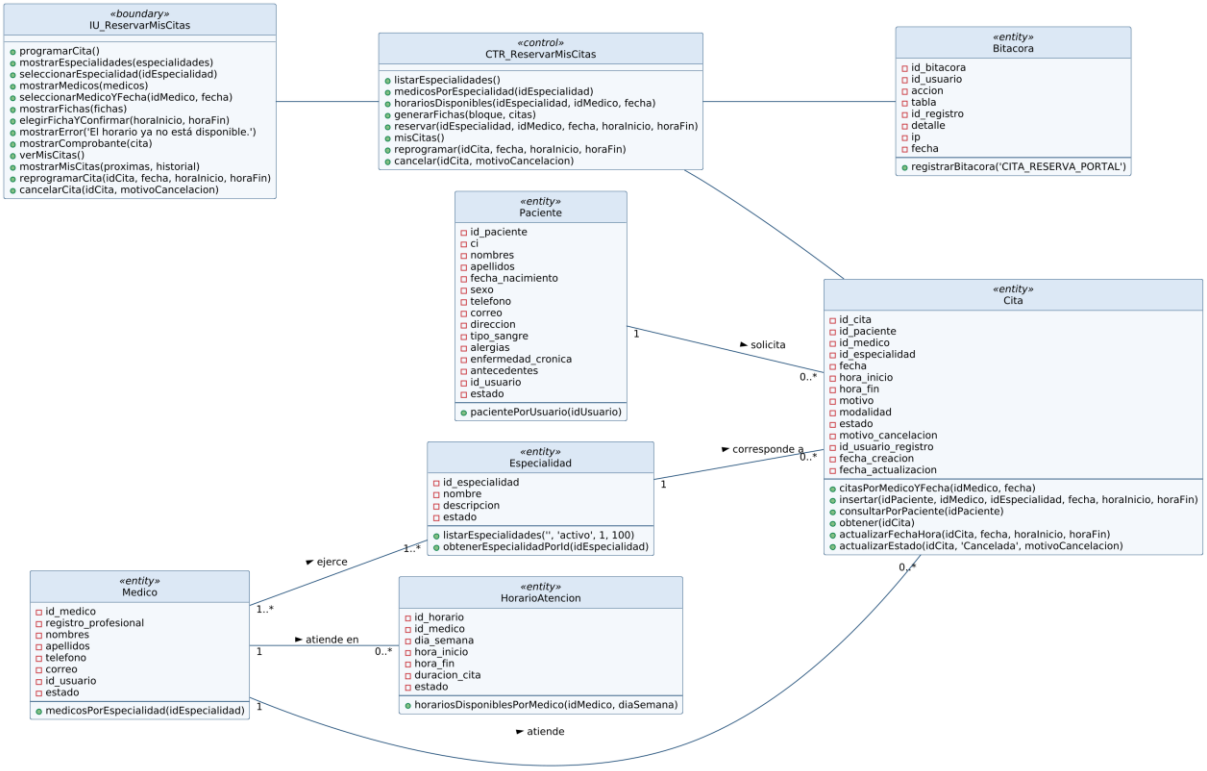


Figura 77. *Análisis de clases — CU17 Reservar y dar seguimiento a mis citas*

3.4 Analizar paquetes

Tabla 33. *Paquetes de análisis*

Paquete	Responsabilidad	Casos de uso	Tablas principales
P1 Seguridad	Acceso, usuarios, roles, permisos y bitácora	CU01, CU02, CU03	usuario, rol, permiso, rol_permiso, bitacora
P2 Personas	Pacientes, médicos y especialidades	CU04, CU05, CU06	paciente, contacto_emergencia, medico, especialidad, medico_especialidad
P3 Agenda	Horarios de atención y disponibilidad	CU07, CU08	horario_atencion, bloqueo_agenda
P4 Citas	Programación, reprogramación, cancelación, asistencia, agenda e historial	CU09, CU10, CU11, CU12, CU13, CU14	cita, historial_estado_cita
P5 Reportes	Reportes administrativos de citas y pacientes	CU15	v_agenda_medica, v_resumen_estados
P6 Portal del paciente	Perfil del paciente y reserva en línea	CU16, CU17	paciente, contacto_emergencia, cita

El diagrama muestra los paquetes de análisis y sus dependencias funcionales. Una dependencia <<use>> indica que un paquete utiliza clases entidad o reglas de otro paquete; nunca sus interfaces.

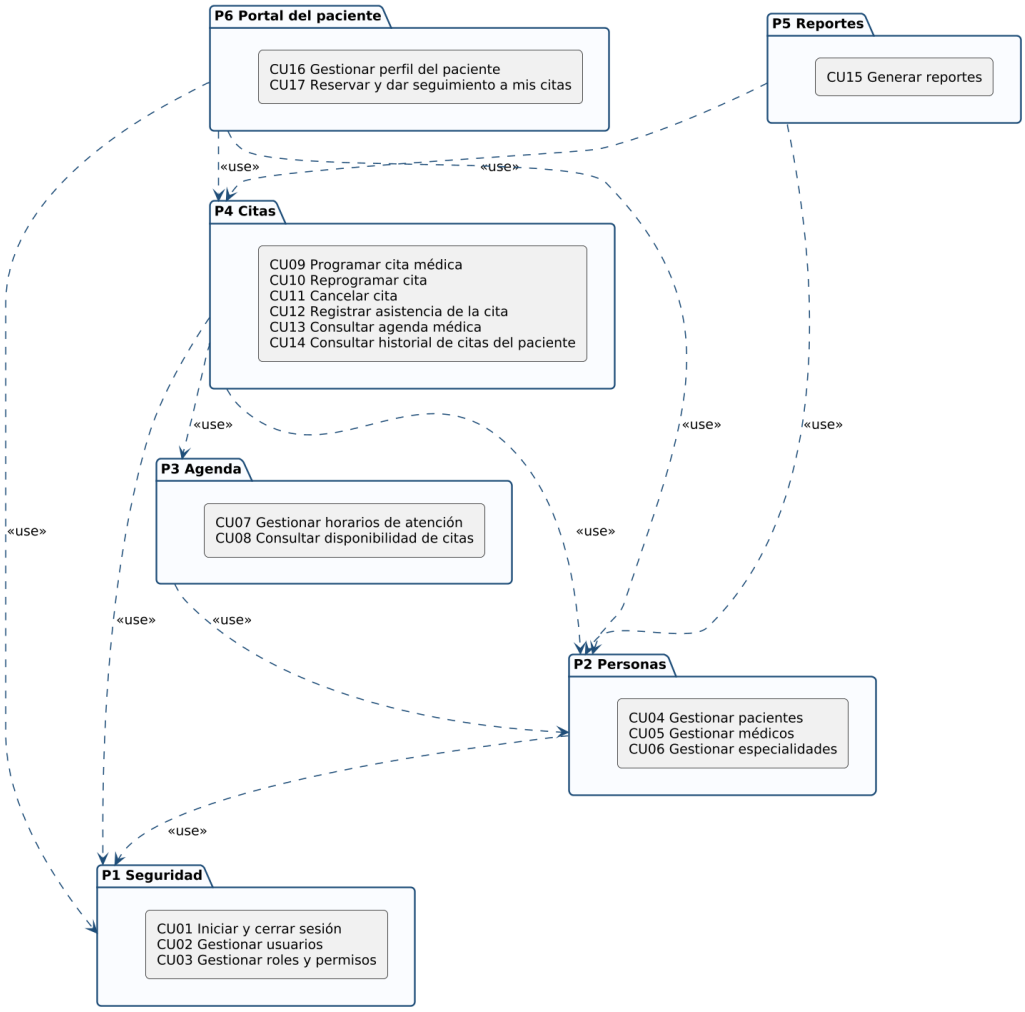


Figura 78. *Análisis de paquetes — dependencias*

3.4.1 Acoplamiento y cohesión

Dependencias entre paquetes:

- **Personas usa Seguridad:** los médicos y pacientes pueden tener una cuenta de usuario.
- **Agenda usa Personas:** cada horario pertenece a un médico activo.
- **Citas usa Personas:** cada cita referencia a un paciente, un médico y una especialidad.
- **Citas usa Agenda:** la cita debe estar dentro de un horario de atención y se usa el cálculo de disponibilidad.
- **Reportes usa Citas:** los reportes consultan las citas y sus estados.
- **Reportes usa Personas:** los reportes agrupan por médico, especialidad y pacientes.
- **Portal del paciente usa Citas:** el paciente reserva y cancela citas usando las mismas reglas del paquete Citas.
- **Portal del paciente usa Personas:** el perfil del paciente actualiza la entidad Paciente.
- **Portal del paciente usa Seguridad:** el paciente se registra e inicia sesión con una cuenta de usuario.
- **Citas usa Seguridad:** las operaciones registran al usuario responsable y verifican permisos.

Acoplamiento. Se mide con el acoplamiento eferente Ce (paquetes de los que depende), el acoplamiento aferente Ca (paquetes que dependen de él) y la inestabilidad I = Ce / (Ca + Ce). Un valor cercano a 0 indica un paquete estable del que dependen otros; un valor cercano a 1, un paquete que puede cambiar sin afectar a los demás.

Tabla 34. *Acoplamiento de los paquetes*

Paquete	Depende de (Ce)	Lo usan (Ca)	I	Interpretación
P1 Seguridad	— (0)	Personas, Portal del paciente, Citas (3)	0,00	Estable: base de la que dependen los demás.
P2 Personas	Seguridad (1)	Agenda, Citas, Reportes, Portal del paciente (4)	0,20	Estable: base de la que dependen los demás.
P3 Agenda	Personas (1)	Citas (1)	0,50	Intermedio: acoplamiento moderado y controlado.
P4 Citas	Personas, Agenda, Seguridad (3)	Reportes, Portal del paciente (2)	0,60	Intermedio: acoplamiento moderado y controlado.
P5 Reportes	Citas, Personas (2)	— (0)	1,00	Paquete hoja: puede cambiar sin afectar a otros.
P6 Portal del paciente	Citas, Personas, Seguridad (3)	— (0)	1,00	Paquete hoja: puede cambiar sin afectar a otros.

Las dependencias apuntan hacia los paquetes más estables (Seguridad y Personas) y no existen ciclos entre paquetes, lo que reduce el impacto de los cambios. Citas es el paquete central del dominio y concentra las reglas de disponibilidad.

Cohesión. Un paquete es cohesivo cuando sus casos de uso trabajan sobre las mismas entidades y responden a un mismo propósito.

Tabla 35. *Cohesión de los paquetes*

Paquete	Entidades que comparten	Cohesión
P1 Seguridad	usuario, rol, permiso, bitacora	Alta: todos tratan el acceso y la autorización.
P2 Personas	paciente, medico, especialidad	Alta: registros maestros de la clínica.
P3 Agenda	horario_atencion, bloqueo_agenda	Alta: definen y calculan la disponibilidad.
P4 Citas	cita, historial_estado_cita	Alta: todos operan sobre el ciclo de vida de la cita.
P5 Reportes	vistas v_agenda_medica, v_resumen_estados	Alta: solo consultas de resumen.
P6 Portal del paciente	paciente, contacto_emergencia, cita	Media-alta: autoservicio del paciente; reutiliza reglas de Citas.

CAPÍTULO 4: FLUJO DE TRABAJO: DISEÑO

En el diseño se adapta el modelo de análisis a la plataforma de implementación (PHP, Apache y PostgreSQL). Se define la arquitectura física y lógica, se realizan los casos de uso mediante diagramas de secuencia y se diseña la base de datos a nivel lógico y físico.

4.1 Diseño de la arquitectura

4.1.1 Diseño físico de la arquitectura

El sistema se despliega en tres nodos: el dispositivo cliente con un navegador web; el servidor de aplicaciones con Apache y PHP (XAMPP), donde residen las vistas, controladores y modelos; y el servidor de base de datos PostgreSQL. El navegador se comunica con Apache por HTTP/HTTPS y PHP accede a PostgreSQL mediante PDO (pdo_pgsql) por TCP en el puerto 5432. En el entorno de desarrollo los dos servidores pueden estar en la misma computadora.

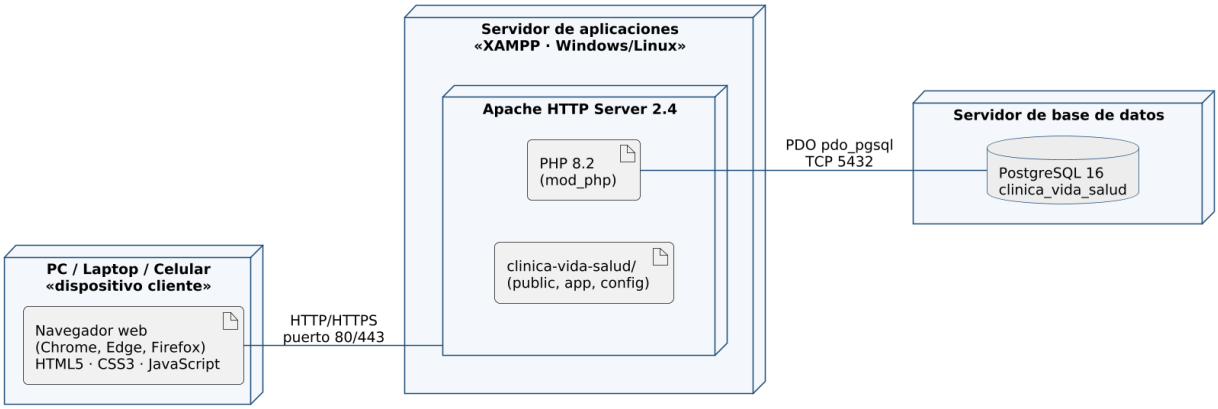


Figura 79. *Diagrama de despliegue*

Tabla 36. *Nodos del diagrama de despliegue*

Nodo	Software	Responsabilidad
Dispositivo cliente	Navegador (Chrome, Edge, Firefox)	Mostrar las vistas HTML/CSS y ejecutar las validaciones JavaScript.
Servidor de aplicaciones	Apache 2.4 + PHP 8.2 (XAMPP)	Recibir las peticiones, controlar sesiones y permisos, ejecutar controladores y modelos.
Servidor de base de datos	PostgreSQL 16	Persistir los datos y garantizar la integridad (PK, FK, CHECK, EXCLUDE, triggers).

4.1.2 Diseño lógico de la arquitectura

La arquitectura lógica se organiza en paquetes en capas siguiendo el patrón MVC: la capa de presentación contiene las vistas de cada paquete; la capa de negocio, los controladores (uno por caso de uso o grupo de casos de uso relacionados); y la capa de datos, los modelos que acceden a PostgreSQL a través del núcleo compartido (core) con la conexión PDO, la autenticación, las sesiones y la validación. Cada capa solo depende de la capa inferior.

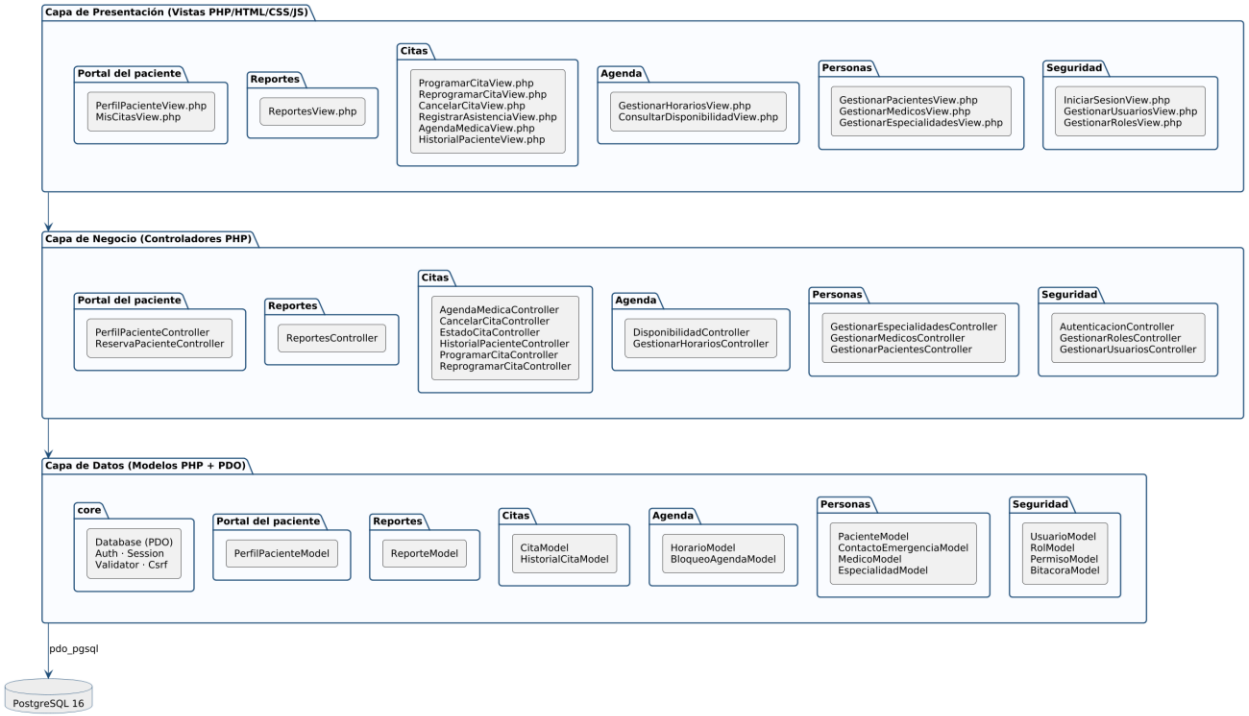


Figura 80. *Diagrama de paquetes organizado en capas*

4.2 Diseñar casos de uso

Cada caso de uso se realiza en el diseño mediante un diagrama de secuencia que muestra la interacción entre el actor, la vista PHP/HTML, el controlador, los modelos y la base de datos. Las operaciones que modifican varias filas o que deben verificar disponibilidad se ejecutan dentro de una transacción (BEGIN/COMMIT/ROLLBACK); el bloque alt muestra el flujo de excepción principal.

Las transiciones válidas de una cita, que controlan CU10, CU11 y CU12, se resumen en el siguiente diagrama de estados:

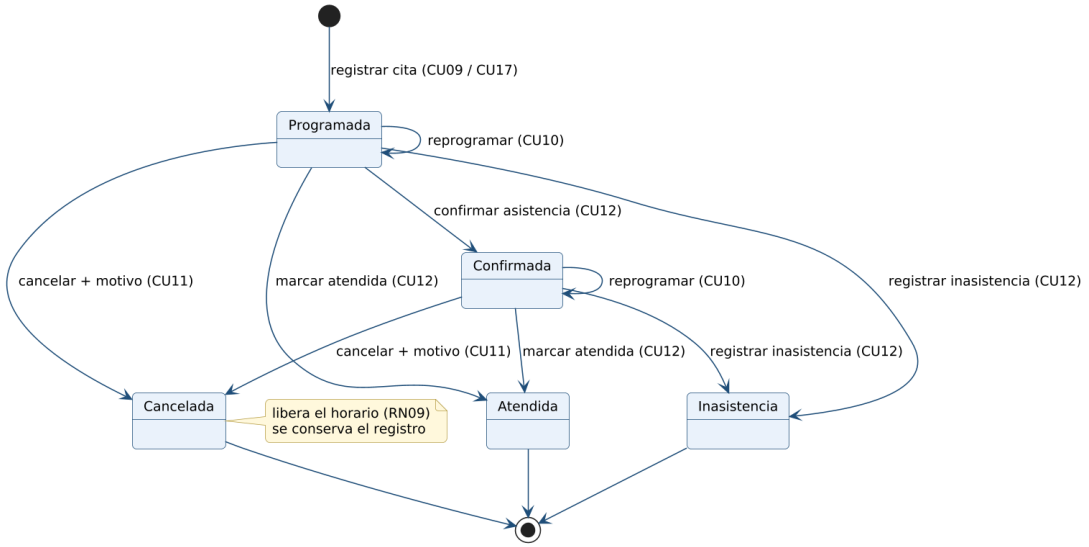


Figura 81. *Diagrama de estados de la cita (RF10)*

Los diagramas de secuencia muestran, en orden temporal, los mensajes entre los objetos que realizan cada caso de uso. Cada diagrama se encierra en un marco «sd» con el nombre de la interacción y usa los mismos objetos, mensajes y números que el diagrama de comunicación y el de clases de análisis del caso de uso. Las líneas de vida se identifican con los estereotipos «frm» (interfaz IU_X, pantalla del frontend en JavaScript), «ctrl» (control CTR_X, clase del caso de uso en PHP; debajo de cada mensaje de la interfaz al control se indica la ruta HTTP que lo invoca) y «entidad» (tabla y funciones del repositorio). Los mensajes síncronos se dibujan con punta llena, las respuestas con línea punteada y los focos de control con rectángulos sobre la línea de vida. Los fragmentos combinados alt (alternativas), opt (opcional) y loop (repetición) representan las validaciones, los errores y los recorridos; las notas indican las restricciones y los disparadores de la base de datos que actúan en ese paso.

Ciclo #1

CU01: Iniciar y cerrar sesión

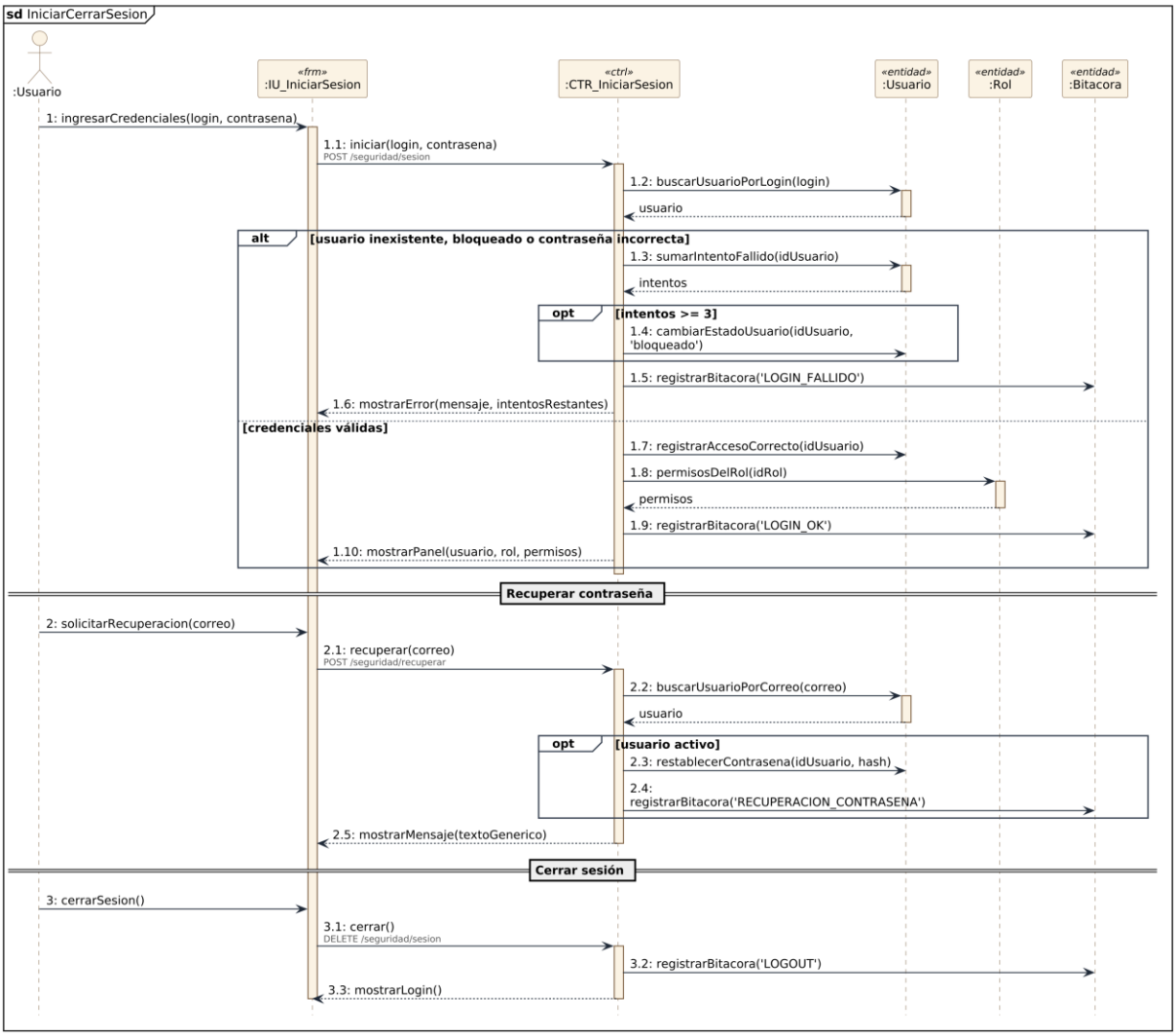


Figura 82. *Diagrama de secuencia — CU01 Iniciar y cerrar sesión*

CU02: Gestionar usuarios

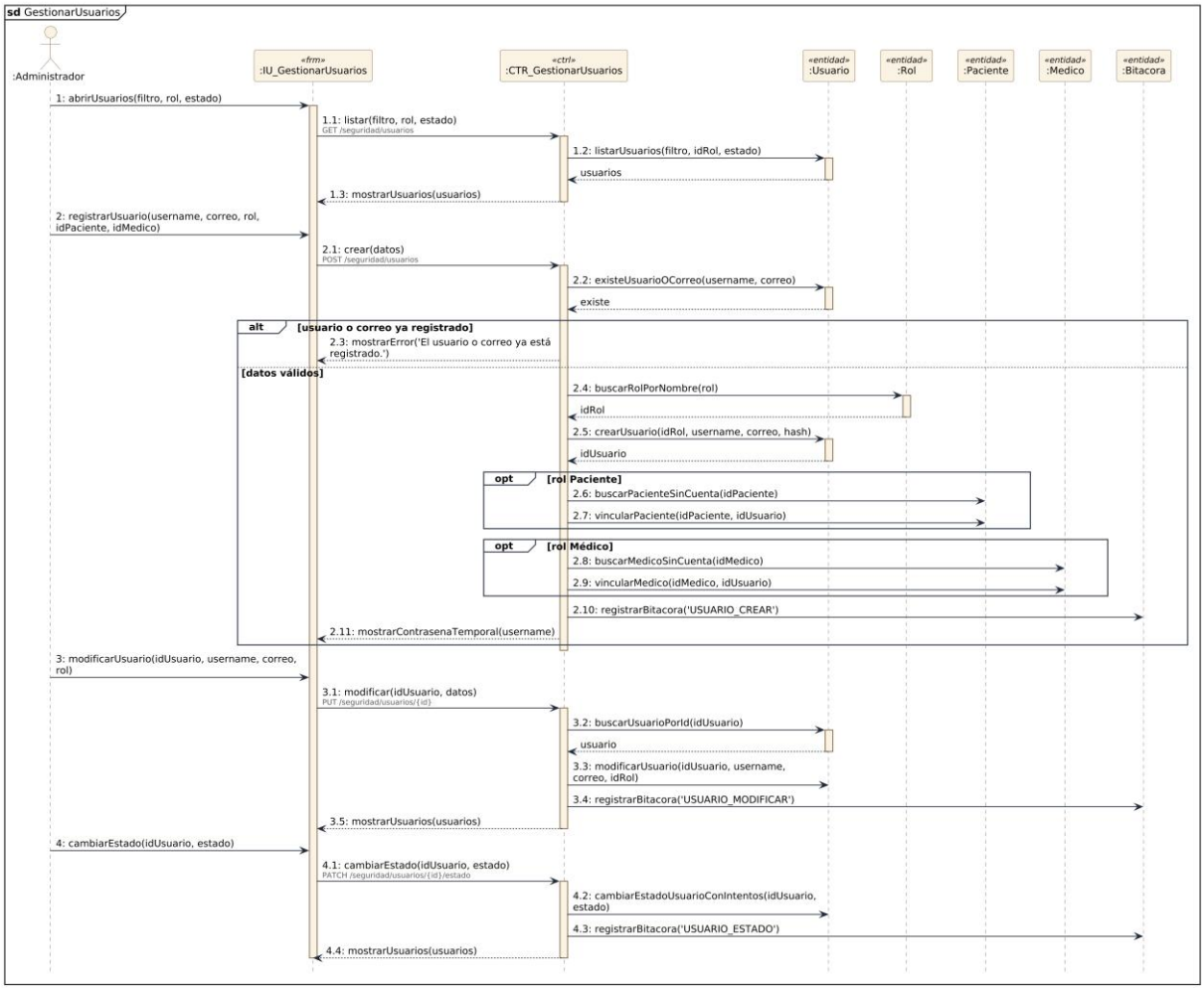


Figura 83. *Diagrama de secuencia — CU02 Gestionar usuarios*

CU03: Gestionar roles y permisos

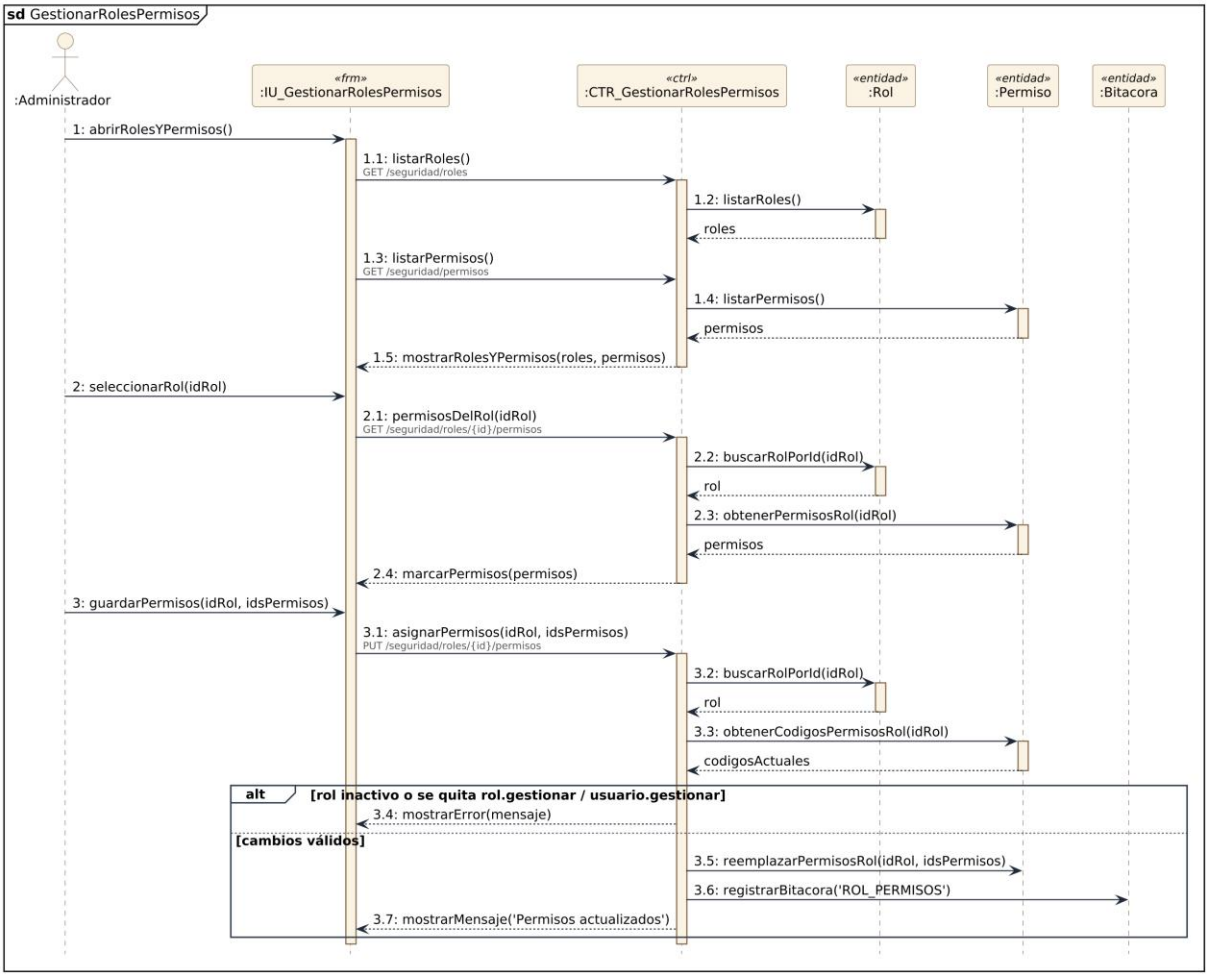


Figura 84. *Diagrama de secuencia — CU03 Gestionar roles y permisos*

CU04: Gestionar pacientes

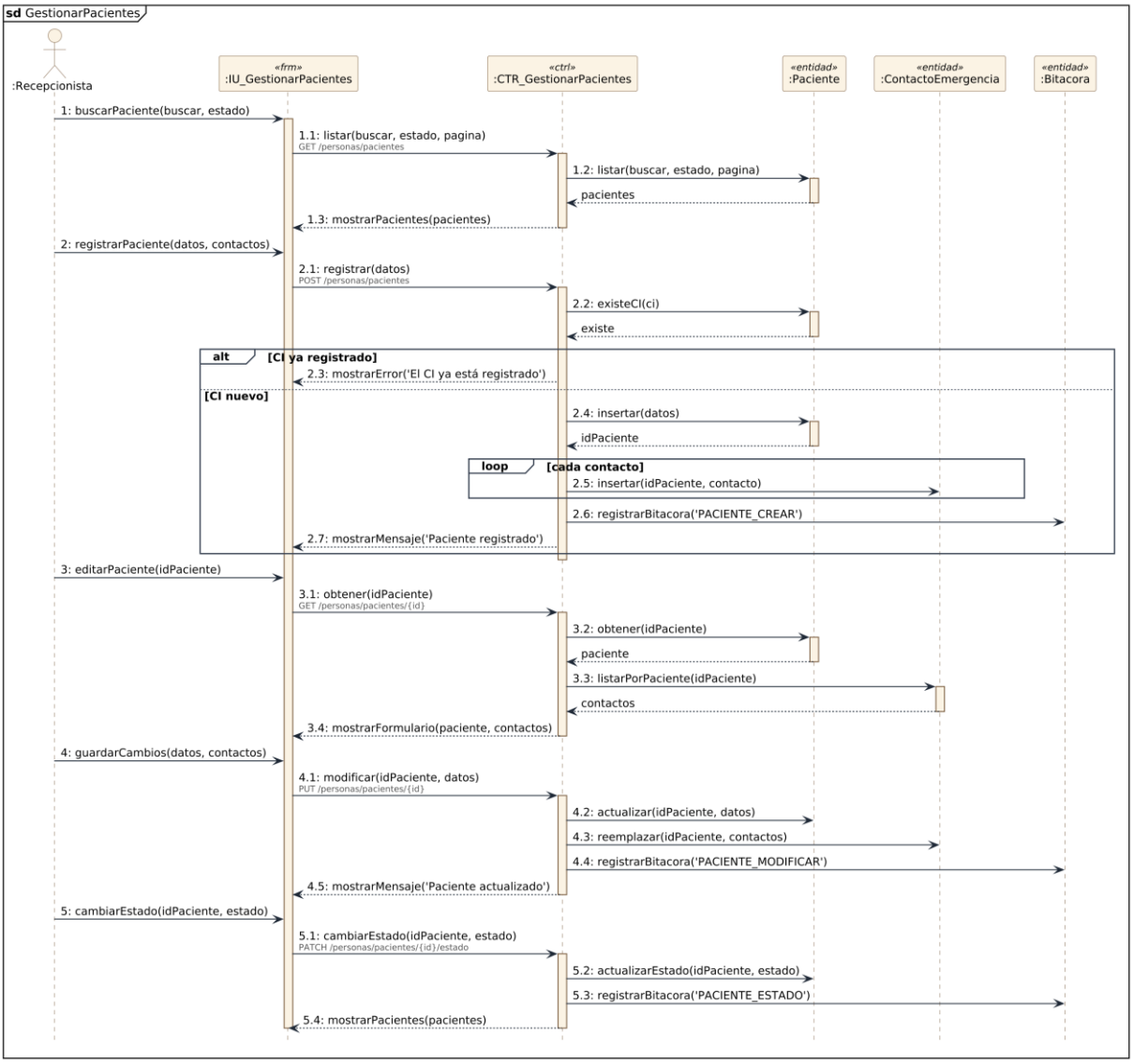


Figura 85. *Diagrama de secuencia — CU04 Gestionar pacientes*

CU05: Gestionar médicos

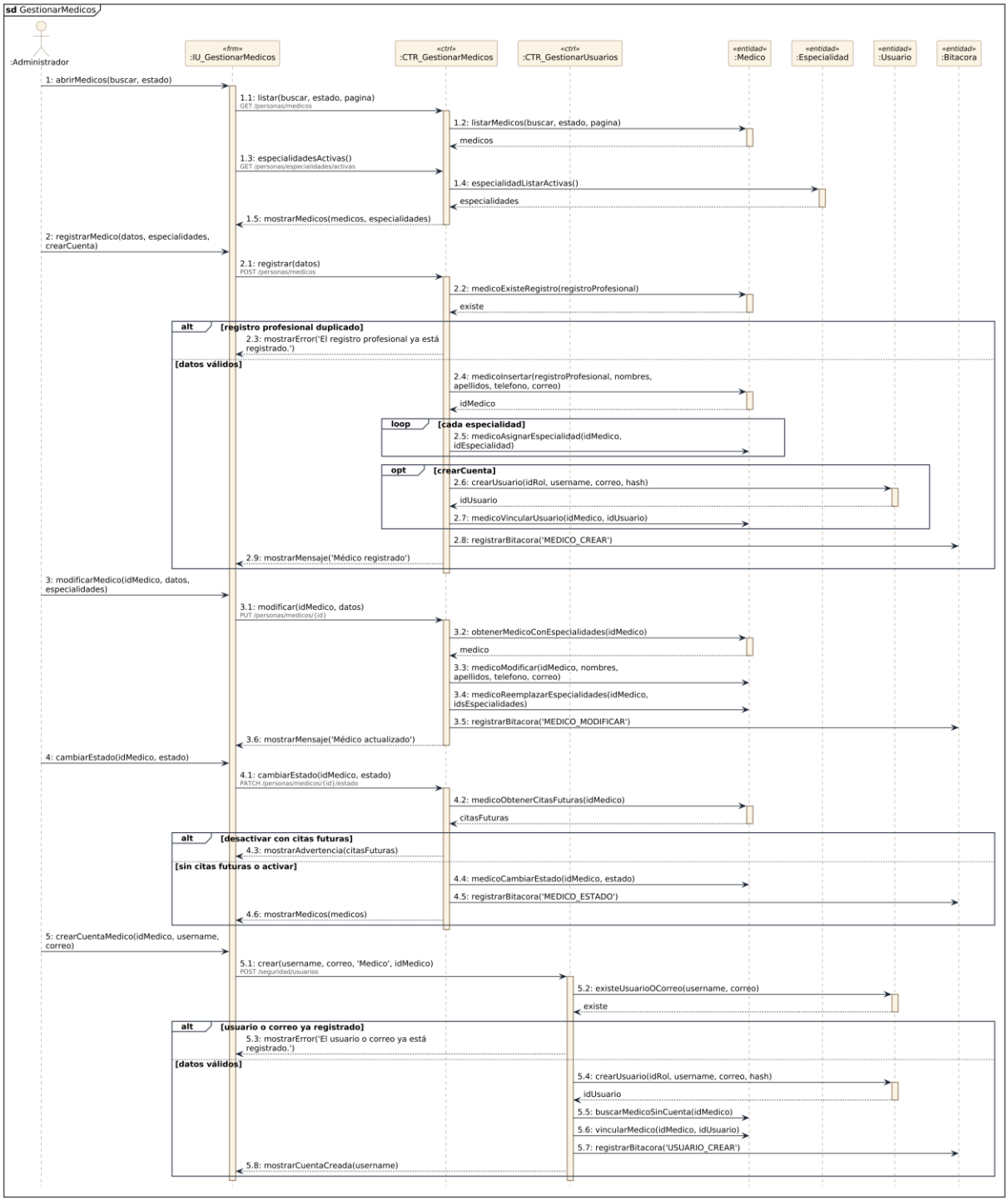


Figura 86. *Diagrama de secuencia — CU05 Gestionar médicos*

CU06: Gestionar especialidades

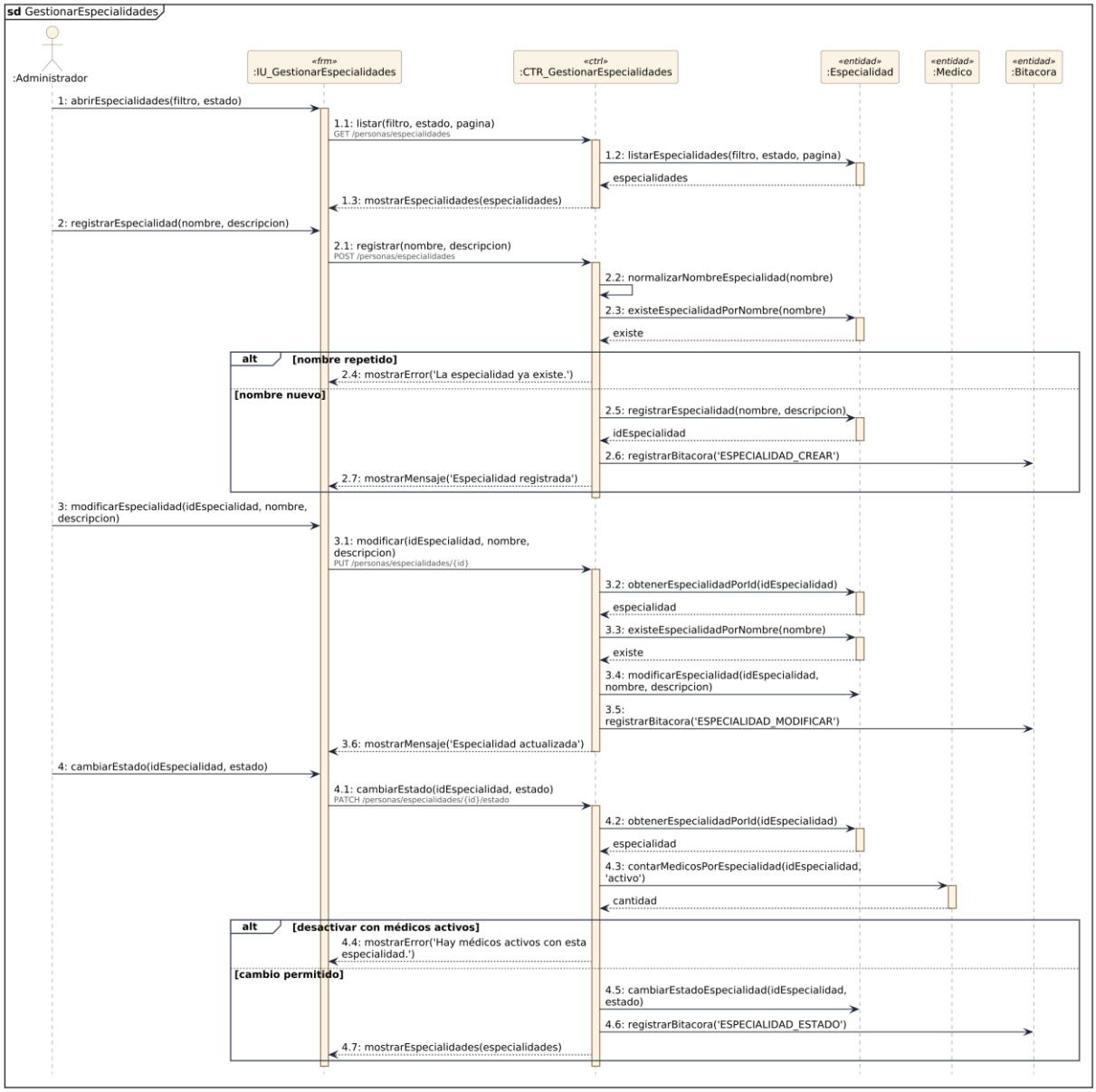


Figura 87. *Diagrama de secuencia — CU06 Gestionar especialidades*

CU07: Gestionar horarios de atención

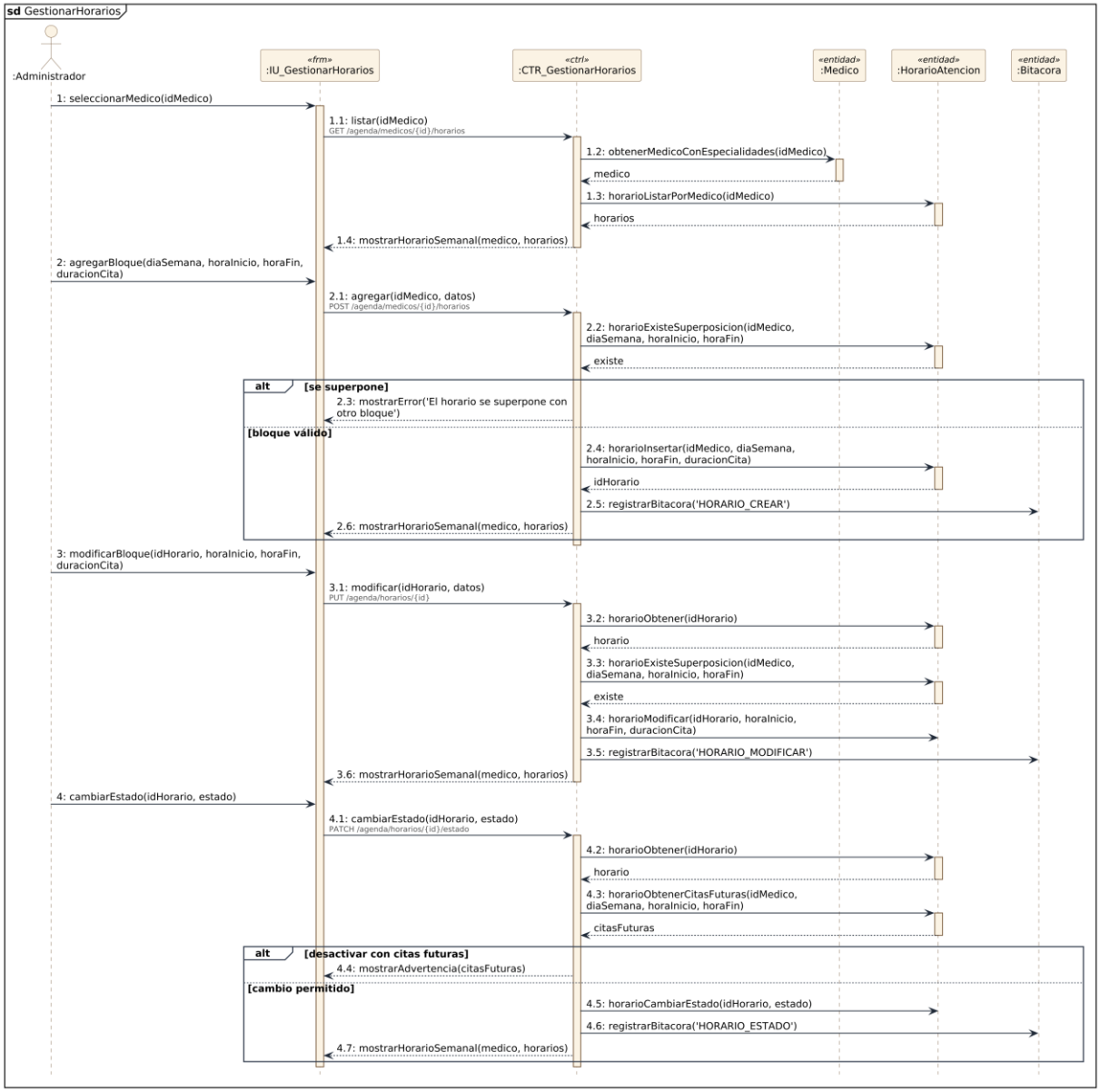


Figura 88. *Diagrama de secuencia — CU07 Gestionar horarios de atención*

CU08: Consultar disponibilidad de citas

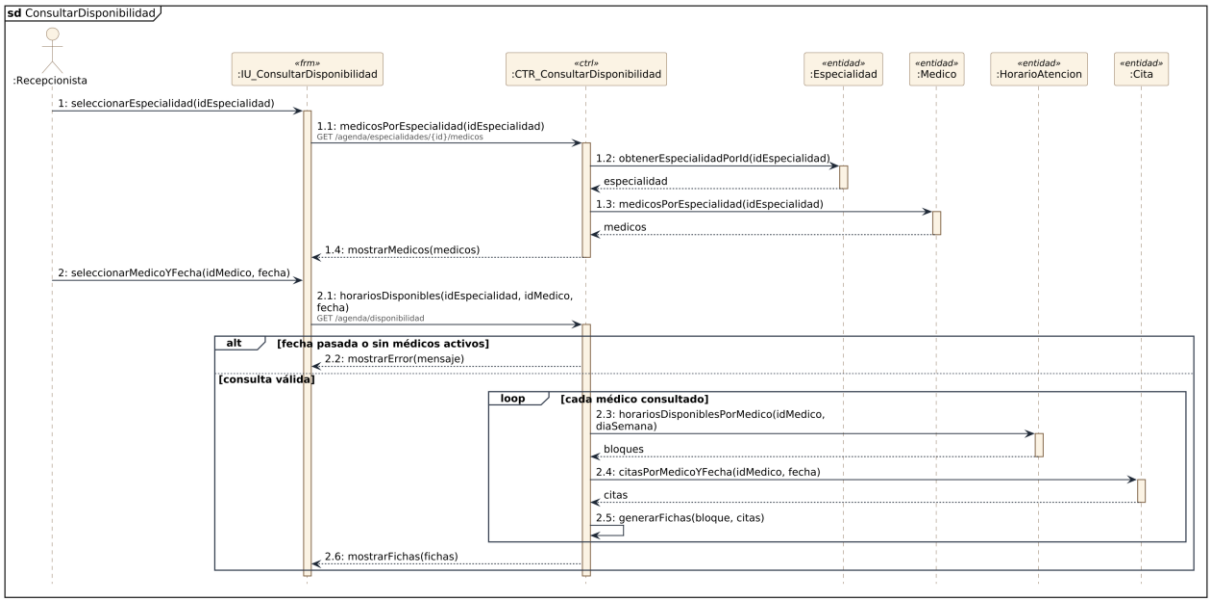


Figura 89. *Diagrama de secuencia — CU08 Consultar disponibilidad de citas*

CU09: Programar cita médica

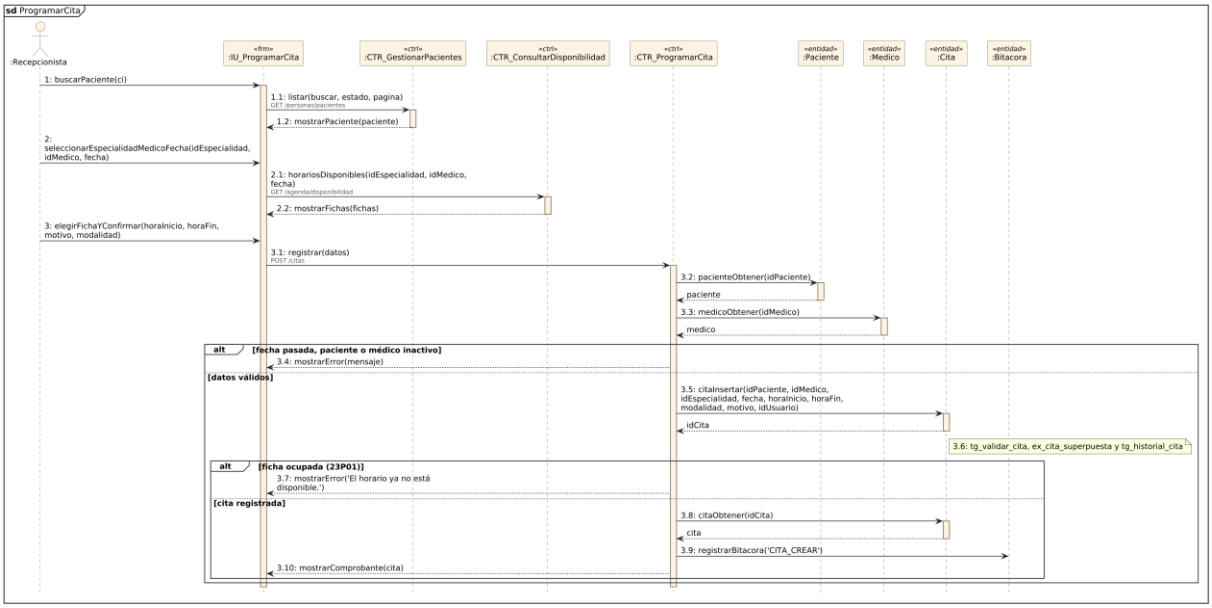


Figura 90. *Diagrama de secuencia — CU09 Programar cita médica*

Ciclo #2

CU10: Reprogramar cita

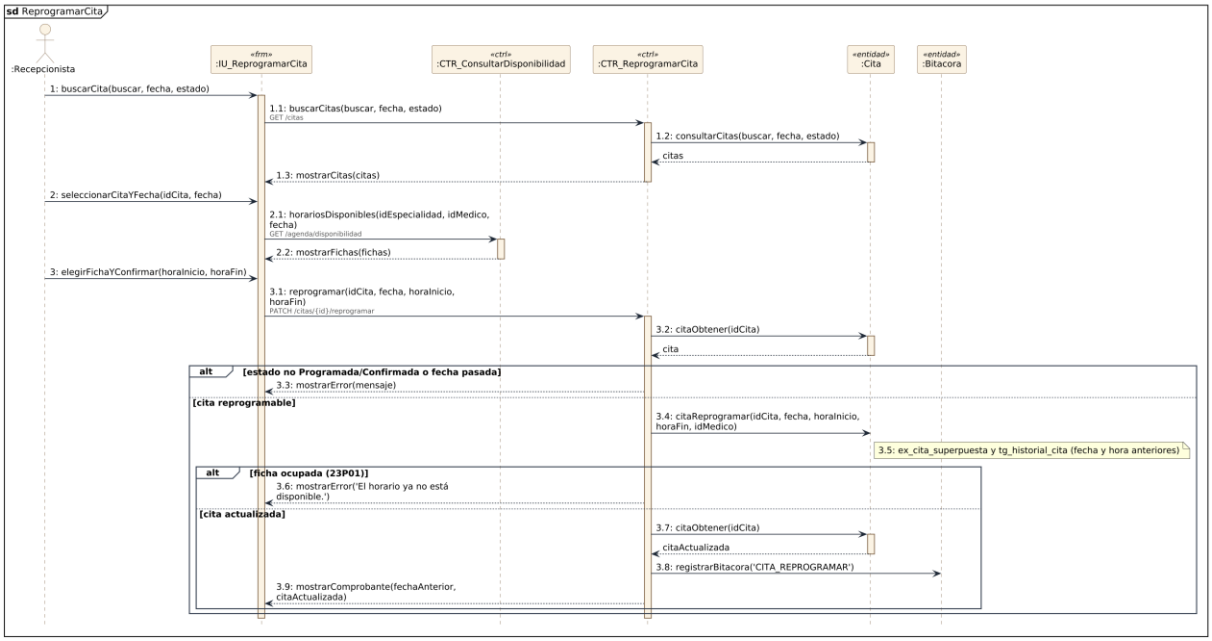


Figura 91. *Diagrama de secuencia — CU10 Reprogramar cita*

CU11: Cancelar cita

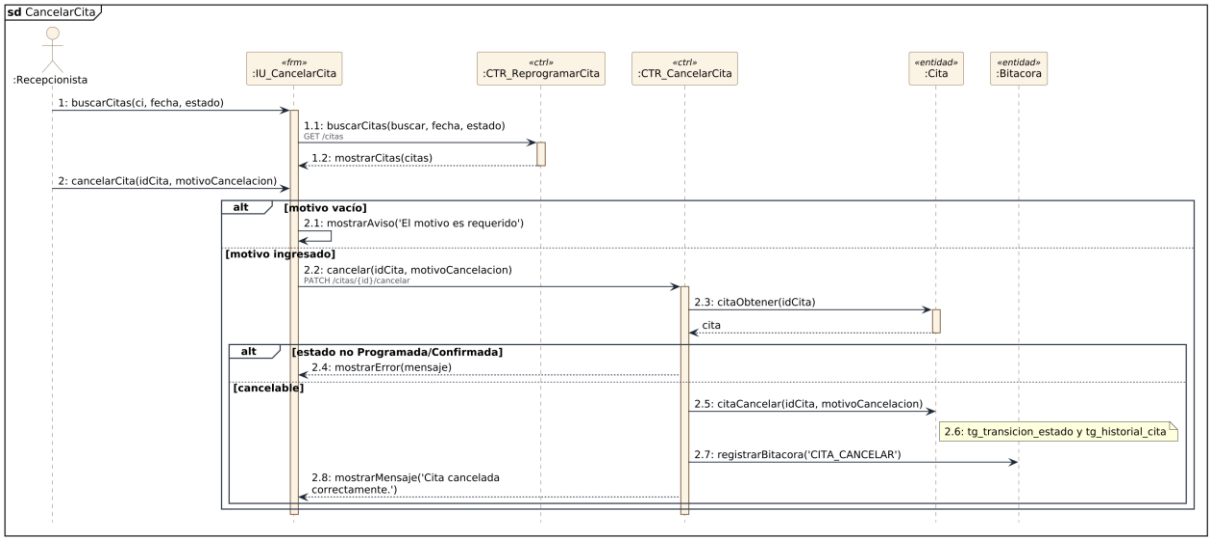


Figura 92. *Diagrama de secuencia — CU11 Cancelar cita*

CU12: Registrar asistencia de la cita

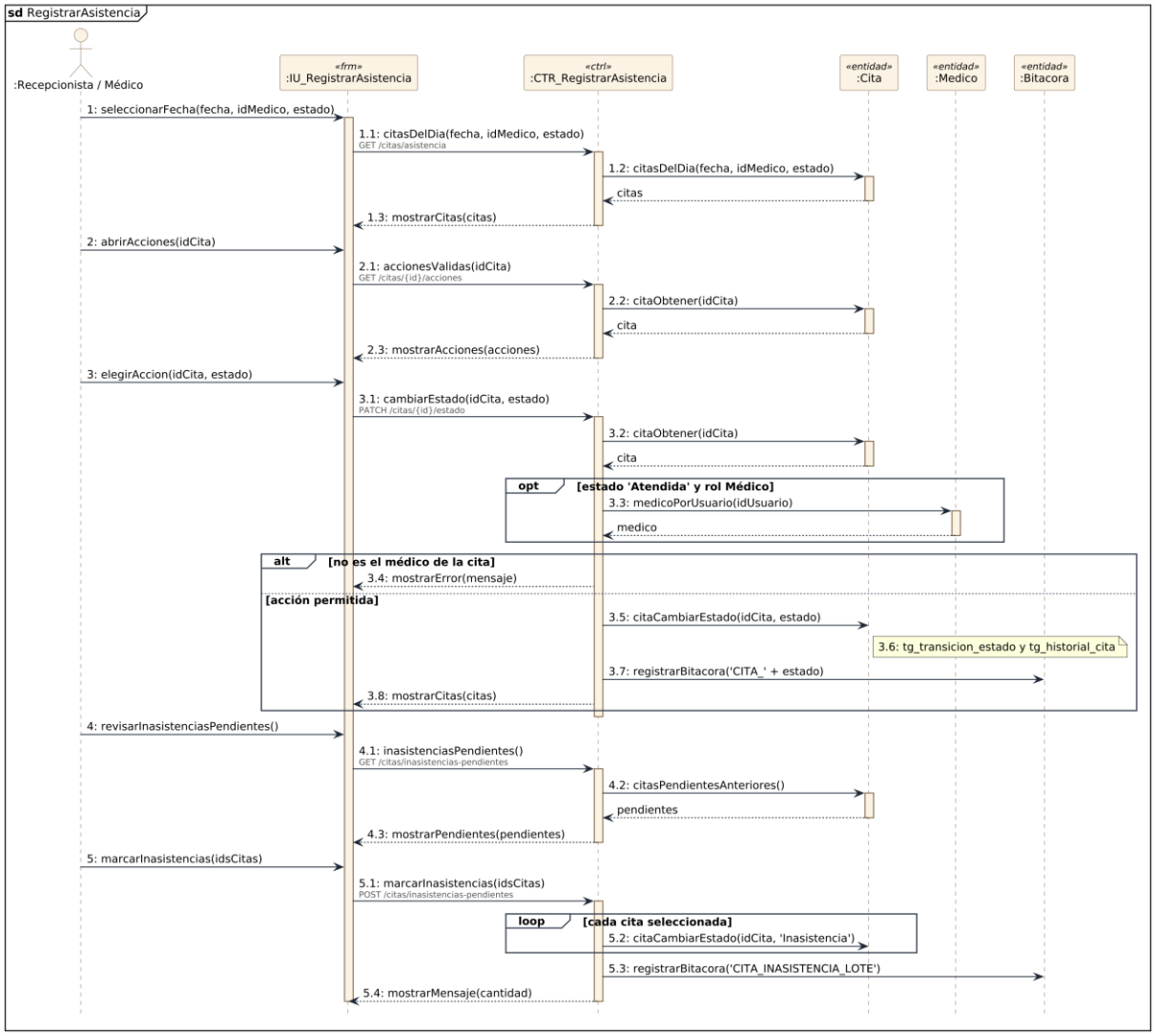


Figura 93. *Diagrama de secuencia — CU12 Registrar asistencia de la cita*

CU13: Consultar agenda médica

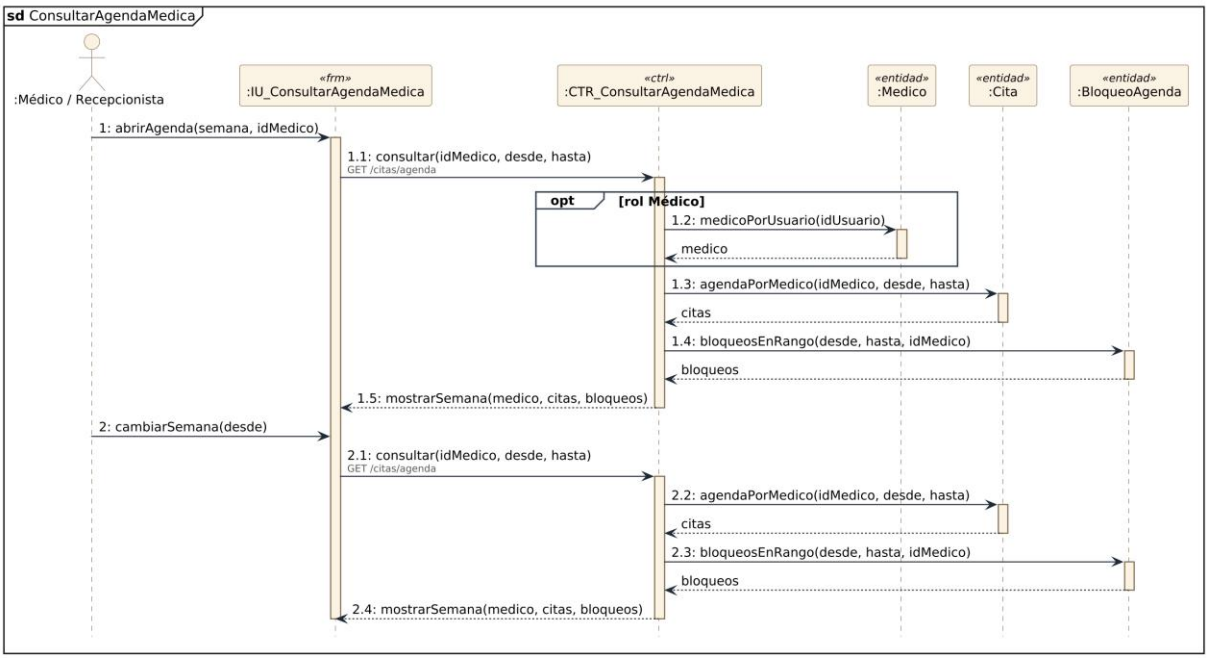


Figura 94. *Diagrama de secuencia — CU13 Consultar agenda médica*

CU14: Consultar historial de citas del paciente

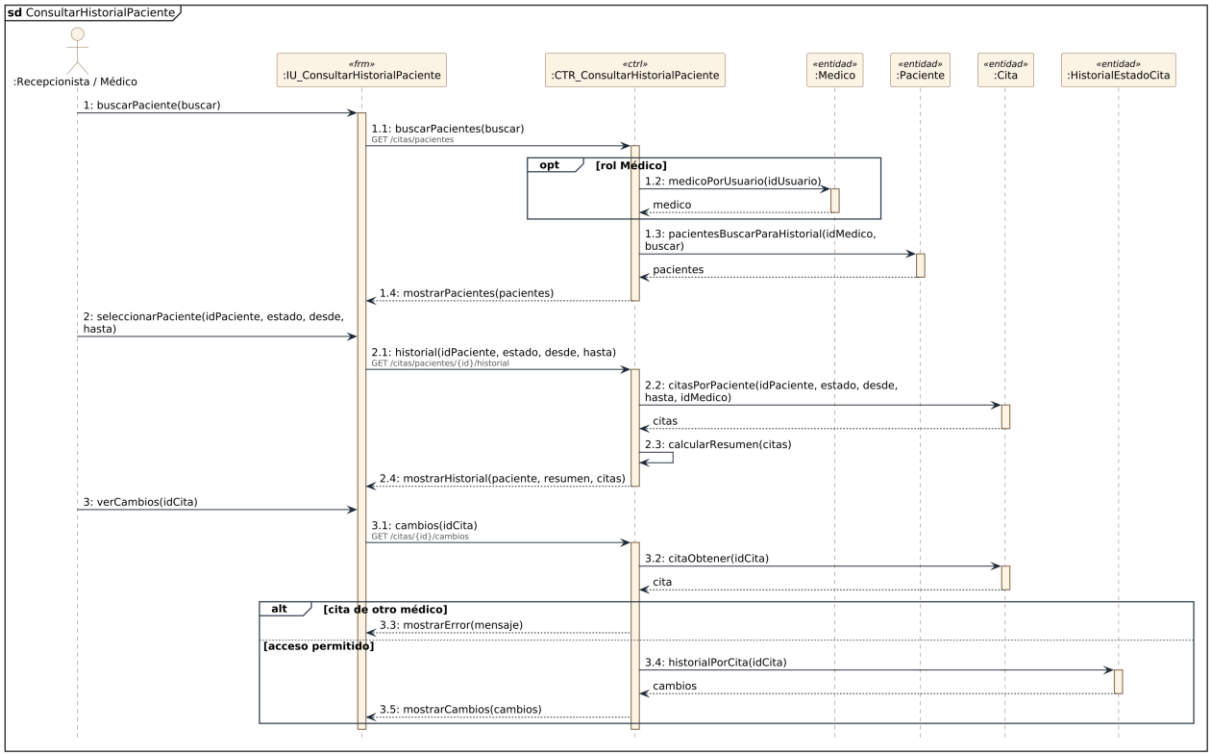


Figura 95. *Diagrama de secuencia — CU14 Consultar historial de citas del paciente*

CU15: Generar reportes

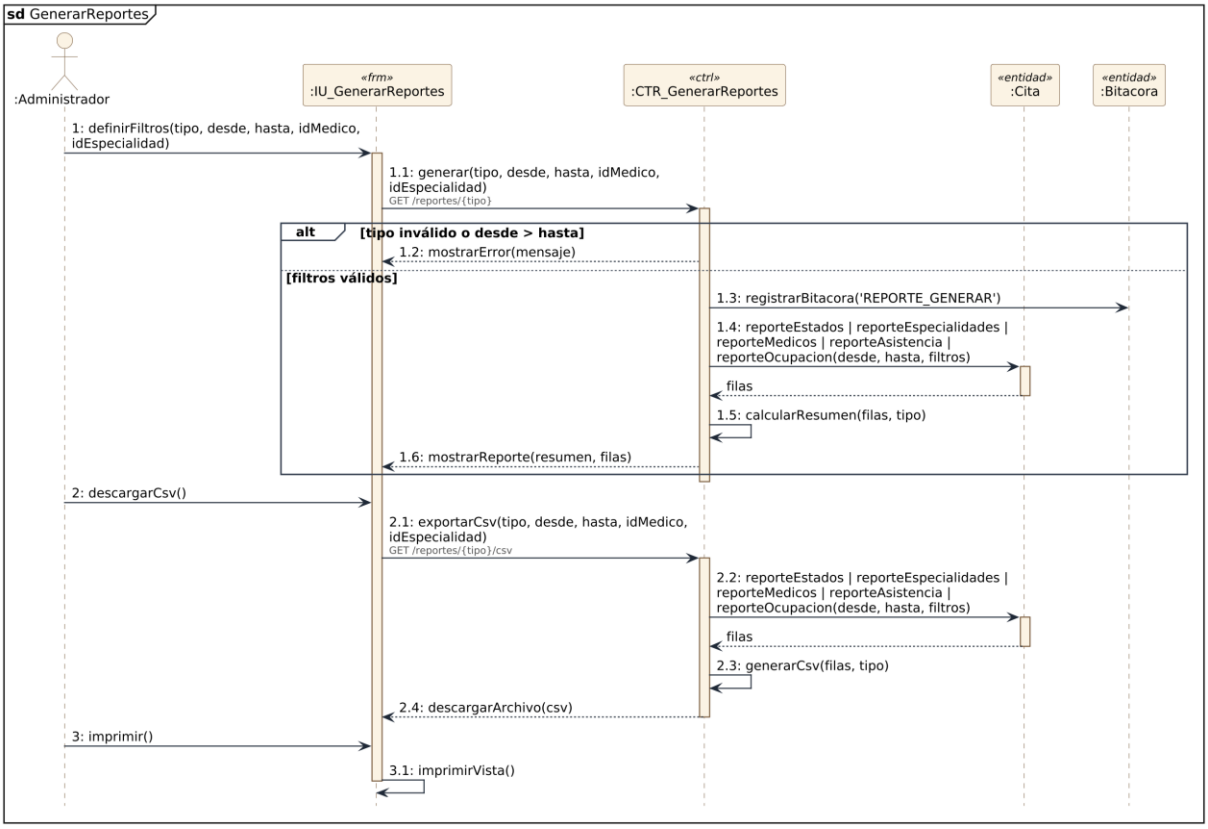


Figura 96. *Diagrama de secuencia — CU15 Generar reportes*

CU16: Gestionar perfil del paciente

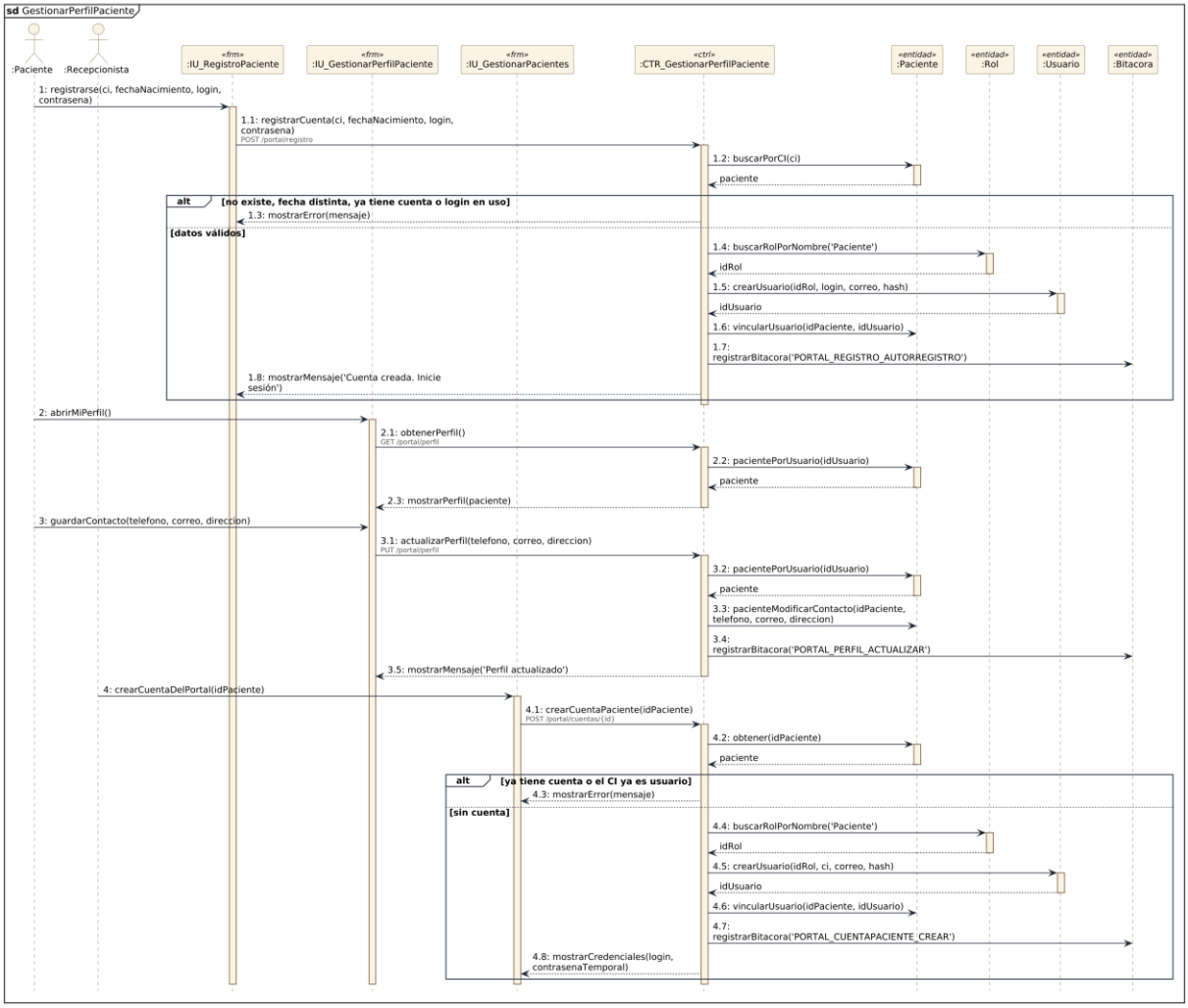


Figura 97. *Diagrama de secuencia — CU16 Gestionar perfil del paciente*

CU17: Reservar y dar seguimiento a mis citas

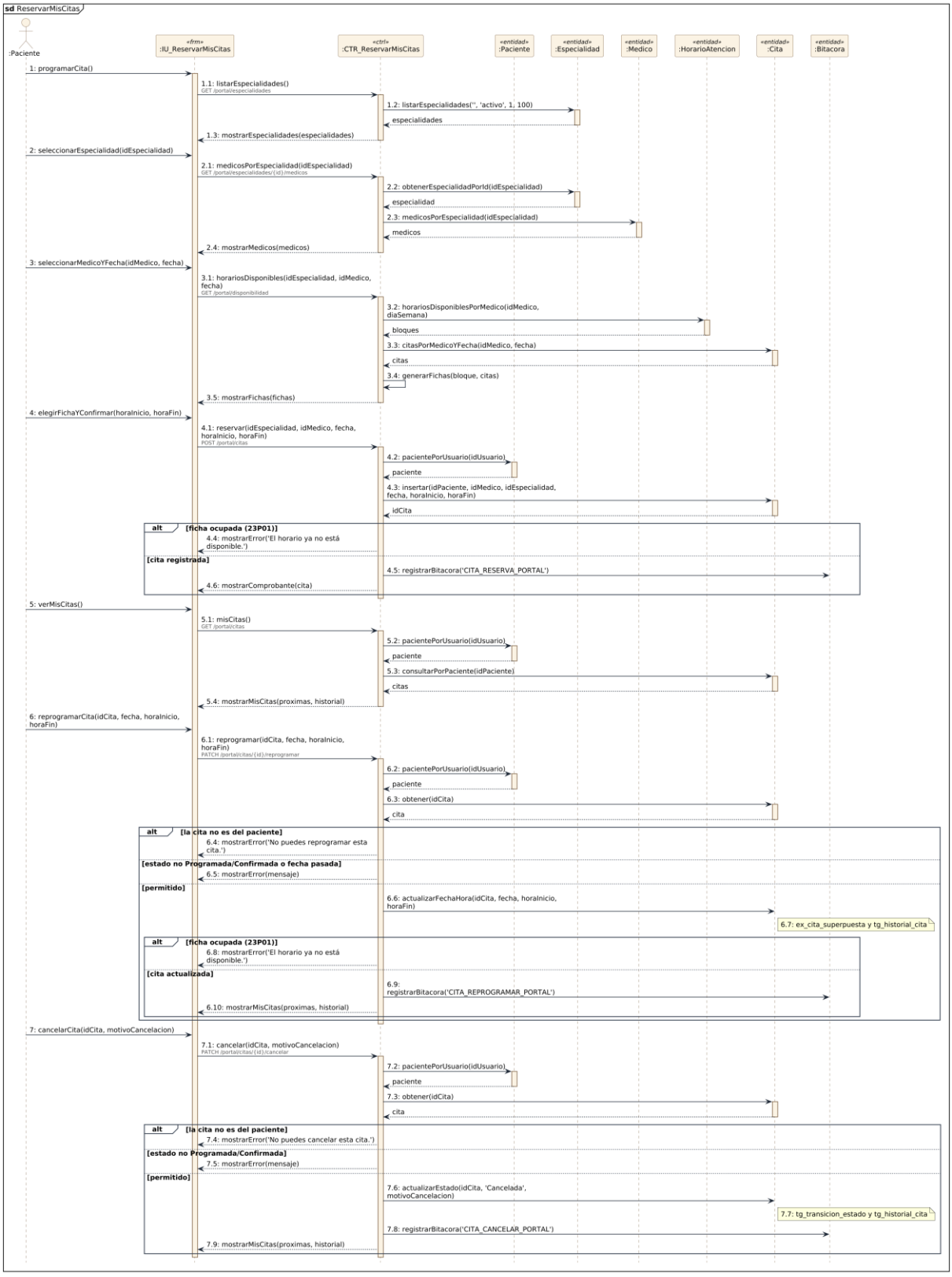


Figura 98. *Diagrama de secuencia — CU17 Reservar y dar seguimiento a mis citas*

4.3 Diseño de datos

4.3.1 Diseño lógico de datos

Diagrama de clases

El diagrama de clases representa la estructura estática del modelo de datos: las clases persistentes, sus atributos y las asociaciones con sus multiplicidades. Las relaciones principales son: un paciente puede tener muchas citas; un médico puede atender muchas citas; un médico tiene una o varias especialidades y una especialidad puede estar asociada a varios médicos; un médico puede tener varios horarios de atención; y un usuario inicia sesión y accede a las operaciones permitidas por su rol.

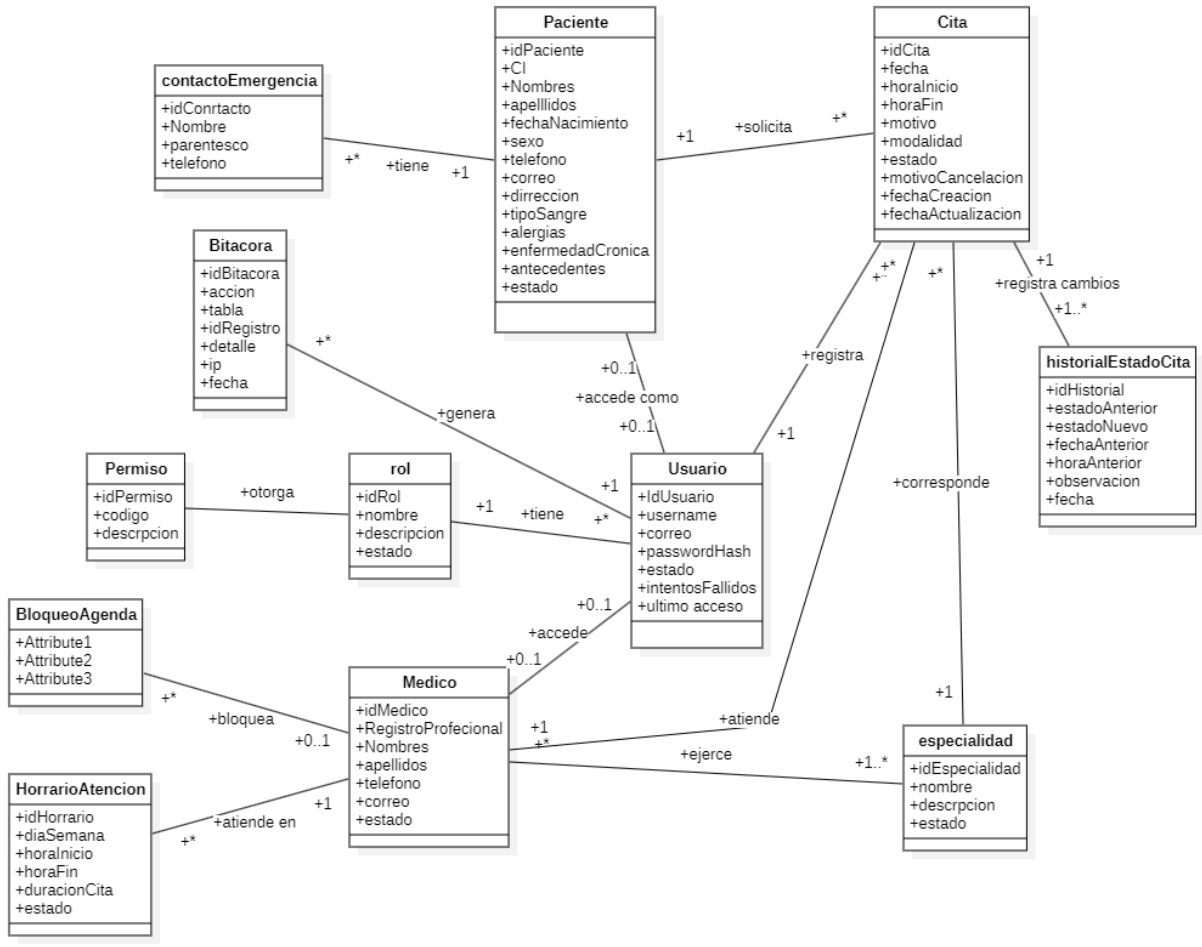


Figura 99. *Diagrama de clases del dominio (página girada)*

Mapeo

Cada clase se transforma en una tabla y cada asociación muchos a muchos en una tabla intermedia. Las asociaciones uno a muchos se implementan con una llave foránea en la tabla del lado «muchos».

ROL			
PK			

ID_ROL	NOMBRE	DESCRIPCION	ESTADO

PERMISO		
PK		
ID_PERMISO	CODIGO	DESCRIPCION

ROL_PERMISO	
PK, FK	PK, FK
ID_ROL	ID_PERMISO

USUARIO						
PK	FK					
ID_USUARIO	ID_ROL	USERNAME	CORREO	PASSWORD_HASH	ESTADO	INTENTOS_FALLIDOS

ULTIMO_ACCESO

BITACORA							
PK	FK						
ID_BITACORA	ID_USUARIO	ACCION	TABLA	ID_REGISTRO	DETALLE	IP	FECHA

PACIENTE						
PK						
ID_PACIENTE	CI	NOMBRES	APELLIDOS	FECHA_NACIMIENTO	SEXO	TELEFONO

CORREO	DIRECCION	TIPO_SANGRE	ALERGIAS	ENFERMEDAD_CRONICA	ANTECEDENTES

FK	
ID_USUARIO	ESTADO

CONTACTO_EMERGENCIA				
PK	FK			
ID_CONTACTO	ID_PACIENTE	NOMBRE	PARENTESCO	TELEFONO

ESPECIALIDAD			
PK			
ID_ESPECIALIDAD	NOMBRE	DESCRIPCION	ESTADO

MEDICO					
PK					
ID_MEDICO	REGISTRO_PROFESIONAL	NOMBRES	APELLIDOS	TELEFONO	CORREO

FK	
ID_USUARIO	ESTADO

MEDICO_ESPECIALIDAD	
PK, FK	PK, FK
ID_MEDICO	ID_ESPECIALIDAD

HORARIO_ATENCION						
PK	FK					
ID_HORARIO	ID_MEDICO	DIA_SEMANA	HORA_INICIO	HORA_FIN	DURACION_CITA	ESTADO

BLOQUEO_AGENDA			
PK	FK		
ID_BLOQUEO	ID_MEDICO	FECHA	MOTIVO

CITA						
PK	FK	FK	FK			
ID_CITA	ID_PACIENTE	ID_MEDICO	ID_ESPECIALIDAD	FECHA	HORA_INICIO	HORA_FIN

				FK
MOTIVO	MODALIDAD	ESTADO	MOTIVO_CANCELACION	ID_USUARIO_REGISTRO

FECHA_CREACION	FECHA_ACTUALIZACION

HISTORIAL_ESTADO_CITA				
PK	FK			
ID_HISTORIAL	ID_CITA	ESTADO_ANTERIOR	ESTADO_NUEVO	FECHA_ANTERIOR

		FK	
HORA_ANTERIOR	OBSERVACION	ID_USUARIO	FECHA

Normalización

- **Primera forma normal (1FN):** todos los atributos son atómicos. Las especialidades de un médico, que son multivaluadas, no se guardan como lista en medico sino en la tabla medico_especialidad; los contactos de emergencia se separan en contacto_emergencia; los permisos de un rol, en rol_permiso.
- **Segunda forma normal (2FN):** en las tablas con llave compuesta (rol_permiso, medico_especialidad) no existen atributos que dependan solo de una parte de la llave; el resto de las tablas tiene llave simple.
- **Tercera forma normal (3FN):** no hay dependencias transitivas: la cita guarda solo las llaves foráneas del paciente, médico y especialidad, no sus nombres; el nombre del rol se obtiene de rol mediante id_rol; la duración de la cita depende del horario de atención y no se repite en cada médico.
- **Datos derivados:** los horarios disponibles no se almacenan; se calculan con la función fn_horarios_disponibles a partir del horario y las citas ocupadas, evitando inconsistencias.

4.3.2 Diseño físico de datos

Tabla de volumen

La tabla de volumen describe, para cada clase persistente, sus atributos con el tipo de dato, si son llave primaria (PK), llave foránea (FK) u obligatorios (Not Null), y su descripción.

Tabla 38. *Tabla de volumen de la base de datos clinica_vida_salud*

Clase	Atributo	Tipo de Dato	PK	FK	Not Null	Descripción
ROL	ID_ROL	serial	SI	NO	SI	Código único que identifica cada rol dentro del sistema
	NOMBRE	varchar(30)	NO	NO	SI	Nombre descriptivo del rol asignado
	DESCRIPCION	varchar(150)	NO	NO	NO	Detalle de las funciones o permisos que tiene el rol
	ESTADO	varchar(10)	NO	NO	SI	Indica si el rol está activo o inactivo
PERMISO	ID_PERMISO	serial	SI	NO	SI	Identificador único del permiso
	CODIGO	varchar(40)	NO	NO	SI	Código de la operación autorizada (p. ej. cita.programar)
	DESCRIPCION	varchar(150)	NO	NO	SI	Descripción de la operación que permite realizar
ROL_PERMISO	ID_ROL	int	SI	SI	SI	Rol al que se asigna el permiso
	ID_PERMISO	int	SI	SI	SI	Permiso asignado al rol
USUARIO	ID_USUARIO	serial	SI	NO	SI	Identificador único del usuario
	ID_ROL	int	NO	SI	SI	Rol que determina los permisos del usuario
	USERNAME	varchar(40)	NO	NO	SI	Nombre de usuario para iniciar sesión
	CORREO	varchar(100)	NO	NO	SI	Correo electrónico del usuario, también válido para iniciar sesión
	PASSWORD_HASH	varchar(255)	NO	NO	SI	Contraseña cifrada con password_hash()
	ESTADO	varchar(10)	NO	NO	SI	Estado de la cuenta: activo, inactivo o bloqueado
	INTENTOS_FALLIDOS	smallint	NO	NO	SI	Número de intentos fallidos de inicio de sesión
	ULTIMO_ACCESO	timestamp	NO	NO	NO	Fecha y hora del último inicio de sesión
BITACORA	ID_BITACORA	bigserial	SI	NO	SI	Identificador único del registro de bitácora
	ID_USUARIO	int	NO	SI	NO	Usuario que realizó la operación
	ACCION	varchar(60)	NO	NO	SI	Operación realizada (inicio de sesión, registro, modificación)
	TABLA	varchar(40)	NO	NO	NO	Tabla afectada por la operación
	ID_REGISTRO	int	NO	NO	NO	Identificador del registro afectado

Clase	Atributo	Tipo de Dato	PK	FK	Not Null	Descripción
	DETALLE	text	NO	NO	NO	Información adicional de la operación
	IP	varchar(45)	NO	NO	NO	Dirección IP desde la que se realizó la operación
	FECHA	timestamp	NO	NO	SI	Fecha y hora de la operación
PACIENTE	ID_PACIENTE	serial	SI	NO	SI	Identificador único del paciente
	CI	bigint	NO	NO	NO	Cédula de identidad del paciente; única cuando se proporciona
	NOMBRES	varchar(60)	NO	NO	SI	Nombres del paciente
	APELLIDOS	varchar(60)	NO	NO	SI	Apellidos del paciente
	FECHA_NACIMIENTO	date	NO	NO	SI	Fecha de nacimiento; no puede ser futura
	SEXO	char(1)	NO	NO	NO	Sexo del paciente (M, F u O)
	TELEFONO	varchar(20)	NO	NO	SI	Número de contacto del paciente
	CORREO	varchar(100)	NO	NO	NO	Correo electrónico de contacto (opcional)
	DIRECCION	varchar(150)	NO	NO	NO	Domicilio del paciente (opcional)
	TIPO_SANGRE	varchar(3)	NO	NO	NO	Grupo sanguíneo y factor Rh
	ALERGIAS	text	NO	NO	NO	Alergias conocidas del paciente
	ENFERMEDAD_ CRONICA	text	NO	NO	NO	Enfermedades crónicas del paciente
	ANTECEDENTES	text	NO	NO	NO	Antecedentes médicos relevantes
	ID_USUARIO	int	NO	SI	NO	Cuenta del portal del paciente (opcional)
	ESTADO	varchar(10)	NO	NO	SI	Indica si el paciente está activo o inactivo
CONTACTO_ EMERGENCIA	ID_CONTACTO	serial	SI	NO	SI	Identificador único del contacto de emergencia
	ID_PACIENTE	int	NO	SI	SI	Paciente al que pertenece el contacto
	NOMBRE	varchar(100)	NO	NO	SI	Nombre completo del contacto
	PARENTESCO	varchar(30)	NO	NO	SI	Relación del contacto con el paciente
	TELEFONO	varchar(20)	NO	NO	SI	Teléfono del contacto
ESPECIALIDAD	ID_ESPECIALIDAD	serial	SI	NO	SI	Identificador único de la especialidad
	NOMBRE	varchar(60)	NO	NO	SI	Nombre de la especialidad; único sin distinguir mayúsculas
	DESCRIPCION	varchar(200)	NO	NO	NO	Descripción opcional de la especialidad
	ESTADO	varchar(10)	NO	NO	SI	Indica si la especialidad está activa o inactiva
	ID_MEDICO	serial	SI	NO	SI	Identificador único del médico

Clase	Atributo	Tipo de Dato	PK	FK	Not Null	Descripción
MEDICO	REGISTRO_ PROFESIONAL	varchar(20)	NO	NO	SI	Número de registro profesional; único
	NOMBRES	varchar(60)	NO	NO	SI	Nombres del médico
	APELLIDOS	varchar(60)	NO	NO	SI	Apellidos del médico
	TELEFONO	varchar(20)	NO	NO	NO	Teléfono del médico
	CORREO	varchar(100)	NO	NO	NO	Correo electrónico del médico
	ID_USUARIO	int	NO	SI	NO	Cuenta de usuario del médico (opcional)
	ESTADO	varchar(10)	NO	NO	SI	Un médico inactivo no recibe nuevas citas
MEDICO_ ESPECIALIDAD	ID_MEDICO	int	SI	SI	SI	Médico que ejerce la especialidad
	ID_ESPECIALIDAD	int	SI	SI	SI	Especialidad ejercida por el médico
HORARIO_ ATENCION	ID_HORARIO	serial	SI	NO	SI	Identificador único del bloque de horario
	ID_MEDICO	int	NO	SI	SI	Médico al que pertenece el horario
	DIA_SEMANA	smallint	NO	NO	SI	Día de la semana (1 = lunes … 7 = domingo)
	HORA_INICIO	time	NO	NO	SI	Hora de inicio de la atención
	HORA_FIN	time	NO	NO	SI	Hora de fin; posterior a la hora de inicio
	DURACION_CITA	smallint	NO	NO	SI	Duración de cada cita en minutos
	ESTADO	varchar(10)	NO	NO	SI	Indica si el bloque está activo o inactivo
BLOQUEO_ AGENDA	ID_BLOQUEO	serial	SI	NO	SI	Identificador único del bloqueo
	ID_MEDICO	int	NO	SI	NO	Médico bloqueado; vacío = toda la clínica (feriado)
	FECHA	date	NO	NO	SI	Fecha en la que no se atiende
	MOTIVO	varchar(100)	NO	NO	SI	Motivo del bloqueo (vacación, feriado)
CITA	ID_CITA	serial	SI	NO	SI	Identificador único de la cita (se muestra como CM-0001)
	ID_PACIENTE	int	NO	SI	SI	Paciente de la cita
	ID_MEDICO	int	NO	SI	SI	Médico que atiende la cita
	ID_ESPECIALIDAD	int	NO	SI	SI	Especialidad de la consulta
	FECHA	date	NO	NO	SI	Fecha de la cita; no puede ser pasada
	HORA_INICIO	time	NO	NO	SI	Hora de inicio de la cita
	HORA_FIN	time	NO	NO	SI	Hora de fin según la duración configurada
	MOTIVO	varchar(200)	NO	NO	NO	Motivo breve de la consulta (opcional)
	MODALIDAD	varchar(12)	NO	NO	SI	Presencial o virtual

Clase	Atributo	Tipo de Dato	PK	FK	Not Null	Descripción
	ESTADO	varchar(12)	NO	NO	SI	Programada, Confirmada, Atendida, Cancelada o Inasistencia
	MOTIVO_ CANCELACION	varchar(200)	NO	NO	NO	Motivo registrado al cancelar la cita
	ID_USUARIO_ REGISTRO	int	NO	SI	NO	Usuario que registró la cita
	FECHA_CREACION	timestamp	NO	NO	SI	Fecha y hora de registro de la cita
	FECHA_ ACTUALIZACION	timestamp	NO	NO	NO	Fecha y hora de la última modificación
HISTORIAL_ ESTADO_CITA	ID_HISTORIAL	serial	SI	NO	SI	Identificador único del registro de historial
	ID_CITA	int	NO	SI	SI	Cita a la que pertenece el cambio
	ESTADO_ANTERIOR	varchar(12)	NO	NO	NO	Estado antes del cambio
	ESTADO_NUEVO	varchar(12)	NO	NO	SI	Estado después del cambio
	FECHA_ANTERIOR	date	NO	NO	NO	Fecha de la cita antes de reprogramar
	HORA_ANTERIOR	time	NO	NO	NO	Hora de la cita antes de reprogramar
	OBSERVACION	varchar(200)	NO	NO	NO	Descripción del cambio
	ID_USUARIO	int	NO	SI	NO	Usuario que realizó el cambio
	FECHA	timestamp	NO	NO	SI	Fecha y hora del cambio

Diagrama relacional

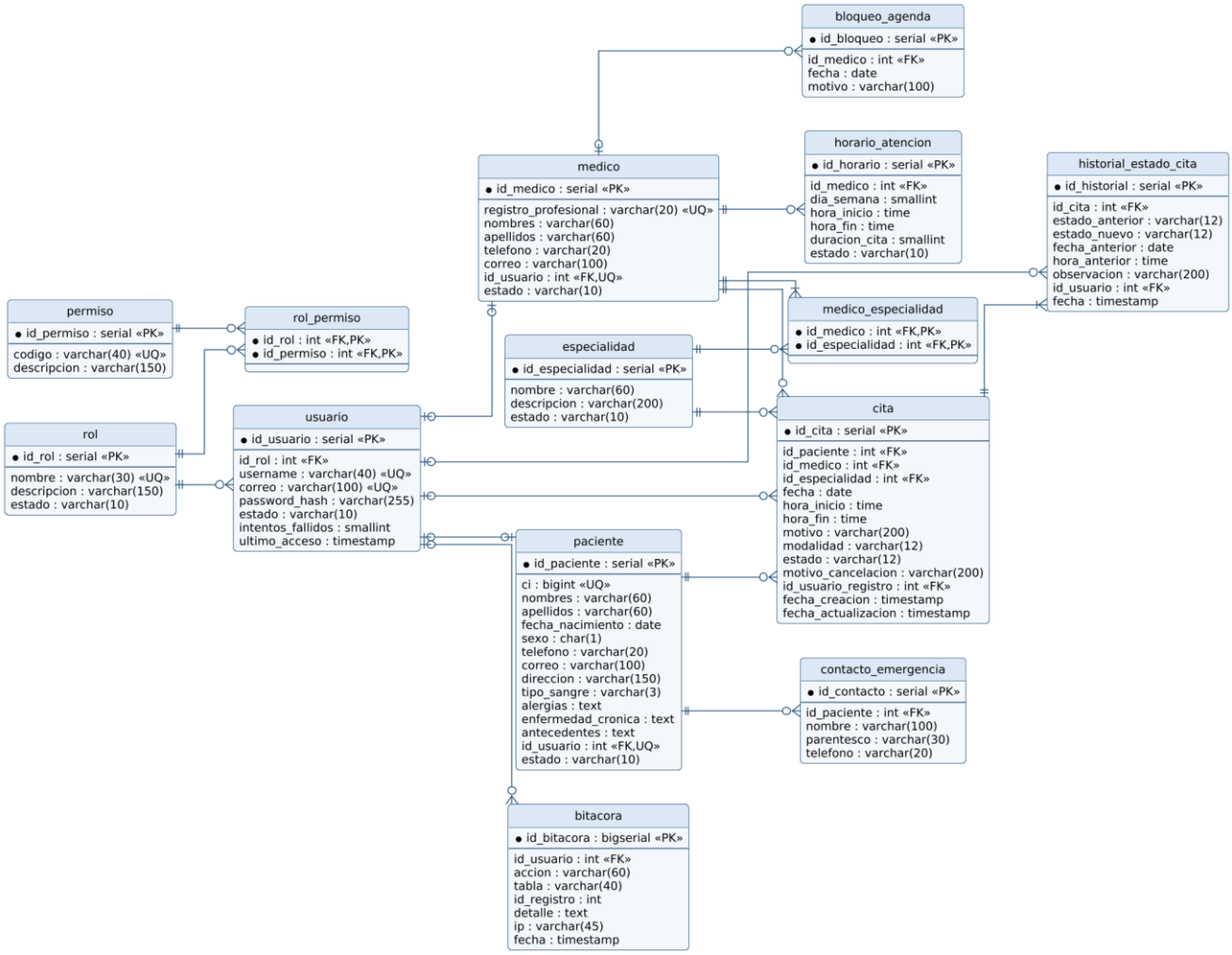


Figura 100. *Diagrama relacional de la base de datos clinica_vida_salud (página girada)*

Script de la base de datos

El siguiente script crea las tablas, restricciones, índices, funciones, disparadores y vistas. Fue ejecutado y verificado en PostgreSQL 16.

```
-- ======================================================================
-- Sistema Web de Gestión de Citas Médicas - Clínica Vida Salud
-- Script de creación de la base de datos (PostgreSQL)
-- ======================================================================
-- CREATE DATABASE clinica_vida_salud ENCODING 'UTF8';
-- \c clinica_vida_salud

CREATE EXTENSION IF NOT EXISTS btree_gist;   -- necesaria para la restricción de exclusión de citas

-- ----------------------------------------------------------------------
-- PAQUETE SEGURIDAD
-- ----------------------------------------------------------------------
CREATE TABLE rol (
    id_rol        SERIAL PRIMARY KEY,
    nombre        VARCHAR(30)  NOT NULL UNIQUE,
    descripcion   VARCHAR(150),
    estado        VARCHAR(10)  NOT NULL DEFAULT 'activo'
                  CHECK (estado IN ('activo','inactivo'))
);
```

```
CREATE TABLE permiso (
    id_permiso    SERIAL PRIMARY KEY,
    codigo        VARCHAR(40)  NOT NULL UNIQUE,      -- p. ej. 'cita.programar'
    descripcion   VARCHAR(150) NOT NULL
);

CREATE TABLE rol_permiso (
    id_rol        INT NOT NULL REFERENCES rol(id_rol),
    id_permiso    INT NOT NULL REFERENCES permiso(id_permiso),
    PRIMARY KEY (id_rol, id_permiso)
);

CREATE TABLE usuario (
    id_usuario     SERIAL PRIMARY KEY,
    id_rol         INT          NOT NULL REFERENCES rol(id_rol),
    username       VARCHAR(40)  NOT NULL UNIQUE,
    correo         VARCHAR(100) NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL,            -- generado con password_hash() de PHP
    estado         VARCHAR(10)  NOT NULL DEFAULT 'activo'
                   CHECK (estado IN ('activo','inactivo','bloqueado')),
    intentos_fallidos SMALLINT  NOT NULL DEFAULT 0,
    ultimo_acceso  TIMESTAMP
);

CREATE TABLE bitacora (
    id_bitacora   BIGSERIAL PRIMARY KEY,
    id_usuario    INT REFERENCES usuario(id_usuario),
    accion        VARCHAR(60)  NOT NULL,
    tabla         VARCHAR(40),
    id_registro   INT,
    detalle       TEXT,
    ip            VARCHAR(45),
    fecha         TIMESTAMP    NOT NULL DEFAULT now()
);

-- ---------------------------------------------------------------------
-- PAQUETE PERSONAS (pacientes, médicos y especialidades)
-- ---------------------------------------------------------------------
CREATE TABLE paciente (
    id_paciente      SERIAL PRIMARY KEY,
    ci               BIGINT       UNIQUE CHECK (ci > 0),  -- RN01: CI único cuando se proporciona
    nombres          VARCHAR(60)  NOT NULL,
    apellidos        VARCHAR(60)  NOT NULL,
    fecha_nacimiento DATE         NOT NULL CHECK (fecha_nacimiento <= CURRENT_DATE),
    sexo             CHAR(1)      CHECK (sexo IN ('M','F','O')),
    telefono         VARCHAR(20)  NOT NULL,
    correo           VARCHAR(100),
    direccion        VARCHAR(150),
    tipo_sangre      VARCHAR(3)   CHECK (tipo_sangre IN ('A+','A-','B+','B-','AB+','AB-','O+','O-')),
    alergias         TEXT,
    enfermedad_cronica TEXT,
    antecedentes     TEXT,
    id_usuario       INT UNIQUE REFERENCES usuario(id_usuario),   -- cuenta del portal (opcional)
    estado           VARCHAR(10)  NOT NULL DEFAULT 'activo'
                     CHECK (estado IN ('activo','inactivo'))
);

CREATE TABLE contacto_emergencia (
    id_contacto   SERIAL PRIMARY KEY,
    id_paciente   INT          NOT NULL REFERENCES paciente(id_paciente),
    nombre        VARCHAR(100) NOT NULL,
    parentesco    VARCHAR(30)  NOT NULL,
    telefono      VARCHAR(20)  NOT NULL
);

CREATE TABLE especialidad (
    id_especialidad SERIAL PRIMARY KEY,
    nombre          VARCHAR(60)  NOT NULL,
    descripcion     VARCHAR(200),
    estado          VARCHAR(10)  NOT NULL DEFAULT 'activo'
                    CHECK (estado IN ('activo','inactivo'))
);
-- RF04: no se permiten dos especialidades con el mismo nombre normalizado
CREATE UNIQUE INDEX ux_especialidad_nombre ON especialidad (lower(trim(nombre)));

CREATE TABLE medico (
    id_medico             SERIAL PRIMARY KEY,
    registro_profesional  VARCHAR(20)  NOT NULL UNIQUE,
    nombres               VARCHAR(60)  NOT NULL,
```

```
    apellidos             VARCHAR(60)  NOT NULL,
    telefono              VARCHAR(20),
    correo                VARCHAR(100),
    id_usuario            INT UNIQUE REFERENCES usuario(id_usuario),
    estado                VARCHAR(10)  NOT NULL DEFAULT 'activo'
                          CHECK (estado IN ('activo','inactivo'))
);

CREATE TABLE medico_especialidad (
    id_medico        INT NOT NULL REFERENCES medico(id_medico),
    id_especialidad  INT NOT NULL REFERENCES especialidad(id_especialidad),
    PRIMARY KEY (id_medico, id_especialidad)
);

-- ---------------------------------------------------------------------
-- PAQUETE AGENDA (horarios de atención)
-- ---------------------------------------------------------------------
CREATE TABLE horario_atencion (
    id_horario     SERIAL PRIMARY KEY,
    id_medico      INT      NOT NULL REFERENCES medico(id_medico),
    dia_semana     SMALLINT NOT NULL CHECK (dia_semana BETWEEN 1 AND 7),  -- 1 = lunes (ISODOW)
    hora_inicio    TIME     NOT NULL,
    hora_fin       TIME     NOT NULL,
    duracion_cita  SMALLINT NOT NULL DEFAULT 30 CHECK (duracion_cita IN (15,20,30,45,60)),
    estado         VARCHAR(10) NOT NULL DEFAULT 'activo'
                   CHECK (estado IN ('activo','inactivo')),
    CONSTRAINT ck_horario_rango CHECK (hora_fin > hora_inicio),
    -- un médico no puede tener dos bloques de horario superpuestos el mismo día
    CONSTRAINT ex_horario_superpuesto EXCLUDE USING gist (
        id_medico WITH =, dia_semana WITH =,
        tsrange(DATE '2000-01-01' + hora_inicio, DATE '2000-01-01' + hora_fin) WITH &&
    ) WHERE (estado = 'activo')
);

CREATE TABLE bloqueo_agenda (
    id_bloqueo   SERIAL PRIMARY KEY,
    id_medico    INT REFERENCES medico(id_medico),   -- NULL = feriado para toda la clínica
    fecha        DATE NOT NULL,
    motivo       VARCHAR(100) NOT NULL
);

-- ---------------------------------------------------------------------
-- PAQUETE CITAS
-- ---------------------------------------------------------------------
CREATE TABLE cita (
    id_cita            SERIAL PRIMARY KEY,
    id_paciente        INT  NOT NULL REFERENCES paciente(id_paciente),
    id_medico          INT  NOT NULL REFERENCES medico(id_medico),
    id_especialidad    INT  NOT NULL REFERENCES especialidad(id_especialidad),
    fecha              DATE NOT NULL,
    hora_inicio        TIME NOT NULL,
    hora_fin           TIME NOT NULL,
    motivo             VARCHAR(200),
    modalidad          VARCHAR(12) NOT NULL DEFAULT 'presencial'
                       CHECK (modalidad IN ('presencial','virtual')),
    estado             VARCHAR(12) NOT NULL DEFAULT 'Programada'
                       CHECK (estado IN ('Programada','Confirmada','Atendida','Cancelada','Inasistencia')),
    motivo_cancelacion VARCHAR(200),
    id_usuario_registro INT REFERENCES usuario(id_usuario),   -- usuario que registra la cita
    fecha_creacion     TIMESTAMP NOT NULL DEFAULT now(),
    fecha_actualizacion TIMESTAMP,
    CONSTRAINT ck_cita_rango CHECK (hora_fin > hora_inicio),
    CONSTRAINT ck_cancelacion CHECK (estado <> 'Cancelada' OR motivo_cancelacion IS NOT NULL),
    -- RF07 / RN07: un médico no puede tener citas superpuestas.
    -- Las citas canceladas no ocupan horario (RN09), por eso se excluyen.
    CONSTRAINT ex_cita_superpuesta EXCLUDE USING gist (
        id_medico WITH =,
        tsrange(fecha + hora_inicio, fecha + hora_fin) WITH &&
    ) WHERE (estado <> 'Cancelada')
);

CREATE TABLE historial_estado_cita (
    id_historial     SERIAL PRIMARY KEY,
    id_cita          INT NOT NULL REFERENCES cita(id_cita),
    estado_anterior  VARCHAR(12),
    estado_nuevo     VARCHAR(12) NOT NULL,
    fecha_anterior   DATE,
    hora_anterior    TIME,
    observacion      VARCHAR(200),
    id_usuario       INT REFERENCES usuario(id_usuario),
```

```
    fecha            TIMESTAMP NOT NULL DEFAULT now()
);

-- Índices de apoyo a las consultas frecuentes
CREATE INDEX ix_cita_medico_fecha   ON cita (id_medico, fecha);
CREATE INDEX ix_cita_paciente       ON cita (id_paciente);
CREATE INDEX ix_cita_estado_fecha   ON cita (estado, fecha);
CREATE INDEX ix_paciente_nombre     ON paciente (lower(apellidos), lower(nombres));
CREATE INDEX ix_horario_medico_dia  ON horario_atencion (id_medico, dia_semana);

-- ---------------------------------------------------------------------
-- FUNCIONES Y TRIGGERS (reglas de negocio protegidas en la BD)
-- ---------------------------------------------------------------------

-- Valida RN04, RN05, RN06 y RN08 antes de insertar o reprogramar una cita
CREATE OR REPLACE FUNCTION fn_validar_cita() RETURNS trigger AS $$
DECLARE
    v_estado_medico VARCHAR(10);
    v_estado_pac    VARCHAR(10);
    v_ok            BOOLEAN;
BEGIN
    -- Solo se valida al crear o al cambiar fecha/hora/médico
    IF TG_OP = 'UPDATE' AND NEW.fecha = OLD.fecha AND NEW.hora_inicio = OLD.hora_inicio
       AND NEW.id_medico = OLD.id_medico THEN
        RETURN NEW;
    END IF;

    SELECT estado INTO v_estado_medico FROM medico WHERE id_medico = NEW.id_medico;
    IF v_estado_medico <> 'activo' THEN
        RAISE EXCEPTION 'El médico está inactivo y no puede recibir nuevas citas';
    END IF;

    SELECT estado INTO v_estado_pac FROM paciente WHERE id_paciente = NEW.id_paciente;
    IF v_estado_pac <> 'activo' THEN
        RAISE EXCEPTION 'El paciente está inactivo';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM medico_especialidad
                   WHERE id_medico = NEW.id_medico AND id_especialidad = NEW.id_especialidad) THEN
        RAISE EXCEPTION 'El médico no atiende la especialidad seleccionada';
    END IF;

    IF (NEW.fecha + NEW.hora_inicio) < now() THEN
        RAISE EXCEPTION 'No se pueden registrar citas en fechas u horas pasadas';
    END IF;

    IF EXISTS (SELECT 1 FROM bloqueo_agenda
               WHERE fecha = NEW.fecha AND (id_medico IS NULL OR id_medico = NEW.id_medico)) THEN
        RAISE EXCEPTION 'La fecha seleccionada está bloqueada en la agenda';
    END IF;

    -- La cita debe caer dentro de un horario de atención y respetar el intervalo configurado
    SELECT EXISTS (
        SELECT 1 FROM horario_atencion h
        WHERE h.id_medico  = NEW.id_medico
          AND h.estado     = 'activo'
          AND h.dia_semana = EXTRACT(ISODOW FROM NEW.fecha)
          AND NEW.hora_inicio >= h.hora_inicio
          AND NEW.hora_fin    <= h.hora_fin
          AND NEW.hora_fin - NEW.hora_inicio = make_interval(mins => h.duracion_cita)
          AND (EXTRACT(EPOCH FROM (NEW.hora_inicio - h.hora_inicio)) / 60)::int % h.duracion_cita = 0
    ) INTO v_ok;

    IF NOT v_ok THEN
        RAISE EXCEPTION 'La cita está fuera del horario de atención del médico o no respeta el intervalo configurado';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_validar_cita
BEFORE INSERT OR UPDATE OF fecha, hora_inicio, hora_fin, id_medico ON cita
FOR EACH ROW EXECUTE FUNCTION fn_validar_cita();

-- Controla las transiciones de estado válidas (RF10)
CREATE OR REPLACE FUNCTION fn_transicion_estado() RETURNS trigger AS $$
```

```
BEGIN
    IF NEW.estado = OLD.estado THEN
        -- Reprogramación: solo citas Programadas o Confirmadas
        IF (NEW.fecha <> OLD.fecha OR NEW.hora_inicio <> OLD.hora_inicio)
           AND OLD.estado NOT IN ('Programada','Confirmada') THEN
            RAISE EXCEPTION 'Una cita % no puede reprogramarse', OLD.estado;
        END IF;
        RETURN NEW;
    END IF;

    IF NOT (
        (OLD.estado = 'Programada' AND NEW.estado IN ('Confirmada','Cancelada','Atendida','Inasistencia')) OR
        (OLD.estado = 'Confirmada' AND NEW.estado IN ('Atendida','Cancelada','Inasistencia'))
    ) THEN
        RAISE EXCEPTION 'Transición de estado no permitida: % -> %', OLD.estado, NEW.estado;
    END IF;

    IF NEW.estado IN ('Atendida','Inasistencia') AND (NEW.fecha + NEW.hora_inicio) > now() THEN
        RAISE EXCEPTION 'No se puede registrar asistencia de una cita futura';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_transicion_estado
BEFORE UPDATE ON cita
FOR EACH ROW EXECUTE FUNCTION fn_transicion_estado();

-- Registra la trazabilidad de cada cambio de estado o reprogramación (RF12)
CREATE OR REPLACE FUNCTION fn_historial_cita() RETURNS trigger AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO historial_estado_cita (id_cita, estado_anterior, estado_nuevo, observacion, id_usuario)
        VALUES (NEW.id_cita, NULL, NEW.estado, 'Registro de la cita', NEW.id_usuario_registro);
    ELSIF NEW.estado <> OLD.estado OR NEW.fecha <> OLD.fecha OR NEW.hora_inicio <> OLD.hora_inicio THEN
        INSERT INTO historial_estado_cita (id_cita, estado_anterior, estado_nuevo, fecha_anterior,
                                           hora_anterior, observacion, id_usuario)
        VALUES (NEW.id_cita, OLD.estado, NEW.estado, OLD.fecha, OLD.hora_inicio,
                CASE WHEN NEW.estado <> OLD.estado THEN 'Cambio de estado' ELSE 'Reprogramación' END,
                NULLIF(current_setting('app.id_usuario', true), '')::int);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_historial_cita
AFTER INSERT OR UPDATE ON cita
FOR EACH ROW EXECUTE FUNCTION fn_historial_cita();

-- RN10 / RN11: no se eliminan físicamente citas, pacientes ni médicos con historial
CREATE OR REPLACE FUNCTION fn_impedir_borrado() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'No se permite eliminar registros de %; utilice la desactivación', TG_TABLE_NAME;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tg_no_borrar_cita     BEFORE DELETE ON cita     FOR EACH ROW EXECUTE FUNCTION fn_impedir_borrado();
CREATE TRIGGER tg_no_borrar_paciente BEFORE DELETE ON paciente FOR EACH ROW EXECUTE FUNCTION fn_impedir_borrado();
CREATE TRIGGER tg_no_borrar_medico   BEFORE DELETE ON medico   FOR EACH ROW EXECUTE FUNCTION fn_impedir_borrado();

-- ---------------------------------------------------------------------
-- FUNCIÓN: horarios disponibles de un médico en una fecha (RF05, RF06)
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_horarios_disponibles(p_id_medico INT, p_fecha DATE)
RETURNS TABLE (hora_inicio TIME, hora_fin TIME) AS $$
    SELECT s.ini::time, (s.ini + make_interval(mins => h.duracion_cita))::time
    FROM horario_atencion h
    CROSS JOIN LATERAL generate_series(
        p_fecha + h.hora_inicio,
        p_fecha + h.hora_fin - make_interval(mins => h.duracion_cita),
        make_interval(mins => h.duracion_cita)) AS s(ini)
    JOIN medico m ON m.id_medico = h.id_medico AND m.estado = 'activo'
    WHERE h.id_medico = p_id_medico
      AND h.estado = 'activo'
      AND h.dia_semana = EXTRACT(ISODOW FROM p_fecha)
      AND s.ini > now()
      AND NOT EXISTS (SELECT 1 FROM bloqueo_agenda b
                      WHERE b.fecha = p_fecha AND (b.id_medico IS NULL OR b.id_medico = p_id_medico))
      AND NOT EXISTS (SELECT 1 FROM cita c
```

```
                    WHERE c.id_medico = p_id_medico AND c.fecha = p_fecha
                      AND c.estado <> 'Cancelada'
                      AND tsrange(p_fecha + c.hora_inicio, p_fecha + c.hora_fin)
                          && tsrange(s.ini, s.ini + make_interval(mins => h.duracion_cita)))
    ORDER BY 1;
$$ LANGUAGE sql STABLE;

-- ---------------------------------------------------------------------
-- VISTAS PARA REPORTES (RF11, RF12, RF13)
-- ---------------------------------------------------------------------
CREATE OR REPLACE VIEW v_agenda_medica AS
SELECT c.id_cita, 'CM-' || lpad(c.id_cita::text, 4, '0') AS codigo, c.fecha, c.hora_inicio, c.hora_fin, c.estado, c.motivo,
       m.id_medico, m.nombres || ' ' || m.apellidos AS medico,
       p.id_paciente, p.nombres || ' ' || p.apellidos AS paciente, p.ci,
       e.id_especialidad, e.nombre AS especialidad
FROM cita c
JOIN medico m       ON m.id_medico = c.id_medico
JOIN paciente p     ON p.id_paciente = c.id_paciente
JOIN especialidad e ON e.id_especialidad = c.id_especialidad;

CREATE OR REPLACE VIEW v_resumen_estados AS
SELECT fecha, id_medico, id_especialidad,
       count(*)                                          AS total,
       count(*) FILTER (WHERE estado = 'Atendida')      AS atendidas,
       count(*) FILTER (WHERE estado = 'Cancelada')     AS canceladas,
       count(*) FILTER (WHERE estado = 'Inasistencia')  AS inasistencias,
       count(*) FILTER (WHERE estado IN ('Programada','Confirmada')) AS pendientes
FROM cita
GROUP BY fecha, id_medico, id_especialidad;
```

Actualización de datos

```
-- ======================================================================
-- Población de datos de prueba (datos ficticios)
-- Contraseña de todos los usuarios de prueba: Clinica2026*
-- ======================================================================
INSERT INTO rol (nombre, descripcion) VALUES
 ('Administrador','Configura y supervisa el sistema'),
 ('Recepcionista','Gestiona pacientes y citas'),
 ('Medico','Consulta su agenda y registra atenciones'),
 ('Paciente','Gestiona su perfil y sus citas desde el portal');

INSERT INTO permiso (codigo, descripcion) VALUES
 ('usuario.gestionar','Registrar y administrar usuarios'),
 ('rol.gestionar','Asignar roles y permisos'),
 ('paciente.gestionar','Registrar, modificar y desactivar pacientes'),
 ('medico.gestionar','Registrar, modificar y desactivar médicos'),
 ('especialidad.gestionar','Administrar especialidades'),
 ('horario.gestionar','Registrar horarios de atención'),
 ('cita.programar','Programar, reprogramar y cancelar citas'),
 ('cita.asistencia','Registrar asistencia o inasistencia'),
 ('cita.atender','Marcar una cita como atendida'),
 ('agenda.consultar','Consultar la agenda médica'),
 ('reporte.generar','Generar reportes administrativos'),
 ('perfil.paciente','Gestionar el perfil propio y reservar citas');

INSERT INTO rol_permiso (id_rol, id_permiso)
SELECT 1, id_permiso FROM permiso WHERE codigo NOT IN ('perfil.paciente','cita.atender');
INSERT INTO rol_permiso (id_rol, id_permiso)
SELECT 2, id_permiso FROM permiso WHERE codigo IN
('paciente.gestionar','cita.programar','cita.asistencia','agenda.consultar','reporte.generar');
INSERT INTO rol_permiso (id_rol, id_permiso)
SELECT 3, id_permiso FROM permiso WHERE codigo IN ('agenda.consultar','cita.atender');
INSERT INTO rol_permiso (id_rol, id_permiso)
SELECT 4, id_permiso FROM permiso WHERE codigo = 'perfil.paciente';

INSERT INTO usuario (id_rol, username, correo, password_hash) VALUES
 (1,'admin','admin@vidasalud.bo','$2y$10$ecgkFu2CBDODAsgbTDbipeZQjLhxl.J6Nf/ft3T/V6E3tCObonJsO'),
 (2,'recepcion1','recepcion@vidasalud.bo','$2y$10$ecgkFu2CBDODAsgbTDbipeZQjLhxl.J6Nf/ft3T/V6E3tCObonJsO'),
 (3,'jperez','jperez@vidasalud.bo','$2y$10$ecgkFu2CBDODAsgbTDbipeZQjLhxl.J6Nf/ft3T/V6E3tCObonJsO'),
 (3,'arojas','arojas@vidasalud.bo','$2y$10$ecgkFu2CBDODAsgbTDbipeZQjLhxl.J6Nf/ft3T/V6E3tCObonJsO'),
 (4,'mlopez','maria.lopez@correo.com','$2y$10$ecgkFu2CBDODAsgbTDbipeZQjLhxl.J6Nf/ft3T/V6E3tCObonJsO');

INSERT INTO especialidad (nombre, descripcion) VALUES
 ('Medicina general','Atención primaria y consulta general'),
 ('Pediatría','Atención de niños y adolescentes'),
 ('Ginecología','Salud de la mujer'),
 ('Odontología','Salud bucal'),
```

```
 ('Cardiología','Enfermedades del corazón');

INSERT INTO medico (registro_profesional, nombres, apellidos, telefono, correo, id_usuario) VALUES
 ('MP-12345','Juan','Pérez Gómez','70011122','jperez@vidasalud.bo',3),
 ('MP-23456','Ana','Rojas Suárez','70022233','arojas@vidasalud.bo',4),
 ('MP-34567','Carlos','Méndez Vaca','70033344','cmendez@vidasalud.bo',NULL);

INSERT INTO medico_especialidad VALUES (1,1),(2,2),(2,1),(3,4);

-- Horarios: Médico A lunes 08-12 y miércoles 14-18; Médico B martes 09-13 (ejemplo del enunciado)
INSERT INTO horario_atencion (id_medico, dia_semana, hora_inicio, hora_fin, duracion_cita) VALUES
 (1,1,'08:00','12:00',30),(1,3,'14:00','18:00',30),
 (2,2,'09:00','13:00',30),(2,4,'09:00','13:00',30),
 (3,5,'08:00','12:00',30);

INSERT INTO paciente (ci, nombres, apellidos, fecha_nacimiento, sexo, telefono, correo, direccion, tipo_sangre, alergias,
id_usuario) VALUES
 (7845123,'María','López Arteaga','1990-04-12','F','76543210','maria.lopez@correo.com','Av. Banzer 4to
anillo','O+','Penicilina',5),
 (6523147,'Pedro','Gutiérrez Flores','1985-09-30','M','71234567',NULL,'B/ Equipetrol','A+',NULL,NULL),
 (8541236,'Lucía','Vargas Ribera','2018-02-05','F','72345678',NULL,NULL,'O-',NULL,NULL);

INSERT INTO contacto_emergencia (id_paciente, nombre, parentesco, telefono) VALUES
 (1,'Rosa Arteaga','Madre','77711122'),
 (3,'Jorge Vargas','Padre','77722233');

-- Citas históricas (se desactiva temporalmente la validación de fecha pasada solo para la carga inicial)
ALTER TABLE cita DISABLE TRIGGER tg_validar_cita;
ALTER TABLE cita DISABLE TRIGGER tg_transicion_estado;
INSERT INTO cita (id_paciente, id_medico, id_especialidad, fecha, hora_inicio, hora_fin, motivo, estado,
motivo_cancelacion, id_usuario_registro) VALUES
 (1,1,1,'2026-09-07','09:00','09:30','Control general','Atendida',NULL,2),
 (2,1,1,'2026-09-07','09:30','10:00','Dolor de cabeza','Inasistencia',NULL,2),
 (3,2,2,'2026-09-08','10:00','10:30','Control pediátrico','Atendida',NULL,2),
 (1,2,1,'2026-09-15','09:00','09:30','Chequeo','Cancelada','El paciente viajó',2);
ALTER TABLE cita ENABLE TRIGGER tg_validar_cita;
ALTER TABLE cita ENABLE TRIGGER tg_transicion_estado;
```

Restricciones, funciones y disparadores

Tabla 52. *Mecanismos de integridad en la base de datos*

Objeto	Tipo	Propósito
ex_cita_superpuesta	Restricción EXCLUDE (gist)	Impide que un médico tenga dos citas no canceladas con intervalos superpuestos (RF07, RN07, RN09), incluso con usuarios concurrentes.
ex_horario_superpuesto	Restricción EXCLUDE (gist)	Impide bloques de horario superpuestos para el mismo médico y día.
ck_horario_rango / ck_cita_rango	CHECK	La hora final debe ser posterior a la hora inicial.
ux_especialidad_nombre	Índice único sobre lower(trim(nombre))	Evita especialidades con el mismo nombre normalizado (RF04).
fn_validar_cita / tg_validar_cita	Función PL/pgSQL + trigger BEFORE	Valida médico y paciente activos, especialidad del médico, fecha no pasada, fecha no bloqueada, horario de atención e intervalo (RN04–RN06, RN08).
fn_transicion_estado	Función + trigger BEFORE UPDATE	Permite solo las transiciones válidas de estado y prohíbe reprogramar citas atendidas o canceladas (RF08, RF10).
fn_historial_cita	Función + trigger AFTER	Registra en historial_estado_cita cada creación, cambio de estado y reprogramación (RF12).
fn_impedir_borrado	Función + triggers BEFORE DELETE	Impide eliminar físicamente citas, pacientes y médicos (RN10, RN11).

Objeto	Tipo	Propósito
fn_horarios_disponibles	Función SQL	Calcula los intervalos libres de un médico en una fecha (RF05, RF06).
v_agenda_medica / v_resumen_estados	Vistas	Simplifican la agenda, el historial y los reportes (RF11–RF13).

Explicación de los disparadores (triggers)

Un disparador es una función que PostgreSQL ejecuta automáticamente cuando ocurre un INSERT, UPDATE o DELETE sobre una tabla. En el sistema se usan para que las reglas críticas del negocio se cumplan aunque la operación llegue por una vía distinta de la aplicación:

tg_validar_cita (BEFORE INSERT OR UPDATE ON cita): antes de guardar una cita llama a fn_validar_cita, que comprueba que el paciente y el médico estén activos, que el médico tenga la especialidad elegida, que la fecha no sea pasada, que la hora caiga dentro de un bloque de horario activo y que la fecha no esté bloqueada en bloqueo_agenda. Si algo falla, la operación se cancela con un mensaje de error.

tg_transicion_estado (BEFORE UPDATE ON cita): solo permite los cambios de estado válidos (Programada → Confirmada, Cancelada, Atendida o Inasistencia; Confirmada → Atendida, Cancelada o Inasistencia), impide marcar como Atendida o Inasistencia una cita que todavía no ocurrió y no deja reprogramar citas que ya no están Programadas o Confirmadas.

tg_historial_cita (AFTER INSERT OR UPDATE ON cita): después de crear una cita, cambiar su estado o reprogramarla, inserta una fila en historial_estado_cita con el estado anterior, el nuevo, la fecha y hora anteriores y el usuario que hizo el cambio (tomado de app.id_usuario, que la aplicación fija con set_config dentro de la transacción). Este historial es el que se consulta en CU14.

tg_no_borrar_cita, tg_no_borrar_paciente, tg_no_borrar_medico (BEFORE DELETE): ejecutan fn_impedir_borrado y rechazan cualquier eliminación física; los registros solo se desactivan o se cancelan, para conservar la trazabilidad.

CAPÍTULO 5: FLUJO DE TRABAJO: IMPLEMENTACIÓN

En la implementación el diseño se transforma en componentes ejecutables. El sistema es una aplicación web de una sola página (SPA): el frontend, escrito en JavaScript con módulos nativos, se ejecuta en el navegador y se comunica mediante peticiones HTTP con intercambio de JSON con una API REST escrita en PHP, que a su vez accede a la base de datos PostgreSQL mediante PDO. Tanto el frontend como el backend se organizan en los mismos paquetes del análisis (seguridad, personas, agenda, citas, reportes y portal del paciente), de modo que cada caso de uso tiene su pantalla y su clase de caso de uso con el mismo código (CU01 … CU17).

5.1 Elección de la plataforma de desarrollo del software

Tabla 53. *Plataforma de desarrollo*

Elemento	Elección	Justificación
Lenguaje del servidor	PHP 8.3 con PDO (pdo_pgsql)	Disponible en XAMPP y en hosting gratuito; PDO con consultas preparadas evita la inyección SQL.
Lenguaje del cliente	JavaScript (módulos ES), HTML5 y CSS3	No requiere compilación ni frameworks; cada caso de uso es un módulo independiente.
Base de datos	PostgreSQL 17 (local) y Supabase (nube)	Soporta restricciones EXCLUDE (btree_gist), funciones PL/pgSQL, triggers y vistas, necesarios para las reglas del negocio.
Sistema operativo	Windows 10/11 (desarrollo) y Linux Debian (producción en contenedor)	El equipo desarrolla con XAMPP en Windows; el servidor en la nube corre en un contenedor Docker con Linux.
Servidor web	Apache 2.4	Incluido en XAMPP y en la imagen php:8.3-apache; sirve el frontend y redirige /api/v1 al backend.
Despliegue	Render (aplicación) + Supabase (base de datos)	Planes gratuitos; cada push a la rama main vuelve a publicar la aplicación.
Otros	Git y GitHub, Docker, Brevo (SMTP), pgAdmin 4, Visual Studio Code, PlantUML	Control de versiones, contenedor, envío de correos de contraseña, administración de la BD, edición y diagramas.

5.2 Implementación de la arquitectura del sistema

La Figura 101 muestra el diagrama de componentes principal. En el navegador, index.html carga rutas.js, que según la dirección (#/ruta) abre la pantalla del caso de uso; cada pantalla usa los componentes comunes (comunes.js, iconos.js, modal.js) y llama a la API únicamente a través de api.js. En el servidor, todas las peticiones entran por public/index.php; el Enrutador busca la ruta en el router.php del paquete y delega en la clase del caso de uso, que verifica la sesión y el permiso (Sesion::requerir), valida los datos (schemas.php), aplica las reglas (politicas.php) y accede a los datos (repository.php) mediante BaseDatos.php. La base de datos completa la validación con restricciones y triggers.

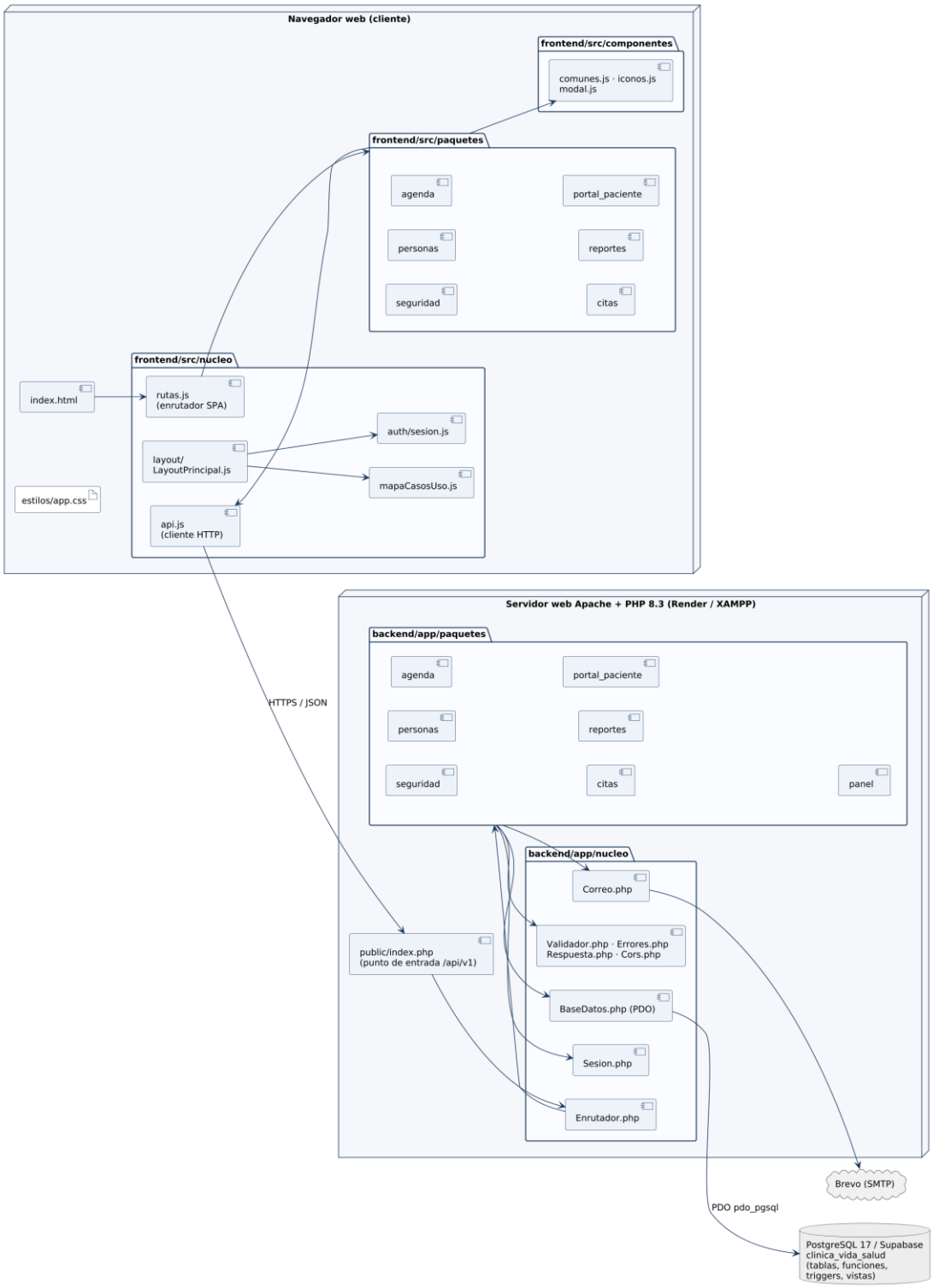


Figura 101. *Diagrama de componentes principal del sistema*

Estructura de carpetas del proyecto:

```
clinica-vida-salud/
├── frontend/                  # cliente (navegador)
│   ├── index.html
│   └── src/
│       ├── nucleo/            # rutas.js, api.js, config.js, mapaCasosUso.js, layout/, auth/
```

```
│       ├── componentes/       # comunes.js, iconos.js, modal.js
│       ├── estilos/app.css
│       └── paquetes/<paquete>/  # casos_uso/CUxx*.js, componentes/, api.js, rutas.js, schemas.js
├── backend/                    # API REST en PHP
│   ├── public/index.php        # punto de entrada /api/v1
│   └── app/
│       ├── nucleo/             # Enrutador, BaseDatos, Sesion, Correo, Validador, Errores, Respuesta
│       └── paquetes/<paquete>/  # router.php, casos_uso/cuxx_*.php, schemas.php, politicas.php, repository.php,
models.php
├── basededato/sql/           # 01_esquema … 07_datos_demo, pruebas/10_pruebas.sql
├── despliegue/               # apache.conf, php.ini, iniciar.sh
└── Dockerfile
```

Código fuente principal

Conexión a PostgreSQL con PDO (backend/app/nucleo/BaseDatos.php):

```
public static function conexion(): PDO {
    if (self::$pdo === null) {
        $dsn = sprintf('pgsql:host=%s;port=%s;dbname=%s', Config::obtener('DB_HOST'),
                       Config::obtener('DB_PUERTO'), Config::obtener('DB_NOMBRE'));
        $ssl = Config::obtener('DB_SSLMODE', '');      // Supabase exige sslmode=require
        if ($ssl !== '') { $dsn .= ";sslmode={$ssl}"; }
        self::$pdo = new PDO($dsn, Config::obtener('DB_USUARIO'), Config::obtener('DB_CLAVE'), [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false,           // consultas preparadas reales
        ]);
        self::$pdo->exec("SET TIME ZONE 'America/La_Paz'");
    }
    return self::$pdo;
}
```

Rutas del paquete citas (backend/app/paquetes/citas/router.php):

```
return static function (Enrutador $r): void {
    $r->post('/citas', [Cu09ProgramarCita::class, 'registrar']);
    $r->patch('/citas/{id}/reprogramar', [Cu10ReprogramarCita::class, 'reprogramar']);
    $r->patch('/citas/{id}/cancelar', [Cu11CancelarCita::class, 'cancelar']);
    $r->patch('/citas/{id}/estado', [Cu12RegistrarAsistencia::class, 'cambiarEstado']);
    $r->get('/citas/agenda', [Cu13ConsultarAgendaMedica::class, 'consultar']);
    $r->get('/citas/pacientes/{id}/historial', [Cu14ConsultarHistorialPaciente::class, 'historial']);
};
```

Registro de una cita en una transacción (CU09, cu09_programar_cita.php):

```
try {
    $idCita = BaseDatos::transaccion(function () use (...): int {
        // el trigger de historial registra qué usuario hizo el cambio
        BaseDatos::ejecutar('SELECT set_config(\'app.id_usuario\', :id, true)', [':id' => (string) $s['id_usuario']]);
        return citaInsertar($idPaciente, $idMedico, $idEspecialidad, $fecha, $horaInicio, $horaFin, $modalidad,
$motivo, $s['id_usuario']);
    });
    registrarBitacora($s['id_usuario'], 'CITA_CREAR', $idCita, ..., 'cita', $ip);
    return ['mensaje' => 'Cita programada correctamente.', 'id_cita' => $idCita, 'comprobante' => [...]];
} catch (\PDOException $e) {
    if ($e->getCode() === '23P01') {   // ex_cita_superpuesta: la ficha ya fue ocupada
        throw new ErrorApi(409, 'El horario ya no está disponible. Otro usuario ocupó este espacio.');
    }
    throw $e;
}
```

Cliente HTTP del frontend (frontend/src/nucleo/api.js):

```
async function solicitar(metodo, ruta, cuerpo) {
  const resp = await fetch(API_URL + ruta, {
    method: metodo, credentials: 'include',            // envía la cookie de sesión
    headers: cuerpo !== undefined ? { 'Content-Type': 'application/json' } : {},
    body: cuerpo !== undefined ? JSON.stringify(cuerpo) : undefined,
  })
  const datos = await resp.json().catch(() => ({}))
  if (!resp.ok) throw new ErrorApi(resp.status, datos.error?.mensaje || 'Error de comunicación con el servidor.')
  return datos
}
```

5.3 Implementación de la arquitectura de los subsistemas

Cada paquete de diseño se implementa como un subsistema con la misma estructura en el frontend (pantallas de los casos de uso, api.js, rutas.js y schemas.js) y en el backend (router.php, clases de los casos de uso,

schemas.php, politicas.php, repository.php y models.php). Las dependencias entre subsistemas son las mismas del análisis de paquetes: por ejemplo, citas usa el componente SelectorFichas y la disponibilidad de agenda, y el portal del paciente reutiliza agenda y citas. Las Figuras 102 a 108 muestran el diagrama de componentes de cada paquete.

Paquete Seguridad

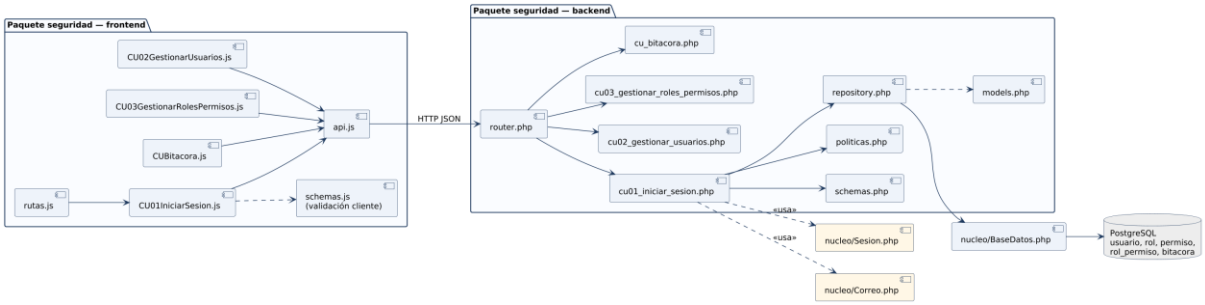


Figura 102. *Diagrama de componentes del paquete Seguridad*

Paquete Personas

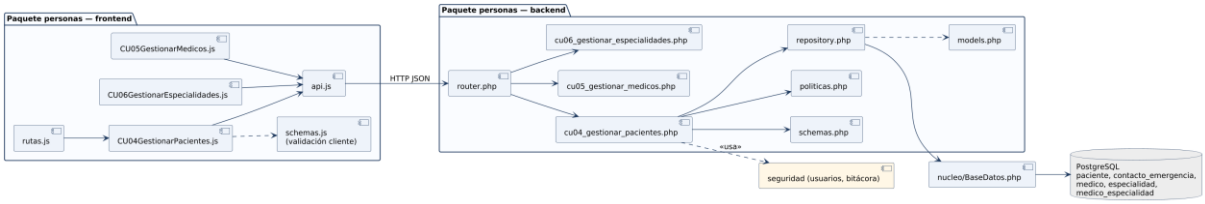


Figura 103. *Diagrama de componentes del paquete Personas*

Paquete Agenda

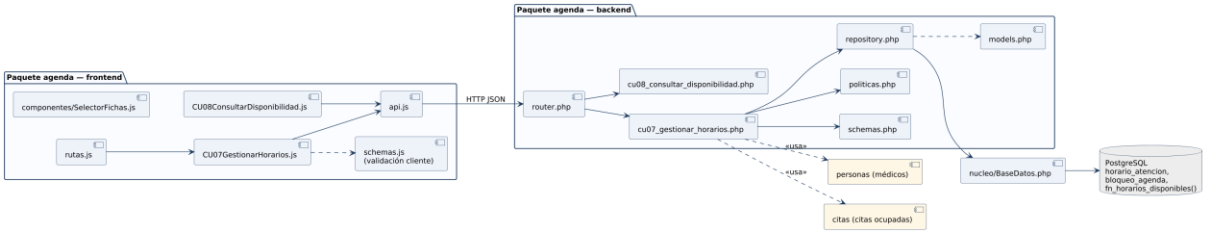


Figura 104. *Diagrama de componentes del paquete Agenda*

Paquete Citas

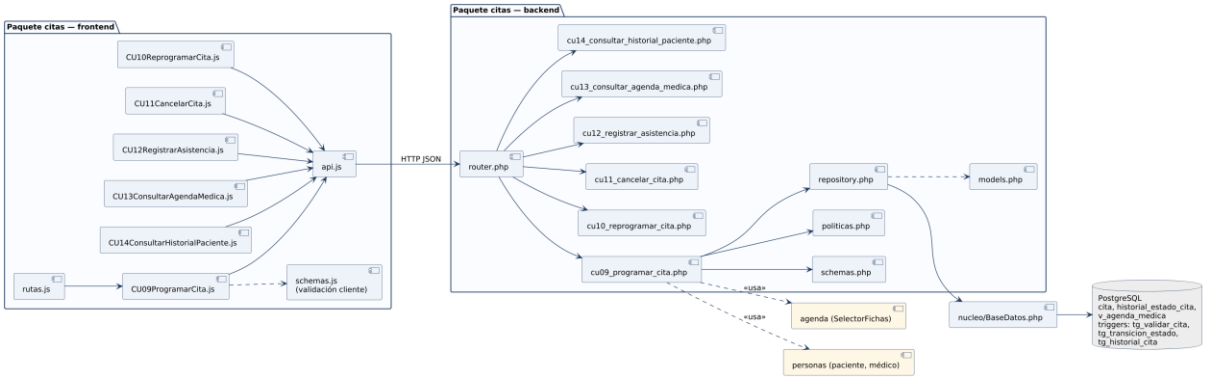


Figura 105. *Diagrama de componentes del paquete Citas*

Paquete Reportes

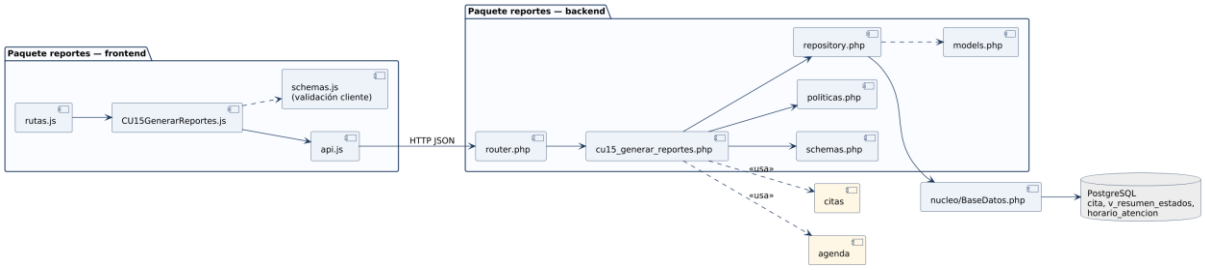


Figura 106. *Diagrama de componentes del paquete Reportes*

Paquete Portal del paciente

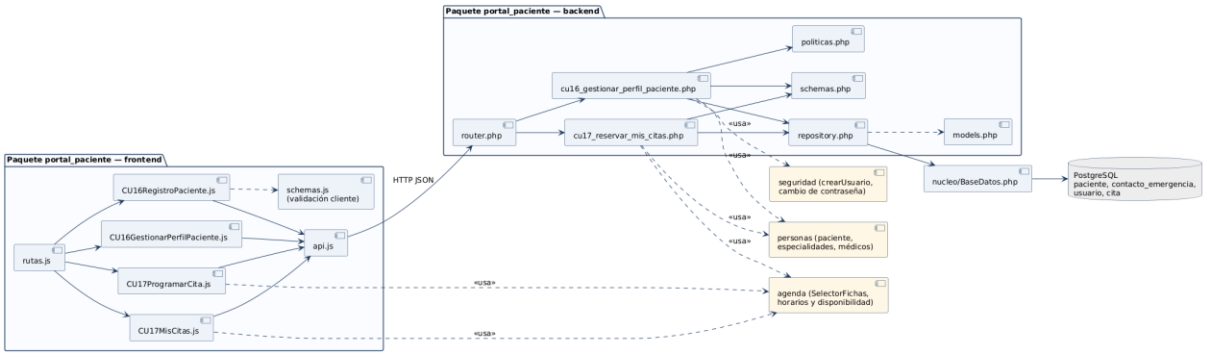


Figura 107. *Diagrama de componentes del paquete Portal del paciente*

Paquete Panel principal

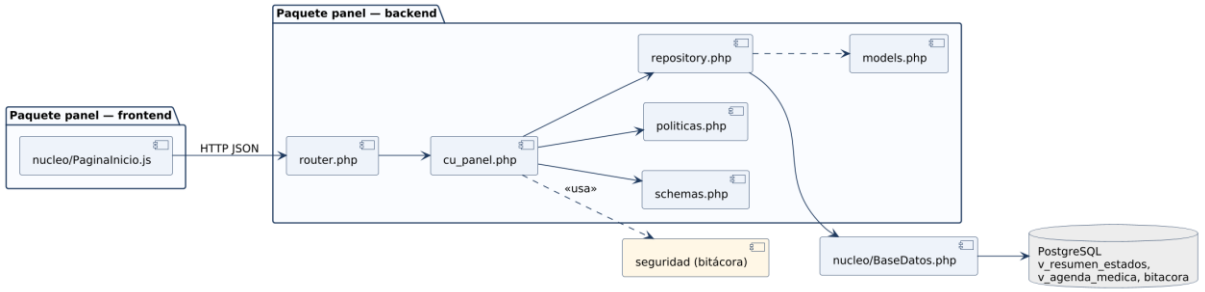


Figura 108. *Diagrama de componentes del paquete Panel principal*

Tabla 54. *Componentes de implementación por caso de uso*

CU	Pantalla (frontend)	Caso de uso (backend)	Ruta principal de la API
CU01	seguridad/CU01IniciarSesion.js	cu01_iniciar_sesion.php	POST /seguridad/sesion
CU02	seguridad/CU02GestionarUsuarios.js	cu02_gestionar_usuarios.php	/seguridad/usuarios
CU03	seguridad/CU03GestionarRolesPermisos.js	cu03_gestionar_roles_permisos.php	/seguridad/roles/{id}/permisos
CU04	personas/CU04GestionarPacientes.js	cu04_gestionar_pacientes.php	/personas/pacientes
CU05	personas/CU05GestionarMedicos.js	cu05_gestionar_medicos.php	/personas/medicos
CU06	personas/CU06GestionarEspecialidades.js	cu06_gestionar_especialidades.php	/personas/especialidades
CU07	agenda/CU07GestionarHorarios.js	cu07_gestionar_horarios.php	/agenda/medicos/{id}/horarios

CU	Pantalla (frontend)	Caso de uso (backend)	Ruta principal de la API
CU08	agenda/CU08ConsultarDisponibilidad.js + SelectorFichas.js	cu08_consultar_disponibilidad.php	GET /agenda/disponibilidad
CU09	citas/CU09ProgramarCita.js	cu09_programar_cita.php	POST /citas
CU10	citas/CU10ReprogramarCita.js	cu10_reprogramar_cita.php	PATCH /citas/{id}/reprogramar
CU11	citas/CU11CancelarCita.js	cu11_cancelar_cita.php	PATCH /citas/{id}/cancelar
CU12	citas/CU12RegistrarAsistencia.js	cu12_registrar_asistencia.php	PATCH /citas/{id}/estado
CU13	citas/CU13ConsultarAgendaMedica.js	cu13_consultar_agenda_medica.php	GET /citas/agenda
CU14	citas/CU14ConsultarHistorialPaciente.js	cu14_consultar_historial_paciente.php	GET /citas/pacientes/{id}/historial
CU15	reportes/CU15GenerarReportes.js	cu15_generar_reportes.php	GET /reportes/{tipo}
CU16	portal_paciente/CU16GestionarPerfilPaciente.js + CU16RegistroPaciente.js	cu16_gestionar_perfil_paciente.php	/portal/perfil, POST /portal/registro, POST /portal/cuentas/{id}
CU17	portal_paciente/CU17ProgramarCita.js + CU17MisCitas.js	cu17_reservar_mis_citas.php	/portal/citas, GET /portal/disponibilidad

CAPÍTULO 6: FLUJO DE TRABAJO: PRUEBAS

Las pruebas verifican que cada caso de uso se comporte según su especificación y que las reglas del negocio se cumplan tanto en la aplicación como en la base de datos. Las pruebas de la API se ejecutaron con peticiones HTTP sobre la base de datos con datos de prueba y de demostración (scripts 06 y 07); las pruebas de interfaz se hicieron en el navegador con los distintos roles. Las pruebas de la capa de datos se ejecutan con el script basededato/sql/pruebas/10_pruebas.sql.

6.1 Casos de prueba

Tabla 55. *Casos de prueba y resultados*

N.º	CU	Prueba	Entrada	Resultado esperado	Resultado obtenido
1	CU01	Iniciar sesión con contraseña incorrecta	admin / clave errónea	Mensaje genérico e intentos restantes	«Usuario o contraseña incorrectos. Le quedan 2 intentos». Aprobado
2	CU01	Tres intentos fallidos seguidos	Usuario existente	Bloqueo progresivo de 1, 2 y 3 minutos	Cuenta bloqueada y mensaje con los minutos. Aprobado
3	CU02	Filtrar usuarios por rol y estado	Rol Medico, estado Inactivo	Solo usuarios médicos inactivos	Se listan solo doccuenta y doccuenta2. Aprobado
4	CU04	Buscar paciente sin tilde	«andres»	Encuentra «Andrés»	Encuentra a Andrés Aguilera Cuéllar. Aprobado
5	CU05	Registrar médico con registro duplicado	MP-99999 repetido	Rechazo del duplicado	«El registro profesional ya está registrado». Aprobado
6	CU05	Registrar médico con formato inválido o sin especialidad	«12345» / sin especialidades	Rechazo con mensaje por campo	«Debe tener formato MP-XXXXX» y «Debe seleccionar al menos una especialidad». Aprobado
7	CU05	Registrar médico creando su cuenta	MP-55555, usuario doccuenta2	Médico vinculado a un usuario con rol Médico	id_usuario 8 vinculado, rol Medico. Aprobado
8	CU05	Desactivar médico con citas futuras	Dr. Juan Pérez (id 1)	Advertencia con la lista, sin desactivar	Advierte 23 citas futuras y sigue activo. Aprobado
9	CU05	Buscar médicos por especialidad	«pediatria»	Médicos de Pediatría	Ana, Gabriela y Patricia. Aprobado
10	CU07	Agregar bloque superpuesto	Viernes 10:00–13:00 sobre 09:00–12:00	Rechazo por superposición	«El horario se superpone con otro bloque del médico ese día». Aprobado
11	CU08	Consultar fichas de un día	Medicina general, lunes 02/11/2026	Fichas por médico y turno, ocupadas en gris	Fichas correctas; las citas canceladas liberan la ficha. Aprobado
12	CU09	Programar cita en ficha libre	Ficha 08:00–08:20	Cita registrada con comprobante	Comprobante CM-1422 generado. Aprobado
13	CU09	Programar en una ficha ya ocupada	Misma ficha, otro paciente	Rechazo por superposición (23P01)	HTTP 409 «El horario ya no está disponible». Aprobado
14	BD	Cancelar una cita sin motivo	UPDATE estado = Cancelada	Rechazo por ck_cancelacion	Error 23514 ck_cancelacion. Aprobado

N.º	CU	Prueba	Entrada	Resultado esperado	Resultado obtenido
15	BD	Script de pruebas de la base de datos	pruebas/10_pruebas.sql	Todas las pruebas pasan	Todas aprobadas. Aprobado
16	CU10	Reprogramar una cita a otra ficha libre	Cita CM-1413 (Rosa Áñez), nueva hora 15:40 del 12/11/2026	Cita actualizada, comprobante con fecha anterior y nueva, cambio en el historial	Comprobante correcto (12/11/2026, 15:40 - 16:00) y cambio «Reprogramada» en el historial. Aprobado
17	CU11	Cancelar una cita con y sin motivo	Cita de prueba del paciente 108 con el médico 11	Sin motivo: aviso en el formulario. Con motivo: estado Cancelada	Sin motivo no se envía; con «Cita de prueba» queda Cancelada (HTTP 200). Aprobado
18	CU12	Registrar asistencia según la fecha de la cita	Citas del día y citas futuras	«Atendida» e «Inasistencia» solo en citas cuya hora ya pasó	Las citas futuras solo permiten Confirmar o Cancelar. Aprobado
19	CU13	Consultar la agenda semanal	jperez (Médico) y recepcion1 → Odontología → Sergio Velasco	El médico ve solo su agenda; recepción elige especialidad y médico	jperez ve «Agenda de: Juan Pérez Gómez» sin selectores; recepción ve la cita del 12/11 a las 15:40. Aprobado
20	CU14	Historial de citas de un paciente	Rosa Áñez Rojas (CI 5115838); jperez busca «marco»	Resumen por estado, tabla de citas y cambios de cada cita; el médico solo ve sus pacientes	13 citas (8 atendidas, 1 cancelada, 2 inasistencias) y cambios de CM-1413; jperez solo ve sus pacientes. Aprobado
21	CU15	Generar, exportar e imprimir un reporte	Reporte por médicos del 25/10/2026 al 07/11/2026	Tarjetas con totales y % de asistencia, barras, CSV e impresión	Total 253, asistencia 89,2 %, barras por médico, CSV abierto en Excel con tildes e impresión sin menú. Aprobado
22	CU16	Crear cuenta del portal y autorregistro	Recepción: paciente CI 5931495. Sin sesión: CI 6042361, fecha 04/06/1965	Usuario con rol Paciente vinculado al paciente; ingreso al portal	Cuenta creada con contraseña temporal; autorregistro de «carlosaguilera» e ingreso a su portal. Aprobado
23	CU17	Programar, reprogramar y cancelar desde el portal	mlopez → Medicina general → Valeria Cuéllar → 13/11/2026	La cita aparece en «Mis citas»; al cancelarla pasa al historial	CM-1438 creada a las 09:00, reprogramada a las 10:40 y cancelada. Aprobado

6.2 Pruebas de la base de datos

El script pruebas/10_pruebas.sql ejecuta las reglas del negocio directamente sobre la base de datos: registro de citas válidas, rechazo de citas superpuestas (ex_cita_superpuesta), fuera de horario o en fechas pasadas (tg_validar_cita), transiciones de estado no permitidas (tg_transicion_estado), eliminación física de citas, pacientes y médicos (tg_no_borrar_*) y registro automático del historial (tg_historial_cita). Todas las pruebas se aprobaron con los datos de prueba (06) y también después de cargar los datos de demostración (07), que generan citas coherentes con los horarios y especialidades de cada médico.

CONCLUSIÓN

Se elaboró la documentación del Sistema Web de Gestión de Citas Médicas de la Clínica Vida Salud siguiendo los flujos de trabajo del PUDS: captura de requisitos, análisis, diseño, implementación y pruebas, en dos ciclos. El método de Ishikawa permitió identificar los problemas y sus causas —principalmente la ausencia de una fuente única de información sobre horarios y citas y la falta de reglas que impidan registrar citas inválidas— y el modelo de negocio permitió definir el proceso propuesto.

El análisis, el diseño y la implementación mantuvieron la misma división en paquetes, lo que permite seguir cada caso de uso desde su especificación hasta su pantalla y su clase en el código. La base de datos, normalizada hasta la tercera forma normal, protege las reglas críticas con restricciones y disparadores: la restricción de exclusión garantiza que un médico no tenga citas superpuestas incluso con usuarios simultáneos, y los triggers controlan las transiciones de estado y registran el historial de cada cita. El sistema quedó publicado en la nube (Render y Supabase) y puede usarse desde cualquier navegador.

RECOMENDACIÓN

- Conectar la aplicación con un usuario de base de datos propio (no el superusuario) con solo los permisos necesarios.
- Mantener la validación en el servidor y en la base de datos aunque exista validación con JavaScript, ya que esta última puede desactivarse en el navegador.
- Usar siempre HTTPS (ya activo en Render) y cambiar la contraseña inicial del administrador antes de usar el sistema con datos reales.
- Realizar copias de seguridad periódicas de la base de datos (pg_dump o copias de Supabase) y documentar la restauración.
- En siguientes versiones, incorporar recordatorios de citas por correo electrónico, la exportación de reportes a PDF y la gestión de feriados desde la interfaz.
- Capacitar a la recepcionista y a los médicos antes de la puesta en marcha y migrar los pacientes de las hojas de cálculo actuales verificando los CI duplicados.

BIBLIOGRAFÍA

Jacobson, I., Booch, G. y Rumbaugh, J. (2000). El Proceso Unificado de Desarrollo de Software. Madrid: Addison Wesley.

Booch, G., Rumbaugh, J. y Jacobson, I. (2006). El Lenguaje Unificado de Modelado. Guía del usuario (2.ª ed.). Madrid: Addison Wesley.

Torossi, G. (s.f.). El Proceso Unificado de Desarrollo de Software. Apunte de la cátedra Diseño de Sistemas.

Larman, C. (2003). UML y Patrones. Una introducción al análisis y diseño orientado a objetos (2.ª ed.). Madrid: Prentice Hall.

Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7.ª ed.). México: McGraw-Hill.

Elmasri, R. y Navathe, S. B. (2007). Fundamentos de Sistemas de Bases de Datos (5.ª ed.). Madrid: Pearson Addison Wesley.

Laudon, K. C. y Laudon, J. P. (2016). Sistemas de información gerencial (14.ª ed.). México: Pearson.

The PHP Group. Manual de PHP: PDO, password_hash() y password_verify(). https://www.php.net/manual/es/

The PostgreSQL Global Development Group. PostgreSQL 17 Documentation: Constraints, Exclusion Constraints, Triggers, PL/pgSQL. https://www.postgresql.org/docs/17/

OWASP Foundation (2021). OWASP Top 10. https://owasp.org/Top10/

Garzón, A. (2026). Diagrama de secuencia. Material de la materia Sistemas de Información I, UAGRM.

REPOSITORIO DEL CÓDIGO FUENTE

El código fuente organizado y comentado se publica en GitHub y la aplicación está desplegada en la nube:

Repositorio: https://github.com/JCLID3R/Clinica-VS

Aplicación en línea: https://clinica-vida-salud-02e8.onrender.com/

ANEXOS

Anexo A. Glosario

Tabla 56. *Glosario de términos*

Término	Definición
Cita	Reserva de un intervalo de atención de un médico para un paciente, en una fecha y hora.
Horario de atención	Bloque semanal (día, hora inicial y final) en el que un médico atiende, dividido en intervalos según la duración configurada.
Disponibilidad	Intervalos de un horario de atención que no están ocupados por citas no canceladas, ni bloqueados, ni en el pasado.
Inasistencia	Estado de una cita cuyo paciente no se presentó.
Reprogramación	Cambio de fecha y hora de una cita Programada o Confirmada.
Restricción de exclusión	Restricción de PostgreSQL que impide que dos filas cumplan simultáneamente una condición, como rangos de tiempo superpuestos.
Hash de contraseña	Transformación irreversible de la contraseña (password_hash) que se almacena en lugar de la contraseña original.